

Harte Echtzeit-Rechnersysteme

Vorhersagbare Scheduling-Algorithmen und Anwendungen

Stephan Epp

Basierend auf Giorgio C. Buttazzo: *Hard Real-Time Computing Systems*

2026

Inhaltsverzeichnis

Vorwort	2
1 Allgemeine Betrachtung	6
1.1 Einführung	6
1.2 Was bedeutet Echtzeit?	6
1.2.1 Der Begriff Zeit	6
1.2.2 Typen von Echtzeit-Tasks	7
1.3 Vorhersagbarkeit erreichen	7
1.3.1 Quellen der Nicht-Determinismus	7
2 Grundlegende Konzepte des Echtzeit-Schedulings	8
2.1 Einführung	8
2.2 Task-Modell und Zeitparameter	8
2.2.1 Aperiodische Tasks – Das Job-Modell	8
2.2.2 Periodische Tasks – Das τ -Modell	9
2.2.3 Sporadische und aperiodische Tasks	10
2.3 Klassifikation von Scheduling-Problemen	10
2.3.1 Graham-Notation	10
2.3.2 Gütekriterien	11
2.4 Klassifikation von Scheduling-Algorithmen	11
2.5 Vorrangrelationen und Ressourcenabhängigkeiten	12
2.6 Scheduling-Anomalien – Formale Analyse	12
2.7 Der Dhall-Effekt – EDF auf Mehrprozessoren	12
2.8 Vollständige Notation und Annahmen	13
3 Aperiodisches Task-Scheduling	14
3.1 Einführung	14
3.2 Jacksons Algorithmus (EDD)	14
3.3 Horns Algorithmus (EDF) – Scharf optimal	14
3.3.1 Das EDF-Optimalitätstheorem	14
3.3.2 Warum EDF scharf optimal ist	15
3.3.3 Garantiebedingung für EDF	16
3.4 Nicht-preemptives Scheduling	16
3.4.1 Bratley's Algorithmus	16
3.5 Scheduling mit Vorrangrelationen	16

4	Periodisches Task-Scheduling	17
4.1	Einführung	17
4.2	Notation und Annahmen	17
4.3	Timeline Scheduling	18
4.4	Rate Monotonic Scheduling – Detaillierte Analyse	18
4.4.1	Einführung und Grundidee	18
4.4.2	Kritischer Augenblick – Das Fundament der RM-Analyse	18
4.4.3	Vollständiger Optimalitätsbeweis von RM	19
4.4.4	Berechnung des Least Upper Bound für zwei Tasks	20
4.4.5	Berechnung des Least Upper Bound für n Tasks	21
4.4.6	Praktische Anwendung: Hinreichende Schedulierbarkeitstest	23
4.4.7	Hyperbolic Bound – Engere Schranke für RM	23
4.4.8	Response Time Analysis – Exakter Schedulierbarkeitstest	24
4.4.9	Schedulierbarkeit bei kritischem Augenblick – Direkte Methode	25
4.4.10	Workload-Analyse nach Lehoczky, Sha und Ding	26
4.4.11	Warum RM schwach optimal ist – Präzisierung	26
4.4.12	Vollständiges Beispiel: Vergleich RM und EDF bei hoher Auslastung	27
4.4.13	Übungsaufgaben zu Rate Monotonic	27
4.5	Earliest Deadline First (EDF) – Scharf optimal	28
4.5.1	Algorithmus	28
4.5.2	Schedulierbarkeitsanalyse für EDF	28
4.5.3	Die Schärfe von EDF im Vergleich zu RM	29
4.6	Deadline Monotonic Scheduling	29
4.7	EDF mit eingeschränkten Deadlines	29
4.8	Vergleich RM und EDF	29
5	Fixed-Priority Server – Aperiodische Tasks unter RM	31
5.1	Einführung und Problemstellung	31
5.2	Background Scheduling – Einfachster Ansatz	31
5.2.1	Informelle Beschreibung	31
5.2.2	Korrektheit und Optimalitätsklassifikation	32
5.3	Polling Server – Expliziter Server	32
5.3.1	Informelle Beschreibung	32
5.3.2	Schedulierbarkeitsnachweis	33
5.3.3	Worst-Case-Antwortzeit aperiodischer Anfragen	33
5.3.4	Optimalitätsklassifikation und Laufzeit	33
5.4	Deferrable Server – Kapazitätserhaltung	34
5.4.1	Informelle Beschreibung	34
5.4.2	Warum DS KEINE äquivalente periodische Task ist	34
5.4.3	Aperiodische Antwortzeit unter DS	35
5.5	Priority Exchange Server	35
5.5.1	Informelle Beschreibung	35
5.5.2	Schedulierbarkeitsnachweis	35
5.5.3	Optimalitätsklassifikation und Vergleich	36
5.6	Sporadic Server – Optimale Schedulierbarkeitsschranke unter RM	36
5.6.1	Informelle Beschreibung	36
5.6.2	Hauptsatz: SS verhält sich wie eine periodische Task	37
5.6.3	Schedulierbarkeitsanalyse	37

5.6.4	Optimalitätsklassifikation und Laufzeit	38
5.7	Slack Stealing – Maximale Responsivität	38
5.7.1	Informelle Beschreibung	38
5.7.2	Slack-Funktion	38
5.7.3	Nicht-Existenz optimaler Server – Fundamentales Ergebnis	39
5.8	Dimensionierung von Servern	39
5.9	Zusammenfassender Vergleich der Fixed-Priority Server	40
6	Dynamic Priority Server – Dynamische Prioritäts-Server unter EDF	41
6.1	Einführung und Motivation	41
6.2	Dynamic Priority Exchange Server (DPE)	41
6.2.1	Grundidee und Algorithmus	41
6.2.2	Schedulierbarkeitsnachweis – Korrektheit	42
6.2.3	Laufzeitanalyse	43
6.2.4	Optimalitätsklassifikation von DPE	43
6.3	Dynamic Sporadic Server (DSS)	43
6.3.1	Grundidee und Algorithmus	43
6.3.2	Schedulierbarkeitsnachweis	44
6.3.3	Laufzeitanalyse und Optimalitätsklassifikation	44
6.4	Total Bandwidth Server (TBS)	45
6.4.1	Grundidee und Algorithmus	45
6.4.2	Korrektheitsbeweis: Auslastungsbeschränkung	45
6.4.3	Optimalitätsklassifikation von TBS	46
6.5	Constant Bandwidth Server (CBS)	46
6.5.1	Definition und Motivation	46
6.5.2	CBS-Algorithmus (Pseudocode)	47
6.5.3	Isolationseigenschaft – formaler Beweis	47
6.5.4	Worst-Case-Antwortzeit und optimale CBS-Parameter	48
6.5.5	Vergleich der dynamischen Server	48
6.6	EDL-Server – Optimale aperiodische Antwortzeiten	48
6.7	Nicht-Existenz optimaler Server – Fundamentales Ergebnis	49
7	Ressourcenzugriffsprotokolle – Prioritätsinversion und Lösungsstrategien	50
7.1	Das Prioritätsinversionsproblem	50
7.2	Non-Preemptive Protocol (NPP)	51
7.2.1	Algorithmus	51
7.2.2	Blockierzeit unter NPP	51
7.3	Priority Inheritance Protocol (PIP)	51
7.3.1	Protokolldefinition	51
7.3.2	Korrektheit und Eigenschaften	52
7.3.3	Blockierzeitberechnung unter PIP	52
7.4	Priority Ceiling Protocol (PCP)	53
7.4.1	Protokolldefinition	53
7.4.2	Eigenschaften und Nachweis	53
7.4.3	Schedulierbarkeitsanalyse unter PCP	54
7.5	Stack Resource Policy (SRP)	54
7.5.1	Vergleich der Protokolle	55

8	Limited Preemptive Scheduling – Begrenzte Preemption	56
8.1	Motivation: Kosten und Nutzen der Preemption	56
8.2	Nicht-Preemptives Scheduling	56
8.2.1	Blockierzeit und Schedulierbarkeit	56
8.2.2	Machbarkeitsanalyse für nicht-preemptives Scheduling	57
8.3	Preemption Thresholds	57
8.3.1	Modell und Algorithmus	57
8.3.2	Machbarkeitsanalyse	58
8.3.3	Optimalitätsklassifikation	58
8.4	Deferred Preemptions – Aufgeschobene Preemption	58
8.4.1	Floating Model und Aktivierungsbasiertes Modell	58
8.4.2	Längste machbare nicht-preemptive Intervalle	59
8.4.3	Laufzeitanalyse	59
8.5	Task Splitting – Kooperatives Scheduling	59
8.5.1	Modell und optimale Preemptionspunkt-Auswahl	59
8.5.2	Laufzeitanalyse von SelectEPP	60
8.6	Vergleich der Ansätze	60
9	Überlastbehandlung – Scheduling unter Überlast	61
9.1	Einführung: Arten und Ursachen der Überlast	61
9.2	Aperiodische Überlast – Value-Based Scheduling	61
9.2.1	Algorithmus EARLIEST DEADLINE LATE (EDL) / RED	62
9.3	Überlastbehandlung unter RM vs. EDF	62
9.3.1	Permanente Überlast unter RM	62
9.3.2	Permanente Überlast unter EDF – Cervin-Theorem	62
9.4	Elastic Scheduling – Dynamische Periodeneanpassung	63
9.5	(π, λ) -Algorithmen für harte Überlast	63
10	Der EDF⁺ Algorithmus – Optimales Scheduling mit Blocking-Penalty	64
10.1	Motivation und Einordnung	64
10.2	Grundlegende Definitionen	64
10.3	Der EDF ⁺ -Algorithmus	65
10.3.1	Algorithmus-Beschreibung	65
10.3.2	Pseudocode	66
10.4	Optimalitätsbeweis für EDF ⁺	66
10.4.1	Fundament: EDF-Optimalitätstheorem	66
10.4.2	Hauptsatz: Optimalität von EDF ⁺	66
10.4.3	Korollare	67
10.5	Laufzeitanalyse	67
10.6	EDF ⁺ für dynamische Task-Mengen	68
10.7	Integration in AUTOSAR und ISO 26262	68
11	Kernel-Design-Aspekte für Echtzeit-Systeme	69
11.1	Struktur eines Echtzeit-Kernels	69
11.1.1	Prozesszustände in einem Echtzeit-Kernel	69
11.1.2	Warteschlangen-Implementierung	69
11.2	Interrupt-Behandlung in Echtzeit-Systemen	70
11.3	Speicherverwaltung	70
11.4	Kernel-Overhead und Schedulierbarkeitsanalyse	70

12 Anwendungsdesign für Echtzeit-Systeme	71
12.1 Zeitschränkenableitung aus Anwendungsanforderungen	71
12.2 Hierarchisches Entwurfsmuster	71
12.3 Robotersteuerungsbeispiel – Vollständige Analyse	72
12.4 AUTOSAR-Anwendungsbeispiel	72
13 Zusammenfassung, Schlussfolgerungen und Ausblick	73
13.1 Kernaussagen dieses Buches	73
13.2 Schwache vs. Scharfe Optimalität – Gesamtüberblick	73
13.3 EDF ⁺ – Erweiterung der Schärfe auf Blocking-Szenarien	73
13.4 Praktische Empfehlungen	73
13.5 Ausblick: Offene Forschungsfragen und Zukünftige Entwicklungen	74
13.5.1 Mehrprozessor-Scheduling – Das größte offene Problem	74
13.5.2 Mixed-Criticality Scheduling	75
13.5.3 Maschinelles Lernen im Echtzeit-Scheduling	75
13.5.4 Cyber-Physical Systems und das Jitter-Problem	76
13.5.5 Formale Verifikation von Scheduling-Algorithmen	76
13.5.6 Der Subgraph Algorithmus im verteilten Echtzeit-Kontext	77
13.5.7 Hardware-Beschleunigung für Scheduling	77
13.5.8 Sicherheitszertifizierung – ISO 26262 und POSIX	77
13.5.9 Zusammenfassung der Forschungsrichtungen	78
13.5.10 Schlusswort	78

Vorwort

Motivation und Zielsetzung

Dieses Buch behandelt harte Echtzeit-Rechnersysteme aus der Perspektive vorhersagbarer Scheduling-Algorithmen. Es verfolgt ein doppeltes Ziel: einerseits die präzise Darstellung der klassischen Theorie, die seit dem Grundlagenaufsatz von Liu und Layland [6] im Jahr 1973 entstanden ist, andererseits die Integration neuerer Ergebnisse – insbesondere des hier erstmals ausführlich präsentierten EDF⁺-Algorithmus [40] – in einen einheitlichen theoretischen Rahmen.

Die Hauptgrundlage ist das international maßgebliche Lehrbuch von Giorgio C. Buttazzo [1], das seit seiner ersten Auflage 1997 als Standardreferenz der Echtzeit-Scheduling-Theorie gilt. Es wird ergänzt durch eine Vielzahl von Primärquellen, darunter die Originalarbeiten von Horn [7], Dertouzos [8], Jackson [9], Lehoczky, Sha und Ding [14] sowie die einflussreichen Beiträge zur Ressourcenverwaltung von Sha, Rajkumar und Lehoczky [28] und Baker [29].

Der fundamentale Kernsatz

Der rote Faden dieses Buches ist eine Aussage, die sich aus dem Vergleich zweier grundlegend verschiedener Scheduling-Philosophien ergibt – der statischen Prioritätszuweisung (RM) und der dynamischen Prioritätszuweisung (EDF):

**Wenn es einen machbaren Schedule gibt, findet RM diesen sofort.
RM ist schwach optimal.**

**Wenn es einen machbaren Schedule gibt, findet EDF diesen sofort.
Die kürzeste Ausführungszeit aller Tasks findet EDF sofort. EDF ist
scharf optimal.**

Diese Aussage ist nicht trivial. Sie unterscheidet zwei fundamentale Konzepte der Optimalität: *schwache* Optimalität, die RM innerhalb der Klasse fester Prioritätsalgorithmen auszeichnet, und *scharfe* Optimalität, die EDF über alle präemptiven Einprozessor-Scheduling-Algorithmen hinweg kennzeichnet. Die Schärfe von EDF – im Sinne von: “genau die schedulebaren Task-Sets werden gefunden, alle anderen nicht” – ist das stärkste Optimalitätsergebnis, das für Einprozessor-Echtzeit-Scheduling bekannt ist.

Der Beweis der scharfen Optimalität von EDF, der auf Arbeiten von Horn [7] und Dertouzos [8] zurückgeht und später durch Baruah et al. [19] für periodische Tasks präzisiert wurde, ist mathematisch elegant und bildet das Herzstück von Kapitel 3 dieses Buches.

Historischer Kontext

Die Echtzeit-Scheduling-Theorie entstand in den frühen 1970er Jahren als Antwort auf die wachsende Verbreitung von Prozesssteuerungssystemen, die zuverlässige zeitliche Garantien erforderten. Die Pionierarbeiten von Liu und Layland [6] legten 1973 mit dem Rate-Monotonic-Algorithmus das Fundament. Fast gleichzeitig bewies Dertouzos [8] die Optimalität von EDF für aperiodische Tasks.

In den 1980er und 1990er Jahren wurde dieses Fundament durch eine Reihe wesentlicher Beiträge erweitert:

- Lehoczky, Sha und Ding [14] lieferten eine exakte Charakterisierung der RM-Schedulierbarkeit.
- Sha, Rajkumar und Lehoczky [28] lösten das Prioritätsinversionsproblem durch das Priority Inheritance Protocol.
- Sprunt, Sha und Lehoczky [22] entwickelten den Sporadic Server, der die Lücke zwischen periodischem und aperiodischem Scheduling überbrückt.
- Audsley et al. [16] etablierten die Response Time Analysis als praktisches Werkzeug für feste Prioritätsschedules.
- Spuri und Buttazzo [25] übertrugen Server-Konzepte auf EDF und entwickelten Total Bandwidth Server sowie Dynamic Sporadic Server.
- Abeni und Buttazzo [26] entwickelten den Constant Bandwidth Server als Grundlage für ressourcenisoliertes Scheduling.

Im neuen Jahrtausend rückte das Mixed-Criticality-Problem (Vestal [37], Baruah et al. [38]) und das Mehrprozessor-Scheduling (Baruah et al. [35]) in den Vordergrund. Beide sind bis heute aktive Forschungsfelder.

Über den EDF⁺-Algorithmus

Eine wichtige Erweiterung der klassischen EDF-Theorie ist der EDF⁺-Algorithmus [40], der in Kapitel 10 ausführlich behandelt wird. EDF⁺ überträgt die scharfe Optimalität von EDF auf Systeme mit stochastischen Blocking-Ereignissen: Wenn Tasks mit Wahrscheinlichkeit p blockieren (etwa durch I/O-Warten oder Semaphore), erzeugt EDF⁺ bei jedem Blocking-Ereignis sofort einen neuen optimalen EDF-Schedule für die verbleibenden nicht-blockierten Tasks.

Der Beweis der Optimalität von EDF⁺ beruht direkt auf dem EDF-Optimalitätstheorem und ist konstruktiv: Durch vollständige Induktion über die Anzahl der Blocking-Ereignisse wird gezeigt, dass EDF⁺ stets einen Deadline-freien Schedule liefert, sofern die Systemauslastung $U \leq 1$ beträgt. Dies macht EDF⁺ zu einem Algorithmus mit nachweisbarer formaler Korrektheit unter realistischen Systemannahmen.

Struktur des Buches

Das Buch ist in 13 Kapitel gegliedert, die aufeinander aufbauen:

Kapitel 1 – Allgemeine Betrachtung: Grundlegende Eigenschaften von Echtzeit-Systemen, Typen von Echtzeit-Tasks, Quellen des Nicht-Determinismus und die zentrale Anforderung der Vorhersagbarkeit nach Stankovic [21].

Kapitel 2 – Grundlegende Konzepte: Vollständiges Task-Modell mit allen Zeitparametern (Lateness, Laxität, Slack), Klassifikation von Scheduling-Problemen nach der Graham-Notation $\alpha|\beta|\gamma$, Gütekriterien, Scheduling-Anomalien und der Dhall-Effekt auf Mehrprozessoren.

Kapitel 3 – Aperiodisches Task-Scheduling: Jacksons EDD-Algorithmus [9], vollständiger Dertouzos-Beweis der EDF-Optimalität [8], Bratley’s Branch-and-Bound für nicht-preemptives Scheduling [11] und Lawlers LDF-Algorithmus [10] für Tasks mit Vorrangrelationen.

Kapitel 4 – Periodisches Task-Scheduling: Vollständige Analyse von Rate Monotonic (Liu & Layland [6]) einschließlich aller Beweise, Hyperbolic Bound [17], Response Time Analysis [16], EDF-Schedulierbarkeitstheorem [19] sowie der präzise Vergleich zwischen schwacher (RM) und scharfer (EDF) Optimalität.

Kapitel 5 – Fixed-Priority Server: Vollständige Analyse von Background Scheduling, Polling Server, Deferrable Server (Strosnider et al. [23]), Priority Exchange, Sporadic Server [22] und Slack Stealing [24] mit allen Beweisen und Laufzeitanalysen.

Kapitel 6 – Dynamic Priority Server: EDF-basierte Server – DPE, DSS, TBS, CBS [26] und EDL-Server – mit Optimalitätsnachweisen und dem fundamentalen Nicht-Existenz-Ergebnis für optimale Server [27].

Kapitel 7 – Ressourcenzugriffsprotokolle: Prioritätsinversion, NPP, PIP und PCP [28], Stack Resource Policy [29], vollständige Beweise der Blockierzeitschranken.

Kapitel 8 – Limited Preemptive Scheduling: Nicht-preemptives Scheduling, Preemption Thresholds [31], Deferred Preemptions und Task Splitting mit dem optimalen SelectEPP-Algorithmus [32].

Kapitel 9 – Überlastbehandlung: Value-Based Scheduling, Cervin’s Theorem über EDF bei permanenter Überlast [33], Elastic Scheduling und Admission Control.

Kapitel 10 – EDF⁺-Algorithmus: Vollständiger Optimalitätsbeweis, Laufzeitanalyse, Korollare (Dominanz über RM, Minimax-Eigenschaft) und Integration in AUTOSAR/ISO 26262 [40].

Kapitel 11 – Kernel-Design: Interrupt-Behandlung, Speicherverwaltung, Schedulingwarteschlangen und Kernel-Overhead in der Schedulierbarkeitsanalyse.

Kapitel 12 – Anwendungsdesign: Hierarchische Dekomposition, Robotersteuerungsbeispiel, AUTOSAR-Konfiguration.

Kapitel 13 – Zusammenfassung und Ausblick: Gesamtüberblick, praktische Empfehlungen und ein sehr ausführlicher Ausblick auf offene Forschungsfragen – Mehrprozessor-Scheduling, Mixed-Criticality, maschinelles Lernen, formale Verifikation, Hardware-Beschleunigung und autonome Systeme.

Voraussetzungen und Zielgruppe

Das Buch richtet sich an Studierende der Informatik und Elektrotechnik ab dem dritten Semester sowie an Praktiker in der Embedded-Systems-Entwicklung. Vorausgesetzt werden grundlegende Kenntnisse der Algorithmentheorie (O-Notation, Induktionsbeweise) und der Betriebssystemkonzepte (Prozesse, Semaphore, Scheduling). Für ein vollständiges Verständnis der Kapitel über Ressourcenprotokolle und CBS sind Grundkenntnisse über Nebenläufigkeit hilfreich.

Mathematisch werden Kenntnisse der Analysis (Grenzwerte, Ableitungen) benötigt, die für das Verständnis des Liu-Layland-Beweises (Kapitel 4) und des Cervin-Theorems (Kapitel 9) notwendig sind.

Danksagung

Der Autor dankt den Begründern der Echtzeit-Scheduling-Theorie – insbesondere C. L. Liu, J. W. Layland, Giorgio C. Buttazzo, Lui Sha und Sanjoy Baruah – für ihre grundlegenden Beiträge, ohne die dieses Buch nicht möglich wäre. Besonderer Dank gilt der für die wissenschaftliche Infrastruktur und Unterstützung.

Bielefeld, Juni 2026

Stephan Epp

Kapitel 1

Allgemeine Betrachtung

1.1 Einführung

Echtzeit-Systeme sind Rechnersysteme, die innerhalb präziser Zeitschranken auf Ereignisse in ihrer Umgebung reagieren müssen. Die Korrektheit solcher Systeme hängt nicht nur vom Wert des Ergebnisses, sondern auch vom Zeitpunkt ab, zu dem das Ergebnis erzeugt wird [21]. Eine Reaktion, die zu spät erfolgt, kann wertlos oder sogar gefährlich sein. Echtzeit-Computing spielt heute eine entscheidende Rolle in unserer Gesellschaft, da eine wachsende Anzahl komplexer Systeme vollständig oder teilweise auf Computersteuerung angewiesen ist.

Anwendungsbeispiele umfassen:

- Steuerung chemischer und nuklearer Anlagen
- Steuerung komplexer Produktionsprozesse
- Eisenbahn-Weichensysteme
- Automotive-Anwendungen (AUTOSAR, ISO 26262)
- Flugregelungssysteme
- Umwelterfassung und -überwachung
- Telekommunikationssysteme
- Medizinische Systeme
- Industrieautomation und Robotik
- Militärsysteme und Raumfahrtmissionen

1.2 Was bedeutet Echtzeit?

1.2.1 Der Begriff Zeit

Das Hauptmerkmal, das Echtzeit-Computing von anderen Arten der Berechnung unterscheidet, ist die Zeit. Das Wort *Zeit* bedeutet, dass die Korrektheit des Systems nicht nur

vom logischen Ergebnis der Berechnung, sondern auch vom Zeitpunkt abhängt, zu dem die Ergebnisse produziert werden.

Das Wort *real* zeigt an, dass die Reaktion der Systeme auf externe Ereignisse während deren Ablauf erfolgen muss. Folglich muss die Systemzeit (interne Zeit) mit derselben Zeitskala gemessen werden, die zur Messung der Zeit in der kontrollierten Umgebung (externe Zeit) verwendet wird.

Definition 1.1 (Echtzeit-System). Ein **Echtzeit-System** ist ein Rechnersystem, das innerhalb präziser, vorher festgelegter Zeitgrenzen auf Ereignisse in seiner Umgebung reagieren muss. Die Korrektheit hängt sowohl vom Wert als auch vom Zeitpunkt der erzeugten Ergebnisse ab.

1.2.2 Typen von Echtzeit-Tasks

Abhängig von den Konsequenzen einer versäumten Deadline unterscheidet man:

Definition 1.2 (Harte Echtzeit-Task). Eine Echtzeit-Task heißt **hart**, wenn die Erzeugung von Ergebnissen nach ihrer Deadline katastrophale Folgen für das System unter Kontrolle haben kann.

Definition 1.3 (Weiche Echtzeit-Task). Eine Echtzeit-Task heißt **weich**, wenn die Erzeugung von Ergebnissen nach ihrer Deadline zwar eine Leistungsver schlechterung verursacht, aber keinen Schaden.

1.3 Vorhersagbarkeit erreichen

Eine der wichtigsten Eigenschaften, die ein hartes Echtzeit-System haben muss, ist Vorhersagbarkeit [21]. Das System soll basierend auf den Kernel-Merkmalen und den Informationen jeder Task die Entwicklung der Tasks vorhersagen und im Voraus garantieren können, dass alle kritischen Zeitbedingungen eingehalten werden.

1.3.1 Quellen der Nicht-Determinismus

Die Vorhersagbarkeit eines Systems wird durch mehrere Faktoren beeinflusst:

- **DMA**: Der Prozessor muss warten, wenn ein DMA-Gerät einen Speicherzyklus stiehlt. Die Anzahl solcher Wartezeiten ist nicht vorhersagbar.
- **Cache-Speicher**: Cache-Fehlzugriffe (Cache Misses) führen zu variablen Zugriffszeiten. Präemptionen verschlechtern die Cache-Lokalität erheblich.
- **Interrupts**: Können unbeschränkte Verzögerungen in die Task-Ausführung einführen, wenn sie nicht ordnungsgemäß behandelt werden.
- **Systemaufrufe**: Unbeschränkte Ausführungszeiten von Kernelprimitiven verhindern präzise WCET-Abschätzungen.
- **Semaphore und Prioritätsinversion**: Klassische Semaphore können hochprioritäre Tasks durch niederprioritäre Tasks für unbeschränkte Zeit blockieren.
- **Speicherverwaltung**: Demand-Paging verursacht große und unvorhersagbare Verzögerungen durch Seitenfehler.

Kapitel 2

Grundlegende Konzepte des Echtzeit-Schedulings

2.1 Einführung

Dieses Kapitel legt das vollständige begriffliche Fundament für alle Scheduling-Algorithmen der folgenden Kapitel. Es definiert präzise die Parameter von Tasks, die verschiedenen Typen von Zeitschranken, die formale Klassifikation von Scheduling-Problemen nach Graham et al. sowie die grundlegenden Gütekriterien. Besonderes Gewicht liegt auf den Konzepten der *Lateness*, *Laxity* und *Slack*, die die Basis für die Optimalitätsbeweise in Kapitel 3 und Kapitel 4 bilden.

2.2 Task-Modell und Zeitparameter

2.2.1 Aperiodische Tasks – Das Job-Modell

Definition 2.1 (Job (Aperiodische Task)). Ein **Job** J_i ist eine atomare Ausführungseinheit mit folgenden Zeitparametern:

- a_i (Arrival Time, Ankunftszeit): Zeitpunkt, ab dem J_i zur Ausführung bereit ist. Auch: *release time* r_i .
- C_i (Worst-Case Execution Time, WCET): Maximale Ausführungszeit unter allen möglichen Eingaben. Unveränderlich für jede Instanz.
- d_i (Absolute Deadline): Spätester Fertigstellungszeitpunkt.
- D_i (Relative Deadline): $D_i = d_i - a_i$. Zeitdauer ab Ankunft bis zur Deadline.
- f_i (Finishing Time, Fertigstellungszeit): Tatsächlicher Abschluss der Ausführung im konkreten Schedule.
- s_i (Start Time, Startzeitpunkt): Erster Zeitpunkt, zu dem J_i tatsächlich ausgeführt wird.

Definition 2.2 (Abgeleitete Zeitgrößen). Aus den Grundparametern werden folgende Größen abgeleitet:

$$\begin{aligned}
R_i &= f_i - a_i && \text{(Response Time, Antwortzeit)} \\
L_i &= f_i - d_i && \text{(Lateness: positiv = zu spät, negativ = Puffer)} \\
E_i &= \max(0, f_i - d_i) && \text{(Tardiness: Verspätung, nie negativ)} \\
X_i &= d_i - a_i - C_i && \text{(Laxity, Spielraum: maximale Wartezeit)} \\
\text{slack}_i(t) &= d_i - t - c_i(t) && \text{(Slack zur Zeit } t, \text{ mit verbleib. Rechenzeit } c_i(t))
\end{aligned}$$

Bemerkung 2.1 (Zusammenhänge der Zeitgrößen). Die Laxität $X_i = D_i - C_i$ gibt an, wie lange J_i insgesamt warten darf, ohne ihre Deadline zu verpassen. Der Slack $\text{slack}_i(t)$ ist die verbleibende Wartekapazität zum Zeitpunkt t :

- $\text{slack}_i(t) > 0$: J_i kann noch warten.
- $\text{slack}_i(t) = 0$: J_i muss *sofort* starten, sonst Deadline-Verletzung.
- $\text{slack}_i(t) < 0$: Deadline bereits verpasst (bei Betrieb oder bei unrealisierbaren Parametern).

Beispiel 2.1 (Zeitparameter). J_1 : $a_1 = 2$, $C_1 = 3$, $d_1 = 8$.

- $D_1 = 8 - 2 = 6$, $X_1 = 6 - 3 = 3$ (Laxität: darf 3 Einheiten warten)
- Wenn $s_1 = 4$, $f_1 = 7$: $R_1 = 5$, $L_1 = -1$ (1 Einheit Puffer), $E_1 = 0$
- Wenn $s_1 = 6$, $f_1 = 9$: $R_1 = 7$, $L_1 = 1$ (1 Einheit zu spät), $E_1 = 1$
- $\text{slack}_1(4) = 8 - 4 - 3 = 1$ (kann noch 1 Einheit warten)
- $\text{slack}_1(5) = 8 - 5 - 3 = 0$ (muss sofort starten)

2.2.2 Periodische Tasks – Das τ -Modell

Definition 2.3 (Periodische Task). Eine **periodische Task** τ_i erzeugt in regelmäßigen Abständen Jobs. Sie ist durch folgende Parameter vollständig beschrieben:

- Φ_i : Phase (Ankunftszeit der ersten Instanz)
- T_i : Periode (Abstand zwischen aufeinanderfolgenden Instanzen)
- C_i : WCET jeder Instanz (konstant)
- D_i : Relative Deadline ($D_i \leq T_i$ im Allgemeinen; $D_i = T_i$ unter Annahme A3)

Die k -te Instanz $\tau_{i,k}$ hat:

$$r_{i,k} = \Phi_i + (k-1)T_i, \quad d_{i,k} = r_{i,k} + D_i = \Phi_i + (k-1)T_i + D_i.$$

Definition 2.4 (Hyperperiode). Die **Hyperperiode** H ist das kleinste gemeinsame Vielfache aller Perioden: $H = \text{lcm}(T_1, \dots, T_n)$. Nach der Hyperperiode wiederholt sich der Schedule exakt (bei synchronen Aktivierungen und $D_i = T_i$).

Definition 2.5 (Prozessorauslastung). Die **Prozessorauslastung** U einer Menge von n periodischen Tasks ist:

$$U = \sum_{i=1}^n \frac{C_i}{T_i}.$$

U gibt den Anteil der Prozessorzeit an, der für die periodischen Tasks *im Durchschnitt* benötigt wird. Wenn $U > 1$, ist kein machbarer Schedule unter irgendeinem Algorithmus möglich.

Definition 2.6 (Synchroner vs. asynchroner Task-Set). – **Synchron:** Alle Tasks starten gleichzeitig bei $t = 0$: $\Phi_i = 0$ für alle i .

– **Asynchron:** Tasks haben unterschiedliche Phasen Φ_i .

Die kritische Analyse erfolgt stets für synchrone Aktivierungen (kritischer Augenblick – schlimmster Fall für RM, vgl. Kapitel 4).

2.2.3 Sporadische und aperiodische Tasks

Definition 2.7 (Sporadische Task). Eine **sporadische Task** ist eine Task mit unregelmäßigen Aktivierungen, jedoch mit einer Mindestankunftsabstand (Minimum Interarrival Time) $T_i^{\min}$: $a_{i,k+1} - a_{i,k} \geq T_i^{\min}$. Sporadische Tasks können offline garantiert werden, indem man $T_i^{\min}$ als Worst-Case-Periode verwendet.

Definition 2.8 (Firme aperiodische Task). Eine **firm**e aperiodische Task hat eine Deadline. Sie wird online garantiert: Beim Eintreffen prüft ein Admission-Controller, ob sie ohne Deadline-Verletzung anderer Tasks integriert werden kann.

2.3 Klassifikation von Scheduling-Problemen

2.3.1 Graham-Notation

Definition 2.9 (Graham-Notation $\alpha|\beta|\gamma$). Jedes Scheduling-Problem wird durch drei Felder beschrieben:

- α : **Maschinenmodell** – z. B. 1 (Einzelprozessor), P_m (m identische Prozessoren), Q_m (uniforme Geschwindigkeiten), R_m (unverwandte Prozessoren).
- β : **Task-Constraints** – z. B. **preem** (preemptiv), **no-preem** (nicht-preemptiv), **prec** (Vorrangrelationen), **sync** (synchrone Ankunft), $D_i = T_i$ (Deadline gleich Periode).
- γ : **Gütekriterium** – z. B. $L_{\max}$ (maximale Lateness), $\sum L_i$ (Summe der Latenesses), $E_{\max}$ (maximale Tardiness), **feasible** (machbarer Schedule).

Beispiel 2.2 (Scheduling-Notationen der behandelten Probleme). – $1 \mid \text{sync} \mid L_{\max}$: Einzelprozessor, synchrone Ankunft, minimiere maximale Lateness $\rightarrow$ **EDD/Jackson** (Kap. 3)

– $1 \mid \text{preem} \mid L_{\max}$: Einzelprozessor, preemptiv, beliebige Ankunftszeiten, minimiere $L_{\max} \rightarrow$ **EDF/Horn** (Kap. 3)

– $1 \mid \text{preem}, D_i = T_i \mid \text{feasible}$: Periodisch, preemptiv $\rightarrow$ **RM, EDF** (Kap. 4)

– $1 \mid \text{no-preem} \mid \text{feasible}$: Nicht-preemptiv, NP-schwer $\rightarrow$ **Bratley** (Kap. 3)

2.3.2 Gütekriterien

Definition 2.10 (Gütekriterien im Überblick). – $L_{\max} = \max_i L_i$: **Maximale Lateness**. Negativ $\Rightarrow$ alle Deadlines eingehalten. Optimalitätsmaß für EDF und EDD.

- $E_{\max} = \max_i E_i$: **Maximale Tardiness**.
- $\sum L_i$: **Summe der Latenesses**. Equivalent zur Minimierung der Gesamtfertigungszeit ($\sum f_i$).
- $n_{\text{late}} = |\{J_i \mid f_i > d_i\}|$: **Anzahl verspäteter Tasks**. NP-schwer für beliebige Ankunftszeiten.
- **Feasibility**: Existenz eines Schedules mit $L_{\max} \leq 0$.

Bemerkung 2.2 (Zusammenhang Gütekriterien). $L_{\max} \leq 0 \iff$ Schedule ist machbar $\iff E_{\max} = 0$. Ein Algorithmus, der $L_{\max}$ minimiert, findet automatisch einen machbaren Schedule, wenn einer existiert. Deshalb ist die Minimierung von $L_{\max}$ das stärkere und nützlichere Kriterium gegenüber bloßer Machbarkeit.

2.4 Klassifikation von Scheduling-Algorithmen

Definition 2.11 (Taxonomie der Scheduling-Algorithmen). – **Preemptiv vs. Nicht-Preemptiv**: Darf eine laufende Task unterbrochen werden?

- **Statisch vs. Dynamisch**: Werden Prioritäten vor oder während der Ausführung berechnet?
- **Online vs. Offline**: Sind alle Task-Parameter von Anfang an bekannt (offline) oder werden Tasks dynamisch aktiviert (online)?
- **Optimal vs. Heuristisch**: Findet der Algorithmus stets die beste Lösung, oder nur eine gute Näherung?
- **Clairvoyant vs. Non-Clairvoyant**: Hat der Algorithmus vollständige Kenntnis aller zukünftigen Ankunftszeiten?

Definition 2.12 (Optimalitätsbegriffe – Zusammenfassung). – **Schwach optimal** (innerhalb einer Klasse): Findet den besten Schedule unter allen Algorithmen derselben Klasse. Beispiel: RM ist schwach optimal unter festen Prioritäten.

- **Scharf optimal** (global): Findet jeden machbaren Schedule, der überhaupt existiert. Beispiel: EDF ist scharf optimal für preemptive Einzelprozessoren.
- **Optimal bezüglich γ** : Minimiert das Gütekriterium γ unter allen möglichen Schedules.

2.5 Vorrangrelationen und Ressourcenabhängigkeiten

Definition 2.13 (Vorranggraph (Precedence Graph)). Ein **Vorranggraph** $G = (J, \prec)$ ist ein gerichteter azyklischer Graph (DAG), in dem $J_i \prec J_j$ bedeutet: J_j darf erst starten, wenn J_i vollständig abgeschlossen ist. Notationen:

- $J_i \rightarrow J_j$: J_i ist unmittelbarer Vorgänger von J_j
- $J_i \prec J_j$: J_i ist (direkter oder indirekter) Vorgänger

Definition 2.14 (Ressourcenkonflikt). Zwei Tasks J_i und J_j stehen in einem **Ressourcenkonflikt**, wenn sie dieselbe exklusive Ressource R_k benötigen. Ressourcenkonflikte führen zu Blockierung und sind der Hauptgrund für Prioritätsinversion (Kapitel 7).

2.6 Scheduling-Anomalien – Formale Analyse

Definition 2.15 (Scheduling-Anomalie nach Graham). Eine **Scheduling-Anomalie** tritt auf, wenn eine Verbesserung der Eingabeparameter – weniger Tasks, kürzere WCETs, schwächere Vorrangrelationen, mehr Prozessoren – zu einem *schlechteren* Schedule-Ergebnis (z. B. längerer Makespan) führt.

Beispiel 2.3 (Graham-Anomalie auf Mehrprozessorsystemen). Betrachte 6 Tasks $J_1, \dots, J_6$ mit WCETs $C_i = 3$ und Vorrangrelation $J_1 \prec J_3, J_1 \prec J_4, J_2 \prec J_5, J_2 \prec J_6$ auf 3 Prozessoren. List Scheduling (LPT) erzeugt Makespan 3. Wenn die Vorrangrelation entfernt wird, kann List Scheduling Makespan 4 produzieren – trotz weniger Einschränkungen. Graham (1969) bewies: Solche Anomalien sind bei allgemeinen Mehrprozessorsystemen unter List Scheduling unvermeidbar.

Beispiel 2.4 (Anomalie bei Selbstsuspension). Task τ_1 mit $C_1 = 1, T_1 = 4$, Deadline $d_1 = 4$ hat Slack 3. Führt τ_1 eine Selbstsuspension von 2 Einheiten aus, verpasst sie ihre Deadline. Obwohl τ_1 selbst weniger Rechenzeit benötigt (wegen Suspension), wird das System schlechter – ein direktes Beispiel für eine Anomalie bei Selbstsuspensionen unter RM.

Bemerkung 2.3 (Anomalien und Konsequenzen für den Systementwurf). Scheduling-Anomalien haben direkte praktische Konsequenzen:

1. **WCET-Erhöhung** einer Task kann durch zusätzliche Preemptionen zu Deadline-Verletzungen führen, obwohl das System ursprünglich schedulierbar war.
2. **Selbstsuspensionen** (z. B. für I/O) müssen explizit in der Schedulierbarkeitsanalyse berücksichtigt werden.
3. **Mehrprozessorsysteme**: Globale Scheduling-Anomalien (Dhall-Effekt) zeigen, dass EDF auf Mehrprozessoren nicht mehr optimal ist.

2.7 Der Dhall-Effekt – EDF auf Mehrprozessoren

Theorem 2.1 (Dhall-Effekt, 1978). EDF ist auf Mehrprozessorsystemen *nicht optimal*: Es gibt Task-Sets mit Auslastung beliebig nah an 1, die EDF auf m Prozessoren nicht schedulieren kann.

Beweisidee. Betrachte $m + 1$ Tasks mit $C_i = \varepsilon$, $T_i = \varepsilon + \delta$ für $i = 1, \dots, m$ (kurze Tasks, frühe Deadlines) und $C_{m+1} = 1$, $T_{m+1} = 1 + \varepsilon$ (lange Task, spätere Deadline). Auslastung: $U = m \cdot \varepsilon / (\varepsilon + \delta) + 1 / (1 + \varepsilon) \rightarrow 1$ für kleine ε, δ . EDF belegt alle m Prozessoren mit den m kurzperiodischen Tasks, da diese immer die früheste Deadline haben, und lässt die lange Task verhungern. ■ □

Bemerkung 2.4. Der Dhall-Effekt zeigt, dass der Wechsel von Uniprocessor- zu Multiprocessor-Scheduling grundlegend neue Algorithmen erfordert: Partitioned Scheduling, Global EDF mit Deadline-Adjustment (z. B. GLLF, Pfair), oder Clustered Scheduling. Diese werden in diesem Buch nicht behandelt, sind aber aktives Forschungsgebiet.

2.8 Vollständige Notation und Annahmen

Zur Vereinheitlichung gelten in diesem Buch folgende Standardannahmen, sofern nicht ausdrücklich anders angegeben:

- A1** Alle periodischen Task-Instanzen haben dieselbe WCET C_i .
- A2** Die Perioden T_i sind konstant.
- A3** Relative Deadlines sind gleich den Perioden: $D_i = T_i$.
- A4** Tasks sind unabhängig (keine Vorrangrelationen, keine gemeinsamen Ressourcen).
- A5** Tasks suspendieren sich nicht (kein I/O -Warten).
- A6** Tasks werden sofort bei ihrer Ankunft freigegeben.
- A7** Kernel-Overhead ist null.

Annahmen A3 und A4 werden in späteren Kapiteln gezielt aufgehoben: A3 in Kapitel 4 (Deadline Monotonic, EDF mit $D_i < T_i$), A4 in Kapitel 7 (Ressourcenzugriffsprotokolle).

Tabelle 2.1: Übersicht: Zeitparameter periodischer Tasks

Symbol	Bezeichnung	Definition
Φ_i	Phase	$r_{i,1}$ (Ankunft der 1. Instanz)
T_i	Periode	Abstand zwischen Instanzen
C_i	WCET	Max. Ausführungszeit pro Instanz
D_i	Relative Deadline	$\leq T_i$
$r_{i,k}$	Release Time (Instanz k)	$\Phi_i + (k - 1)T_i$
$d_{i,k}$	Absolute Deadline (Instanz k)	$r_{i,k} + D_i$
U_i	Auslastung	C_i / T_i
U	Gesamtauslastung	$\sum_i C_i / T_i$
H	Hyperperiode	$\text{lcm}(T_1, \dots, T_n)$
R_i	Antwortzeit	$f_i - r_i$
L_i	Lateness	$f_i - d_i$
X_i	Laxität	$D_i - C_i$

Kapitel 3

Aperiodisches Task-Scheduling

3.1 Einführung

Dieses Kapitel behandelt Algorithmen zur Planung aperiodischer Echtzeit-Tasks auf einem Einzelprozessorsystem. Jeder Algorithmus stellt eine Lösung für ein bestimmtes Scheduling-Problem dar.

Der fundamentale Kernsatz gilt bereits für aperiodische Tasks:

**Wenn es einen machbaren Schedule gibt, findet EDF diesen sofort.
EDF ist scharf optimal.**

Dies wird im folgenden streng bewiesen.

3.2 Jacksons Algorithmus (EDD)

Das durch Jackson gelöste Problem ist $1 \mid \text{sync} \mid L_{\max}$: Eine Menge J von n aperiodischen Tasks soll auf einem einzelnen Prozessor eingeplant werden, wobei die maximale Lateness minimiert wird. Alle Tasks haben synchrone Ankunftszeiten.

Theorem 3.1 (Jackson 1955). Gegeben sei eine Menge von n unabhängigen Tasks. Jeder Algorithmus, der die Tasks in Reihenfolge nichtabsteigender Deadlines ausführt, ist optimal bezüglich der Minimierung der maximalen Lateness.

Beweis. Der Beweis erfolgt durch ein Austauschargument. Sei σ ein Schedule, der von einem beliebigen Algorithmus A produziert wird. Wenn $A \neq \text{EDD}$, existieren zwei Tasks J_a und J_b mit $d_a \leq d_b$, sodass J_b in σ unmittelbar vor J_a läuft. Sei σ' ein Schedule, der durch Vertauschen von J_a und J_b in σ entsteht. Das Vertauschen kann die maximale Lateness nicht erhöhen: In beiden Fällen ($L'_a \geq L'_b$ oder $L'_a \leq L'_b$) gilt $L'_{\max}(a, b) < L_{\max}(a, b)$. Durch endlich viele solcher Transpositionen wird σ in σ_{EDD} umgewandelt, der optimal ist. ■ □

3.3 Horns Algorithmus (EDF) – Scharf optimal

3.3.1 Das EDF-Optimalitätstheorem

Wenn Tasks dynamische Ankunftszeiten haben, liefert das Earliest Deadline First Prinzip die Grundlage. Der fundamentale Satz lautet:

Wenn es einen machbaren Schedule gibt, findet EDF diesen sofort. Die kürzeste Ausführungszeit aller Tasks findet EDF sofort. EDF ist scharf optimal.

Theorem 3.2 (Horn 1974 / Dertouzos 1974 – EDF Optimalität). Gegeben sei eine Menge von n unabhängigen Tasks mit beliebigen Ankunftszeiten. Jeder Algorithmus, der zu jedem Zeitpunkt die Task mit der frühesten absoluten Deadline unter allen bereiten Tasks ausführt, ist optimal bezüglich der Minimierung der maximalen Lateness. Insbesondere gilt:

Wenn ein machbarer Schedule existiert, findet EDF diesen sofort.

Beweis (Dertouzos 1974). Sei σ der Schedule eines beliebigen Algorithmus A und σ_{EDF} der EDF-Schedule. Da Preemption erlaubt ist, kann jede Task in getrennten Zeitscheiben ausgeführt werden.

Notationen:

- $\sigma(t)$: Task, die in der Scheibe $[t, t + 1)$ ausgeführt wird.
- $E(t)$: Bereite Task, die zum Zeitpunkt t die früheste Deadline hat.
- $t_E(t)$: Zeitpunkt ($\geq t$), zu dem die nächste Scheibe von $E(t)$ in σ ausgeführt wird.

Wenn $\sigma \neq \sigma_{\text{EDF}}$, existiert ein Zeitpunkt t mit $\sigma(t) \neq E(t)$. Das Vertauschen der Scheiben bei t und t_E kann die maximale Lateness nicht erhöhen: Wird eine Scheibe vorgezogen, bleibt die Machbarkeit offensichtlich erhalten. Wird eine Scheibe von J_i auf t_E verschoben und ist σ machbar, so gilt $(t_E + 1) \leq d_E$. Da $d_E \leq d_i$ für alle i , gilt $t_E + 1 \leq d_i$, was die Machbarkeit der verschobenen Scheibe garantiert.

Durch Anwendung des Vertauschungsalgorithmus auf alle Zeitscheiben von $t = 0$ bis $t = D$ (dem spätesten Deadline-Zeitpunkt) wird σ in σ_{EDF} umgewandelt, ohne die maximale Lateness zu erhöhen. Da EDF die maximale Lateness minimiert, findet EDF jeden machbaren Schedule. ■ □

3.3.2 Warum EDF scharf optimal ist

Definition 3.1 (Scharf optimal). Ein Scheduling-Algorithmus $\mathcal{A}$ heißt **scharf optimal**, wenn er für jede Task-Klasse $\mathfrak{S}$ gilt:

$$\forall \Sigma \in \mathfrak{S} : \quad \Sigma \text{ ist schedulierbar} \iff \mathcal{A} \text{ findet einen machbaren Schedule.}$$

Die Bedingung ist eine Äquivalenz, nicht nur eine Implikation.

EDF ist scharf optimal, weil:

1. EDF findet jeden machbaren Schedule (Satz 3.2).
2. Wenn EDF keinen machbaren Schedule findet, existiert keiner – EDF scheitert genau dann, wenn die Auslastung $U > 1$ ist oder die Deadline-Bedingungen strukturell nicht erfüllbar sind.
3. EDF minimiert $L_{\max}$ unter allen Algorithmen.

Die Schärfe bedeutet: EDF nutzt die gesamte verfügbare Prozessorkapazität aus. Die kürzeste Ausführungszeit aller Tasks findet EDF sofort, da EDF immer die Task mit der frühesten Deadline priorisiert – also diejenige, die am dringlichsten ist.

3.3.3 Garantiebedingung für EDF

Wenn Tasks dynamische Aktivierungszeiten haben, muss der Garantietest dynamisch bei jeder neuen Task-Ankunft durchgeführt werden:

Satz 3.1 (EDF-Garantietest). Sei J die aktuell garantierte Task-Menge. Eine neu ankommende Task J_{new} kann garantiert werden, wenn für die Vereinigung $J' = J \cup \{J_{\text{new}}\}$ gilt:

$$\forall i = 1, \dots, n : \sum_{k=1}^i c_k(t) \leq d_i,$$

wobei $c_k(t)$ die verbleibende Ausführungszeit von J_k zum Zeitpunkt t und d_i die Deadline der i -ten Task (nach Deadline sortiert) ist.

3.4 Nicht-preemptives Scheduling

Wenn Preemption nicht erlaubt ist und Tasks beliebige Ankunftszeiten haben, werden Minimierung der maximalen Lateness und Suche nach einem machbaren Schedule NP-schwer [1].

EDF ist in diesem Modell **nicht mehr optimal**: Es kann Fälle geben, in denen ein machbarer Schedule existiert, EDF diesen aber nicht findet. Dies unterstreicht die Bedeutung der Preemptivität für die Optimalität von EDF.

3.4.1 Bratley's Algorithmus

Bratley's Algorithmus (1971) löst das nicht-preemptive Scheduling durch Branch-and-Bound-Suche. Er verwendet Beschneidungstechniken:

- Ein Ast wird abgebrochen, wenn jede Erweiterung des aktuellen Pfades zu einer versäumten Deadline führt.
- Ein Ast wird abgebrochen, wenn ein machbarer Schedule gefunden wurde.

Die Komplexität ist $O(n \cdot n!)$ im schlimmsten Fall.

3.5 Scheduling mit Vorrangrelationen

Theorem 3.3 (Lawler 1973 – LDF Optimalität). Der Latest Deadline First (LDF) Algorithmus minimiert die maximale Lateness für eine Menge von Tasks mit Vorrangrelationen und synchronen Ankunftszeiten.

Für Tasks mit Vorrangrelationen und dynamischen Aktivierungszeiten existiert der EDF*-Algorithmus von Chetto et al. (1990), der eine Transformation der Zeitparameter durchführt, um EDF anwendbar zu machen.

Kapitel 4

Periodisches Task-Scheduling

4.1 Einführung

In vielen Echtzeit-Kontrollanwendungen stellen periodische Aktivitäten den Hauptrechenaufwand dar. Dieses Kapitel behandelt vier grundlegende Algorithmen: Timeline Scheduling, Rate Monotonic, Earliest Deadline First und Deadline Monotonic.

Der zentrale Vergleich dieses Kapitels:

**Wenn es einen machbaren Schedule gibt, findet RM diesen sofort.
RM ist schwach optimal.**

**Wenn es einen machbaren Schedule gibt, findet EDF diesen sofort.
EDF ist scharf optimal.**

Diese Aussage wird durch die Schedulierbarkeitsanalyse präzisiert: RM kann nur Auslastungen bis $\ln 2 \approx 69\%$ im schlimmsten Fall garantieren, während EDF die volle Prozessorkapazität bis $U = 1$ ausschöpft.

4.2 Notation und Annahmen

- Γ : Menge periodischer Tasks
- τ_i : Generische periodische Task
- T_i : Periode von τ_i
- C_i : Worst-Case Ausführungszeit von τ_i
- $D_i = T_i$: Relative Deadline (Assumption A3)
- $U = \sum_{i=1}^n C_i/T_i$: Prozessorauslastung

Annahmen: A1 (konstante Periode), A2 (konstante WCET), A3 ($D_i = T_i$), A4 (unabhängige Tasks).

4.3 Timeline Scheduling

Timeline Scheduling (Cyclic Executive) unterteilt die Zeitachse in gleich lange Slots und weist Tasks deterministisch zu. Der **Minor Cycle** ist die Länge eines Slots (GCD der Perioden), der **Major Cycle** ist der Hyperperiod (LCM der Perioden).

Vorteile: Einfach zu implementieren, kein Overhead durch Context Switches, deterministische Ausführungsreihenfolge.

Nachteile: Spröde bei Überlastbedingungen, sensibel gegenüber Anwendungsänderungen, schwierig für aperiodische Tasks.

4.4 Rate Monotonic Scheduling – Detaillierte Analyse

4.4.1 Einführung und Grundidee

Der Rate Monotonic (RM) Scheduling-Algorithmus wurde 1973 von Liu und Layland eingeführt [6] und gehört heute zu den meistgenutzten Algorithmen für periodische Echtzeit-Tasks in eingebetteten Systemen. Die Grundidee ist denkbar einfach: Tasks werden statische Prioritäten *umgekehrt proportional* zu ihrer Periode zugewiesen. Eine Task mit kurzer Periode (also hoher Anforderungsrate) erhält eine höhere Priorität als eine Task mit langer Periode.

Definition 4.1 (Rate Monotonic Prioritätszuweisung). Der **Rate Monotonic** Algorithmus weist jeder periodischen Task τ_i eine Priorität P_i zu, die umgekehrt proportional zur Periode ist:

$$T_i < T_j \implies P_i > P_j.$$

Tasks mit gleicher Periode erhalten beliebige, aber feste Prioritäten.

Da die Perioden konstant sind, ist RM eine **feste Prioritätszuweisung**: Die Priorität einer Task ändert sich während der Ausführung nicht. RM ist außerdem **preemptiv**: Wird eine neue Instanz einer hochprioritären Task aktiv, unterbricht sie die laufende niederprioritäre Task sofort.

Beispiel 4.1 (Einfache RM-Zuweisung). Gegeben sei ein Task-Set mit drei Tasks:

$$\tau_1 : T_1 = 4 \text{ ms}, \quad \tau_2 : T_2 = 6 \text{ ms}, \quad \tau_3 : T_3 = 12 \text{ ms}.$$

RM ordnet die Prioritäten: $P_1 > P_2 > P_3$, da $T_1 < T_2 < T_3$. τ_1 ist immer die dringlichste Task und kann τ_2 und τ_3 jederzeit unterbrechen. τ_2 kann nur τ_3 unterbrechen.

4.4.2 Kritischer Augenblick – Das Fundament der RM-Analyse

Das wichtigste Konzept für die Analyse von RM ist der *kritische Augenblick* (critical instant). Er gibt an, unter welchen Bedingungen eine Task die größte Antwortzeit erfährt.

Definition 4.2 (Kritischer Augenblick). Der **kritische Augenblick** einer Task τ_i ist derjenige Ankunftszeitpunkt, der die größte Antwortzeit von τ_i produziert. Die zugehörige **kritische Zeitzone** ist das Intervall zwischen dem kritischen Augenblick und der entsprechenden Antwortzeit.

Lemma 4.1 (Liu & Layland 1973 – Kritischer Augenblick). Der kritische Augenblick für eine Task τ_i unter RM tritt auf, wenn τ_i *gleichzeitig* mit allen Tasks höherer Priorität aktiviert wird.

Beweis. Wir zeigen durch sukzessives Vorrücken der Ankunftszeitpunkte hochpriorer Tasks, dass die Antwortzeit von τ_i maximal wird, wenn alle gleichzeitig ankommen.

Sei τ_n die Task mit der niedrigsten Priorität und τ_i eine Task mit höherer Priorität. Die Antwortzeit von τ_n wird durch die Interferenz von τ_i verzögert. Wie in Abbildung (a) dargestellt, verzögert eine Instanz von τ_i die Ausführung von τ_n um C_i . Das Vorziehen der Ankunftszeit von τ_i kann die Fertigstellungszeit von τ_n weiter erhöhen (Abbildung b), da eine zusätzliche Instanz von τ_i in das Interferenzintervall fallen kann.

Durch wiederholtes Vorrücken aller hochpriorer Tasks wird die Antwortzeit von τ_n maximiert, wenn alle gleichzeitig ankommen. Dieses Argument gilt für jede Task τ_n mit niedrigster Priorität, und durch Rekursion für jede Task im System. ■ □

Bemerkung 4.1. Lemma 4.1 hat eine weitreichende praktische Konsequenz: Um zu prüfen, ob ein Task-Set unter RM schedulierbar ist, genügt es, den Schedule am kritischen Augenblick zu analysieren – also den Fall, dass alle Tasks zum Zeitpunkt $t = 0$ gleichzeitig starten. Wenn alle Tasks in diesem ungünstigsten Fall ihre Deadlines einhalten, sind sie *unter allen* möglichen Ankunftsbedingungen schedulierbar.

4.4.3 Vollständiger Optimalitätsbeweis von RM

Definition 4.3 (Schwach optimal). Ein Scheduling-Algorithmus $\mathcal{A}$ heißt **schwach optimal** innerhalb einer Klasse $\mathfrak{A}$ von Algorithmen, wenn gilt:

$$\forall A \in \mathfrak{A}, \forall \Gamma : \quad A \text{ scheduliert } \Gamma \implies \mathcal{A} \text{ scheduliert } \Gamma.$$

Theorem 4.1 (Liu & Layland 1973 – RM Optimalität). RM ist **schwach optimal** unter allen Algorithmen mit fester Priorität: Wenn ein Task-Set Γ von *irgendeiner* festen Prioritätszuweisung scheduliert werden kann, so kann es auch von RM scheduliert werden.

Beweis. Der Beweis wird durch ein direktes Austauschargument für zwei Tasks geführt und anschließend auf n Tasks verallgemeinert.

Schritt 1: Zwei Tasks, $T_1 < T_2$.

Angenommen, die Prioritäten seien *nicht* gemäß RM zugewiesen, d. h. τ_2 hat eine höhere Priorität als τ_1 . Am kritischen Augenblick ($t = 0$) sieht der Schedule aus wie in Abbildung ??: τ_2 wird zuerst ausgeführt, dann τ_1 . Für Machbarkeit muss gelten:

$$C_1 + C_2 \leq T_1. \tag{4.1}$$

Nun weisen wir die Prioritäten gemäß RM zu, d. h. τ_1 erhält die höhere Priorität ($T_1 < T_2$). Wir müssen zeigen, dass die Machbarkeitsbedingung (4.1) auch unter RM ausreicht. Dazu unterscheiden wir zwei Fälle, je nachdem, wie C_1 und T_2 sich zueinander verhalten. Sei $F = \lfloor T_2/T_1 \rfloor$ die Anzahl vollständiger T_1 -Perioden innerhalb von T_2 .

Fall 1: $C_1 < T_2 - FT_1$.

In diesem Fall werden alle $F + 1$ Instanzen von τ_1 abgeschlossen, bevor die zweite Instanz von τ_2 ihre Deadline erreicht. Die Machbarkeitsbedingung für τ_2 unter RM ist:

$$(F + 1)C_1 + C_2 \leq T_2. \tag{4.2}$$

Wir zeigen: (4.1) $\implies$ (4.2). Aus (4.1) folgt durch Multiplikation mit F : $FC_1 + FC_2 \leq FT_1$, und da $F \geq 1$: $FC_1 + C_2 \leq FT_1$. Addition von C_1 ergibt $(F + 1)C_1 + C_2 \leq FT_1 + C_1$. Da wir in Fall 1 voraussetzen, dass $C_1 < T_2 - FT_1$, erhalten wir: $(F + 1)C_1 + C_2 < FT_1 + (T_2 - FT_1) = T_2$, womit (4.2) erfüllt ist.

Fall 2: $C_1 \geq T_2 - FT_1$.

Hier überlappt die Ausführung von τ_1 mit der zweiten Instanz von τ_2 . Die Machbarkeitsbedingung für τ_2 unter RM ist:

$$FC_1 + C_2 \leq FT_1. \quad (4.3)$$

Aus (4.1) folgt durch Multiplikation mit F : $FC_1 + FC_2 \leq FT_1$, und da $F \geq 1$: $C_2 \leq FC_2$, also: $FC_1 + C_2 \leq FC_1 + FC_2 \leq FT_1$, womit (4.3) erfüllt ist.

In beiden Fällen gilt: Wenn das Task-Set unter der nicht-RM-Zuweisung schedulierbar ist, ist es auch unter RM schedulierbar.

Schritt 2: Verallgemeinerung auf n Tasks.

Sei $\Gamma = \{\tau_1, \dots, \tau_n\}$ ein Task-Set, das von einer beliebigen festen Prioritätszuweisung π scheduliert wird. Wenn π nicht die RM-Reihenfolge ist, existieren zwei Tasks τ_i, τ_j mit $T_i < T_j$, aber $P_i^\pi < P_j^\pi$ (invertierte Priorität). Durch Anwendung des Zwei-Task-Argumentes aus Schritt 1 können die Prioritäten von τ_i und τ_j vertauscht werden, ohne die Schedulierbarkeit zu verletzen. Da das Task-Set endlich ist, führt die iterierte Anwendung dieser Transposition nach endlich vielen Schritten zur RM-Reihenfolge, ohne dass dabei eine Deadline verletzt wird. ■ □

4.4.4 Berechnung des Least Upper Bound für zwei Tasks

Der Least Upper Bound U_{lub} ist die kleinste Schranke der Prozessorauslastung, unterhalb derer ein Task-Set unter RM *garantiert* schedulierbar ist.

Vorgehen

Für $n = 2$ Tasks τ_1 und τ_2 mit $T_1 < T_2$ berechnen wir U_{lub} wie folgt:

1. Zuweisung gemäß RM: τ_1 erhält die höhere Priorität.
2. Erhöhung von C_2 bis der Prozessor vollständig ausgelastet ist.
3. Minimierung der oberen Schranke U_{ub} über alle Periodenrelationen.

Sei wiederum $F = \lfloor T_2/T_1 \rfloor$.

Fall 1: $C_1 \leq T_2 - FT_1$

Der größtmögliche Wert von C_2 beträgt $C_2 = T_2 - C_1(F + 1)$, und die obere Schranke der Auslastung ist:

$$U_{\text{ub}} = \frac{C_1}{T_1} + \frac{C_2}{T_2} = \frac{C_1}{T_1} + \frac{T_2 - C_1(F + 1)}{T_2} = 1 + \frac{C_1}{T_2} \left(\frac{T_2}{T_1} - (F + 1) \right).$$

Da $T_2/T_1 \geq F + 1$ in diesem Fall nicht gilt (der Klammerausdruck ist negativ), ist U_{ub} monoton fallend in C_1 . Das Minimum wird also bei dem größten erlaubten C_1 erreicht, nämlich $C_1 = T_2 - FT_1$.

Fall 2: $C_1 \geq T_2 - FT_1$

Der größtmögliche Wert von C_2 beträgt $C_2 = (T_1 - C_1)F$, und:

$$U_{\text{ub}} = \frac{C_1}{T_1} + \frac{(T_1 - C_1)F}{T_2} = \frac{T_1}{T_2}F + \frac{C_1}{T_2} \left(\frac{T_2}{T_1} - F \right).$$

Hier ist U_{ub} monoton steigend in C_1 . Das Minimum liegt also bei $C_1 = T_2 - FT_1$.

Gemeinsamer Minimierungspunkt

In beiden Fällen wird das Minimum bei $C_1 = T_2 - T_1F$ erreicht. Einsetzen liefert mit der Abkürzung $G = T_2/T_1 - F$ (sodass $0 \leq G < 1$):

$$U_{\text{ub}} = \frac{T_1}{T_2}F + \frac{(T_2 - T_1F)}{T_2} \cdot \frac{T_2/T_1 - F}{1} = \frac{F + G^2}{F + G}.$$

Für $F = 1$ (den ungünstigsten Fall) vereinfacht sich das zu:

$$U = \frac{1 + G^2}{1 + G}.$$

Minimierung über $G \in [0, 1)$ durch Ableiten ergibt:

$$\frac{dU}{dG} = \frac{2G(1 + G) - (1 + G^2)}{(1 + G)^2} = \frac{G^2 + 2G - 1}{(1 + G)^2} = 0,$$

also $G = -1 + \sqrt{2}$ (positive Lösung). Einsetzen liefert:

$$U_{\text{lub}}(2) = \frac{1 + (\sqrt{2} - 1)^2}{1 + (\sqrt{2} - 1)} = \frac{4 - 2\sqrt{2}}{2} = 2(\sqrt{2} - 1) \approx 0,828.$$

Bemerkung 4.2. Wenn T_2 ein ganzzahliges Vielfaches von T_1 ist, gilt $G = 0$ und $U_{\text{ub}} = 1$. Das bedeutet: Für harmonische Perioden ($T_2 = kT_1$) kann RM den Prozessor vollständig auslasten.

4.4.5 Berechnung des Least Upper Bound für n Tasks

Theorem 4.2 (Liu & Layland 1973 – RM Least Upper Bound). Für eine beliebige Menge von n periodischen Tasks unter den Annahmen A1–A4 ist der Least Upper Bound der Prozessorauslastung unter dem Rate Monotonic Algorithmus:

$$U_{\text{lub}}(\text{RM}, n) = n(2^{1/n} - 1).$$

Für $n \rightarrow \infty$ gilt $U_{\text{lub}} \rightarrow \ln 2 \approx 0,693$.

Beweis. Die schlechteste Bedingung für die Schedulierbarkeit, d. h. das globale Minimum der oberen Schranke U_{ub} über alle Task-Sets, die den Prozessor vollständig auslasten, wird durch folgende Parameterwahl erreicht [6]:

$$\begin{cases} T_1 < T_n < 2T_1 \\ C_1 = T_2 - T_1 \\ C_2 = T_3 - T_2 \\ \vdots \\ C_{n-1} = T_n - T_{n-1} \\ C_n = 2T_1 - T_n. \end{cases} \quad (4.4)$$

Die resultierende Prozessorauslastung lässt sich mit den Abkürzungen $R_i = T_{i+1}/T_i$ schreiben als:

$$U = \sum_{i=1}^{n-1} R_i + \frac{2}{R_1 R_2 \cdots R_{n-1}} - n.$$

Minimierung über alle R_i durch partielles Ableiten:

$$\frac{\partial U}{\partial R_k} = 1 - \frac{2}{R_k^2 \cdot \prod_{i \neq k} R_i} = 0.$$

Mit $P = \prod_{i=1}^{n-1} R_i$ erhält man $R_k P = 2$ für alle k , also:

$$R_1 = R_2 = \cdots = R_{n-1} = 2^{1/n}.$$

Einsetzen dieses gemeinsamen Wertes ergibt:

$$\begin{aligned} U_{\text{lub}} &= (n-1) \cdot 2^{1/n} + \frac{2}{2^{(n-1)/n}} - n \\ &= (n-1) \cdot 2^{1/n} + 2^{1/n} - n \\ &= n \cdot 2^{1/n} - n \\ &= n(2^{1/n} - 1). \end{aligned}$$

Grenzwert für $n \rightarrow \infty$:

Mit der Substitution $y = 2^{1/n} - 1$, also $n = \ln 2 / \ln(y+1)$, gilt:

$$\lim_{n \rightarrow \infty} n(2^{1/n} - 1) = \ln 2 \cdot \lim_{y \rightarrow 0} \frac{y}{\ln(y+1)}.$$

Nach der Regel von L'Hôpital:

$$\lim_{y \rightarrow 0} \frac{y}{\ln(y+1)} = \lim_{y \rightarrow 0} \frac{1}{1/(y+1)} = 1.$$

Daher gilt $\lim_{n \rightarrow \infty} U_{\text{lub}} = \ln 2 \approx 0,693$. ■ □

Tabelle 4.1: Werte von U_{lub} für RM als Funktion der Taskanzahl n

n	U_{lub}	n	U_{lub}
1	1,000	6	0,735
2	0,828	7	0,729
3	0,780	8	0,724
4	0,757	9	0,721
5	0,743	10	0,718
∞	$\ln 2 \approx 0,693$		

Bemerkung 4.3. Die Tabelle zeigt, dass der Least Upper Bound mit wachsender Taskanzahl monoton gegen $\ln 2 \approx 0,693$ fällt. Das bedeutet: Für sehr große Task-Sets kann RM nur Task-Mengen mit Auslastung unter 69% garantiert einplanen. EDF hingegen garantiert Schedulierbarkeit bis $U = 1$, also eine um bis zu 31 Prozentpunkte höhere Nutzung der Prozessorkapazität.

4.4.6 Praktische Anwendung: Hinreichende Schedulierbarkeits-test

Der Liu-Layland-Bound liefert einen einfachen, hinreichenden Test:

Satz 4.1 (RM Hinreichende Bedingung – Liu & Layland). Eine Menge von n periodischen Tasks ist unter RM schedulierbar, wenn:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1).$$

Beispiel 4.2 (Anwendung des Liu-Layland-Tests). Gegeben:

$$\tau_1 : C_1 = 1, T_1 = 4 \quad \tau_2 : C_2 = 2, T_2 = 6 \quad \tau_3 : C_3 = 3, T_3 = 12.$$

Auslastung: $U = 1/4 + 2/6 + 3/12 = 0,25 + 0,333 + 0,25 = 0,833$. Schranke: $U_{\text{lub}}(3) = 3(2^{1/3} - 1) \approx 0,780$. Da $U = 0,833 > 0,780$, gibt der Test *keine Garantie* – aber das Task-Set könnte dennoch schedulierbar sein.

Bemerkung 4.4 (Test ist hinreichend, nicht notwendig). Der Liu-Layland-Test ist **hinreichend**, aber **nicht notwendig**. Ein Task-Set mit $U > U_{\text{lub}}$ kann trotzdem mit RM schedulierbar sein, wenn die Perioden günstig zueinander liegen (z. B. bei harmonischen Perioden). Umgekehrt: Wenn $U \leq U_{\text{lub}}$, ist die Schedulierbarkeit garantiert. Zum exakten Test dient die Response Time Analysis (Abschnitt 4.4.8).

4.4.7 Hyperbolic Bound – Engere Schranke für RM

Der Hyperbolic Bound von Bini und Buttazzo [17] ist eine schärfere hinreichende Bedingung als der Liu-Layland-Bound. Er nutzt die individuellen Auslastungsfaktoren $U_i = C_i/T_i$ statt ihrer Summe.

Theorem 4.3 (Bini, Buttazzo & Buttazzo 2001 – Hyperbolic Bound). Sei $\Gamma = \{\tau_1, \dots, \tau_n\}$ mit $U_i = C_i/T_i$. Dann ist Γ unter RM schedulierbar, wenn:

$$\prod_{i=1}^n (U_i + 1) \leq 2.$$

Beweis. Der Beweis nutzt die gleiche Worst-Case-Parametrisierung (4.4). Dort gilt $U_i = R_i - 1$ für $i = 1, \dots, n-1$, also $R_i = U_i + 1$. Damit ist $\prod_{i=1}^{n-1} R_i = T_n/T_1$. Die Schedulierbarkeitsbed. (4.3) für τ_n lautet $U_n \leq 2T_1/T_n - 1$, d. h. $(U_n + 1) \leq 2 / \prod_{i=1}^{n-1} (U_i + 1)$, woraus $\prod_{i=1}^n (U_i + 1) \leq 2$ folgt. ■ □

Bemerkung 4.5 (Vergleich der Schranken). Der Hyperbolic Bound ist stets mindestens so gut wie der Liu-Layland-Bound und häufig besser. Er akzeptiert Task-Sets, die der Liu-Layland-Bound ablehnen würde. Formell gilt:

$$\prod_{i=1}^n (U_i + 1) \leq 2 \text{ ist eine striktere Bedingung als } \sum_{i=1}^n U_i \leq n(2^{1/n} - 1)$$

in dem Sinne, dass die zulässige Schedulierbarkeitsregion durch den Hyperbolic Bound größer ist. Der Gewinn wächst mit der Anzahl der Tasks und nähert sich $\sqrt{2}$ für $n \rightarrow \infty$.

Beispiel 4.3 (Hyperbolic Bound vs. Liu-Layland). Gegeben: $\tau_1 : U_1 = 0,4$, $\tau_2 : U_2 = 0,4$.

Liu-Layland: $U = 0,8 \leq 2(\sqrt{2} - 1) \approx 0,828$. Bestanden.

Hyperbolic Bound: $(1,4)(1,4) = 1,96 \leq 2$. Bestanden.

Nun: $\tau_1 : U_1 = 0,41$, $\tau_2 : U_2 = 0,41$.

Liu-Layland: $U = 0,82 \leq 0,828$. Bestanden.

Hyperbolic Bound: $(1,41)^2 = 1,9881 \leq 2$. Bestanden.

Nun: $\tau_1 : U_1 = 0,43$, $\tau_2 : U_2 = 0,43$.

Liu-Layland: $U = 0,86 > 0,828$. *Kein Ergebnis!*

Hyperbolic Bound: $(1,43)^2 = 2,0449 > 2$. *Kein Ergebnis!*

Aber mit $U_1 = 0,42$, $U_2 = 0,40$:

Liu-Layland: $U = 0,82 \leq 0,828$. Bestanden.

Hyperbolic Bound: $(1,42)(1,40) = 1,988 \leq 2$. Bestanden.

Dagegen mit $U_1 = 0,44$, $U_2 = 0,40$:

Liu-Layland: $U = 0,84 > 0,828$. *Keine Garantie.*

Hyperbolic Bound: $(1,44)(1,40) = 2,016 > 2$. *Keine Garantie.*

4.4.8 Response Time Analysis – Exakter Schedulierbarkeitstest

Der Liu-Layland-Bound und der Hyperbolic Bound sind hinreichend, aber nicht notwendig. Für einen *exakten* Test benötigt man die Response Time Analysis (RTA), die von Audsley et al. [16] entwickelt wurde.

Grundprinzip

Die worst-case Antwortzeit R_i einer Task τ_i setzt sich zusammen aus ihrer eigenen Ausführungszeit C_i und der Interferenz I_i durch alle Tasks mit höherer Priorität. Die Interferenz im Intervall $[0, R_i]$ ist die Summe aller Ausführungen hochpriorer Tasks, die in dieses Intervall fallen:

$$I_i = \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j,$$

wobei $hp(i) = \{\tau_j \mid P_j > P_i\}$ die Menge der Tasks mit höherer Priorität ist und $\lceil R_i/T_j \rceil$ die Anzahl der Instanzen von τ_j im Intervall $[0, R_i]$ angibt.

Definition 4.4 (Worst-Case Antwortzeit unter RM). Die **worst-case Antwortzeit** R_i von τ_i ist die kleinste positive Lösung der impliziten Gleichung:

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j. \quad (4.5)$$

Da R_i auf beiden Seiten von Gleichung (4.5) auftritt, gibt es keine geschlossene Lösung. Die Berechnung erfolgt iterativ.

Iterativer Berechnungsalgorithmus

Require: Task τ_i , geordnet nach RM-Priorität

1: $R_i^{(0)} \leftarrow \sum_{j=1}^i C_j$ ▷ Startschätzung: frühester Fertigstellungszeitpunkt

2: **repeat**

3: $R_i^{(k+1)} \leftarrow C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$

```

4: until  $R_i^{(k+1)} = R_i^{(k)}$  oder  $R_i^{(k+1)} > T_i$ 
5: if  $R_i^{(k+1)} \leq T_i$  then
6:   return SCHEDULIERBAR
7: else
8:   return NICHT SCHEDULIERBAR
9: end if

```

Die Sequenz $R_i^{(0)}, R_i^{(1)}, R_i^{(2)}, \dots$ ist monoton nichtfallend und durch T_i nach oben beschränkt. Sie konvergiert daher entweder zu einem Fixpunkt (der Task schedulierbar) oder überschreitet T_i (Task nicht schedulierbar).

Beispiel 4.4 (Response Time Analysis – Schritt für Schritt). Gegeben:

Task	C_i	T_i	Priorität (RM)
τ_1	1	4	1 (höchste)
τ_2	2	6	2
τ_3	3	10	3 (niedrigste)

Auslastung: $U = 1/4 + 2/6 + 3/10 = 0,25 + 0,333 + 0,30 = 0,883$. Da $U_{\text{lub}}(3) \approx 0,780 < 0,883$, liefert der Liu-Layland-Test kein Ergebnis. Wir nutzen die RTA.

Task τ_1 (keine höherprioräre Task): $R_1 = C_1 = 1 \leq T_1 = 4$. ✓

Task τ_2 ($hp(2) = \{\tau_1\}$):

$$R_2^{(0)} = C_1 + C_2 = 3$$

$$R_2^{(1)} = C_2 + \lceil 3/4 \rceil \cdot C_1 = 2 + 1 \cdot 1 = 3 = R_2^{(0)}.$$

Konvergenz bei $R_2 = 3 \leq T_2 = 6$. ✓

Task τ_3 ($hp(3) = \{\tau_1, \tau_2\}$):

$$R_3^{(0)} = C_1 + C_2 + C_3 = 6$$

$$R_3^{(1)} = 3 + \lceil 6/4 \rceil \cdot 1 + \lceil 6/6 \rceil \cdot 2 = 3 + 2 + 2 = 7$$

$$R_3^{(2)} = 3 + \lceil 7/4 \rceil \cdot 1 + \lceil 7/6 \rceil \cdot 2 = 3 + 2 + 4 = 9$$

$$R_3^{(3)} = 3 + \lceil 9/4 \rceil \cdot 1 + \lceil 9/6 \rceil \cdot 2 = 3 + 3 + 4 = 10$$

$$R_3^{(4)} = 3 + \lceil 10/4 \rceil \cdot 1 + \lceil 10/6 \rceil \cdot 2 = 3 + 3 + 4 = 10 = R_3^{(3)}.$$

Konvergenz bei $R_3 = 10 = T_3$. ✓ (Gerade noch schedulierbar.)

Das Task-Set ist unter RM schedulierbar, obwohl $U > U_{\text{lub}}$. Dies zeigt, dass der Liu-Layland-Bound zu pessimistisch war.

4.4.9 Schedulierbarkeit bei kritischem Augenblick – Direkte Methode

Alternativ zur Response Time Analysis kann die Schedulierbarkeit direkt durch Konstruktion des Schedules am kritischen Augenblick überprüft werden.

Satz 4.2 (Schedulierbarkeit am kritischen Augenblick). Ein Task-Set Γ ist unter RM genau dann schedulierbar, wenn jede Task τ_i ihre Deadline an ihrem kritischen Augenblick einhält, d. h. wenn alle Tasks zum Zeitpunkt $t = 0$ gleichzeitig aktiviert werden und jede Task innerhalb ihrer ersten Periode fertiggestellt wird.

Beispiel 4.5 (Schedulierbarkeit direkt am kritischen Augenblick). Gegeben: $\tau_1 : C_1 = 1, T_1 = 4$; $\tau_2 : C_2 = 2, T_2 = 6$.

Auslastung: $U = 1/4 + 2/6 = 0,583 \leq U_{\text{lub}}(2) \approx 0,828$. Garantiert.

Konstruktion des Schedules ab $t = 0$:

- $[0, 1)$: τ_1 läuft (höhere Priorität, $T_1 < T_2$).
- $[1, 3)$: τ_2 läuft (keine andere Task bereit).
- $[3, 4)$: τ_2 läuft weiter – aber τ_1 hat neue Instanz bei $t = 4$!
- Bei $t = 3$: τ_2 ist fertig ($f_{2,1} = 3 \leq d_{2,1} = 6$). ✓
- $[3, 4)$: Leerlauf.
- $[4, 5)$: τ_1 , zweite Instanz.
- $[5, 7)$: τ_2 , zweite Instanz. $f_{2,2} = 7 \leq d_{2,2} = 12$. ✓

Das Task-Set ist schedulierbar.

4.4.10 Workload-Analyse nach Lehoczky, Sha und Ding

Lehoczky, Sha und Ding [14] entwickelten eine präzisere Methode zur Schedulierbarkeitsanalyse unter fester Priorität, die auf dem Konzept der Level- i -Arbeitslast basiert.

Definition 4.5 (Level- i -Arbeitslast). Die **Level- i -Arbeitslast** $W_i(t)$ ist die kumulative Rechenzeit, die im Intervall $(0, t]$ von Task τ_i und allen Tasks mit höherer Priorität angefordert wird:

$$W_i(t) = C_i + \sum_{h: P_h > P_i} \left\lfloor \frac{t}{T_h} \right\rfloor C_h.$$

Theorem 4.4 (Lehoczky, Sha, Ding 1989). Eine Menge vollständig preemptiver periodischer Tasks ist unter einem Algorithmus mit fester Priorität genau dann schedulierbar, wenn:

$$\forall i = 1, \dots, n : \quad \exists t \in (0, T_i] : \quad W_i(t) \leq t.$$

4.4.11 Warum RM schwach optimal ist – Präzisierung

Definition 4.6 (Schwach optimal – Präzisierung). RM ist **schwach optimal** in folgendem Sinne:

1. RM ist optimal *innerhalb* der Klasse aller Algorithmen mit fester Prioritätszuweisung.
2. Es gibt Task-Mengen mit $\ln 2 < U \leq 1$, die *nicht* unter RM schedulierbar sind, aber unter EDF schedulierbar sind.
3. RM ist *nicht* optimal über alle Scheduling-Algorithmen.

Satz 4.3 (Grenzen der Schwäche von RM). Für jede Auslastung U mit $U_{\text{lub}}(\text{RM}, n) < U \leq 1$ existiert ein Task-Set Γ_U mit n Tasks und Auslastung U , das:

- von EDF schedulierbar ist (da $U \leq 1$),

– von RM *nicht* schedulierbar ist.

Beweis. Wähle Γ_U gemäß den Worst-Case-Parametern (4.4) mit der gegebenen Auslastung U . Per Konstruktion ist Γ_U nicht unter RM schedulierbar (da $U > U_{\text{lub}}$). Da $U \leq 1$, ist Γ_U nach Satz 4.5 unter EDF schedulierbar. ■ □

4.4.12 Vollständiges Beispiel: Vergleich RM und EDF bei hoher Auslastung

Beispiel 4.6 (RM versagt, EDF scheduliert). Gegeben:

$$\tau_1 : C_1 = 3, \quad T_1 = 5 \quad \tau_2 : C_2 = 4, \quad T_2 = 7.$$

Auslastung: $U = 3/5 + 4/7 = 0,6 + 0,571 = 34/35 \approx 0,971$.

RM-Analyse: $U_{\text{lub}}(2) \approx 0,828 < 0,971$. Der Liu-Layland-Test gibt keine Garantie. Exakter Test mittels RTA:

$$\begin{aligned} R_1 &= C_1 = 3 \leq 5. \quad \checkmark \\ R_2^{(0)} &= 7 \\ R_2^{(1)} &= 4 + \lceil 7/5 \rceil \cdot 3 = 4 + 2 \cdot 3 = 10 > T_2 = 7. \end{aligned}$$

τ_2 ist unter RM *nicht schedulierbar*. Am kritischen Augenblick (alle Tasks starten bei $t = 0$) verpasst τ_2 ihre erste Deadline bei $t = 7$.

EDF-Analyse: $U \approx 0,971 \leq 1$. Task-Set ist unter EDF schedulierbar. Der Schedule verläuft ohne Deadline-Verletzungen, da EDF zu jedem Zeitpunkt die Task mit der frühesten absoluten Deadline ausführt und dabei die volle Prozessorkapazität nutzt.

Dieser direkte Vergleich illustriert die Aussage:

**Wenn es einen machbaren Schedule gibt, findet RM diesen sofort.
RM ist schwach optimal. Wenn es einen machbaren Schedule gibt,
findet EDF diesen sofort. EDF ist scharf optimal.**

Für τ_2 existiert ein machbarer Schedule (unter EDF), aber RM findet ihn nicht.

4.4.13 Übungsaufgaben zu Rate Monotonic

Beispiel 4.7 (Übungsaufgabe 1 – Schedulierbarkeitstest). Bestimme, ob folgendes Task-Set unter RM schedulierbar ist:

$$\tau_1 : C = 2, T = 6 \quad \tau_2 : C = 2, T = 8 \quad \tau_3 : C = 2, T = 12.$$

Lösung: $U = 2/6 + 2/8 + 2/12 = 1/3 + 1/4 + 1/6 = 3/4 = 0,75$. $U_{\text{lub}}(3) = 3(2^{1/3} - 1) \approx 0,780 > 0,75$. Schedulierbar. ✓

Beispiel 4.8 (Übungsaufgabe 2 – Nicht schedulierbar). Bestimme, ob folgendes Task-Set unter RM schedulierbar ist:

$$\tau_1 : C = 1, T = 4 \quad \tau_2 : C = 2, T = 6 \quad \tau_3 : C = 3, T = 8.$$

Lösung: $U = 1/4 + 2/6 + 3/8 = 0,25 + 0,333 + 0,375 = 0,958$. $U_{\text{lub}}(3) \approx 0,780 < 0,958$.
Kein Ergebnis aus Liu-Layland-Test. RTA für τ_3 :

$$R_3^{(0)} = 1 + 2 + 3 = 6$$

$$R_3^{(1)} = 3 + \lceil 6/4 \rceil \cdot 1 + \lceil 6/6 \rceil \cdot 2 = 3 + 2 + 2 = 7$$

$$R_3^{(2)} = 3 + \lceil 7/4 \rceil \cdot 1 + \lceil 7/6 \rceil \cdot 2 = 3 + 2 + 4 = 9$$

$$R_3^{(3)} = 3 + \lceil 9/4 \rceil \cdot 1 + \lceil 9/6 \rceil \cdot 2 = 3 + 3 + 4 = 10 > T_3 = 8.$$

τ_3 ist unter RM nicht schedulierbar. Da $U \approx 0,958 \leq 1$, ist das Task-Set unter EDF schedulierbar.

4.5 Earliest Deadline First (EDF) – Scharf optimal

4.5.1 Algorithmus

EDF ist eine dynamische Scheduling-Regel: Tasks mit früheren absoluten Deadlines werden zuerst ausgeführt. EDF ist typischerweise preemptiv.

EDF ist scharf optimal: Wenn ein machbarer Schedule existiert, findet EDF diesen sofort. EDF findet die kürzeste Ausführungszeit aller Tasks sofort.

4.5.2 Schedulierbarkeitsanalyse für EDF

Theorem 4.5 (Liu & Layland 1973 – EDF Schedulierbarkeit). Eine Menge periodischer Tasks ist mit EDF schedulierbar genau dann, wenn:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1.$$

Beweis. **“Nur wenn” Teil:** Wenn $U > 1$, überschreitet der Gesamtrechenaufwand in der Hyperperiode H die verfügbare Zeit H . Kein Algorithmus kann diesen Schedule machbar machen.

“Wenn” Teil (durch Widerspruch): Angenommen, $U \leq 1$, aber der Task-Satz ist nicht schedulierbar. Sei t_2 der erste Zeitpunkt, an dem eine Deadline versäumt wird, und $[t_1, t_2]$ das längste Intervall kontinuierlicher Auslastung vor t_2 , in dem nur Instanzen mit Deadline $\leq t_2$ ausgeführt werden.

Der gesamte Rechenzeitbedarf in $[t_1, t_2]$ ist:

$$C_p(t_1, t_2) = \sum_{i=1}^n \left\lfloor \frac{t_2 - t_1}{T_i} \right\rfloor C_i \leq (t_2 - t_1)U.$$

Da eine Deadline versäumt wird, muss $C_p(t_1, t_2) > (t_2 - t_1)$ gelten, also $U > 1$ – Widerspruch. ■ □

4.5.3 Die Schärfe von EDF im Vergleich zu RM

Der fundamentale Unterschied:

Satz 4.4 (Schärfe von EDF gegenüber RM). Es gibt Task-Mengen, die EDF schedulieren kann, RM aber nicht. Insbesondere gilt für alle U mit $\ln 2 < U \leq 1$:

- EDF garantiert die Machbarkeit (nach Satz 4.5).
- RM kann die Machbarkeit *nicht* garantieren (nach Satz 4.2).

Beispiel 4.9. Betrachte eine periodische Task-Menge mit zwei Tasks:

$$\tau_1 : C_1 = 3, \quad T_1 = 5 \quad \tau_2 : C_2 = 4, \quad T_2 = 7.$$

Die Prozessorauslastung ist:

$$U = \frac{3}{5} + \frac{4}{7} = 0,6 + 0,571 = \frac{34}{35} \approx 0,97.$$

Da $U = 0,97 > 2(\sqrt{2} - 1) \approx 0,83$, kann RM die Schedulierbarkeit *nicht* garantieren – und erzeugt tatsächlich eine Deadline-Verletzung. Da $U \approx 0,97 \leq 1$, ist der Task-Satz unter EDF schedulierbar. Dies zeigt die Schwäche von RM und die Schärfe von EDF.

4.6 Deadline Monotonic Scheduling

Deadline Monotonic (DM) ist eine Erweiterung von RM für Tasks mit Deadlines kleiner als ihre Perioden ($D_i \leq T_i$). DM weist Prioritäten umgekehrt proportional zur relativen Deadline zu.

DM ist optimal unter allen Algorithmen mit fester Priorität für Tasks mit $D_i \leq T_i$ – analog zu RM ist DM damit ebenfalls *schwach optimal* im Sinne dieser Klasse.

4.7 EDF mit eingeschränkten Deadlines

Für Tasks mit $D_i \leq T_i$ kann die Schedulierbarkeit unter EDF durch das Processor Demand Criterion (Baruah et al. 1990) getestet werden:

Satz 4.5 (Schedulierbarkeit unter EDF mit eingeschränkten Deadlines). Eine synchrone Menge periodischer Tasks mit $D_i \leq T_i$ ist mit EDF schedulierbar genau dann, wenn $U \leq 1$ und

$$\forall t > 0 : \quad \text{dbf}(t) = \sum_{i=1}^n \left\lfloor \frac{t + T_i - D_i}{T_i} \right\rfloor C_i \leq t.$$

4.8 Vergleich RM und EDF

Tabelle 4.2 fasst die wesentlichen Unterschiede zusammen:

Tabelle 4.2: Vergleich RM und EDF

Eigenschaft	RM	EDF
Priorität	Fest (umgekehrt proportional zu T_i)	Dynamisch (absolutes Deadline)
Optimalität	Schwach optimal (innerhalb fester Priorität)	Scharf optimal (insgesamt)
U_{lub}	$\ln 2 \approx 0,693$ (worst case)	1,000
Schedulierbarkeitstest	Pseudo-polynomiell (RTA)	$O(n)$ (Auslastungstest)
Überlastverhalten	Niederprioritäre Tasks blockieren	Periodenskalierung (Cervin)
Impl.-Aufwand	$O(1)$ (Multi-Level Queue)	$O(\log n)$ (Heap)
Präemptionsanzahl	Höher	Niedriger

Bemerkung 4.6. Dass RM schwach optimal ist, bedeutet: Wenn ein Task-Satz überhaupt mit *irgendeiner* festen Prioritätszuweisung schedulierbar ist, so ist er auch mit RM schedulierbar. Das schließt jedoch nicht aus, dass ein dynamischer Algorithmus wie EDF mehr Task-Mengen schedulieren kann.

EDF ist scharf optimal: EDF scheduliert genau die Task-Mengen, die überhaupt schedulierbar sind ($U \leq 1$). Es gibt keine Task-Menge mit $U \leq 1$, die EDF nicht schedulieren kann.

Kapitel 5

Fixed-Priority Server – Aperiodische Tasks unter RM

5.1 Einführung und Problemstellung

In realen Echtzeit-Systemen treten stets *hybride Task-Sets* auf: harte periodische Tasks (Sensor-Abfragen, Regelschleifen) müssen zusammen mit weichen aperiodischen Tasks (Benutzereingaben, Diagnoseereignisse) eingeplant werden.

Das zentrale Ziel dieses Kapitels: Für ein hybrides System sollen

1. die Deadlines aller harten periodischen Tasks garantiert werden, und
2. die Antwortzeiten der weichen aperiodischen Tasks minimiert werden – ohne die Garantien der periodischen Tasks zu gefährden.

Alle Algorithmen dieses Kapitels setzen voraus:

- Periodische Tasks werden unter Rate Monotonic (RM) eingeplant.
- Alle periodischen Tasks starten bei $t = 0$ mit $D_i = T_i$.
- Aperiodische Ankunftszeiten sind a priori unbekannt.

Grundidee Server: Ein *Server* ist eine künstliche periodische Task mit Periode T_s und Kapazität (Budget) C_s , die aperiodische Anfragen im Rahmen ihres Budgets bedient. Der Server konkurriert mit den echten periodischen Tasks um den Prozessor.

5.2 Background Scheduling – Einfachster Ansatz

5.2.1 Informelle Beschreibung

Aperiodische Tasks werden nur dann ausgeführt, wenn der Prozessor von keiner periodischen Task benötigt wird. Dies entspricht einer impliziten Priorität unterhalb aller periodischen Tasks.

Algorithm 1 Background Scheduling

```
1: Verwalte periodische Tasks in Hochprioritätswarteschlange (RM)
2: Verwalte aperiodische Tasks in Niedrigprioritätswarteschlange (FCFS)
3: loop
4:   if periodische Task bereit then
5:     Führe höchstpriorre periodische Task aus (RM)
6:   else if aperiodische Task bereit then
7:     Führe nächste aperiodische Task aus (FCFS)
8:   else
9:     Leerlauf
10:  end if
11: end loop
```

5.2.2 Korrektheit und Optimalitätsklassifikation

Satz 5.1 (Background Scheduling – Korrektheit). Unter Background Scheduling wird die Schedulierbarkeit aller periodischen Tasks durch RM nicht beeinflusst. Die aperiodischen Tasks erhalten genau die verbleibende Prozessorzeit $(1 - U_p)$.

Beweis. Aperiodische Tasks werden nur in Leerlaufzeiten ausgeführt. Da sie keine periodische Task preemptieren oder verzögern können, ist der Schedule für die periodischen Tasks identisch mit dem RM-Schedule ohne aperiodische Tasks. Die RM-Schedulierbarkeitsanalyse bleibt unverändert gültig. ■ □

Optimalitätsklassifikation: Background Scheduling ist bezüglich aperiodischer Antwortzeiten *nicht optimal*: Bei hoher periodischer Last $U_p \approx \ln 2$ bleibt nur $1 - \ln 2 \approx 0,307$ für aperiodische Tasks übrig, und diese Bandbreite wird ineffizient genutzt (Tasks müssen auf große Leerlaufintervalle warten).

Laufzeit: $O(1)$ pro Scheduling-Entscheidung (Vergleich zweier Warteschlangen-Kopfelemente).

5.3 Polling Server – Expliziter Server

5.3.1 Informelle Beschreibung

Der Polling Server (PS) ist eine periodische Task (τ_s, C_s, T_s) , die regelmäßig aktiviert wird und beim Aufwachen aperiodische Anfragen bedient – aber nur bis zur Kapazitätsgrenze C_s . Wenn keine aperiodischen Anfragen vorliegen, wird das Budget *verworfen*.

Algorithm 2 Polling Server

```
1: Server wird periodisch alle  $T_s$  Zeiteinheiten aktiviert
2: Setze  $c_s \leftarrow C_s$  ▷ Budget auffüllen
3: while  $c_s > 0$  und aperiodische Anfrage vorhanden do
4:   Bediene nächste aperiodische Anfrage für  $\min(C_{\text{ape}}, c_s)$  Einheiten
5:    $c_s \leftarrow c_s - \text{verbrauchte Zeit}$ 
6: end while
7: Verwerfe verbleibendes Budget  $c_s$ ; wird erst zur nächsten Periode aufgefüllt
```

5.3.2 Schedulierbarkeitsnachweis

Satz 5.2 (PS Schedulierbarkeit). Eine Menge von n periodischen Tasks mit Auslastung U_p und ein Polling Server mit Auslastung $U_s = C_s/T_s$ sind unter RM gemeinsam schedulierbar, wenn:

$$U_p + U_s \leq (n + 1) (2^{1/(n+1)} - 1).$$

Beweis. Der Polling Server verhält sich *exakt* wie eine periodische Task mit Periode T_s und WCET C_s : Unabhängig von der Anzahl der aperiodischen Anfragen verbraucht PS in jeder Serverperiode maximal C_s Zeiteinheiten. Damit ist $U_s = C_s/T_s$ die effektive Auslastung des Servers.

Das erweiterte System $\{\tau_1, \dots, \tau_n, \tau_s\}$ hat $n + 1$ Tasks mit Gesamtauslastung $U_p + U_s$. Nach dem Liu-Layland-Bound (Satz 4.2) ist dieses System unter RM schedulierbar, wenn $U_p + U_s \leq (n + 1)(2^{1/(n+1)} - 1)$. ■ □

Engerer Bound: Hyperbolic Test für PS

Mit dem Hyperbolic Bound (Theorem 4.3):

$$\prod_{i=1}^n (U_i + 1) \leq \frac{2}{U_s + 1}.$$

5.3.3 Worst-Case-Antwortzeit aperiodischer Anfragen

Im schlimmsten Fall muss eine aperiodische Anfrage bis zur nächsten Serveraktivierung warten (wenn der Server gerade sein Budget aufgebraucht hat oder keine Anfrage vorlag). Für eine Anfrage J_a mit $C_a \leq C_s$:

$$R_a^{\max} \leq 2T_s - \varepsilon + C_a.$$

Für beliebige C_a mit exakt $\lceil C_a/C_s \rceil$ Serverzugriffen:

$$R_a \leq T_s + \left\lceil \frac{C_a}{C_s} \right\rceil T_s.$$

5.3.4 Optimalitätsklassifikation und Laufzeit

Optimalitätsklassifikation: PS ist *schwach optimal* innerhalb der Klasse der periodischen Server unter RM: Bei vorgegebenem (C_s, T_s) garantiert PS dieselbe periodische Schedulierbarkeit wie jede äquivalente periodische Task. PS ist jedoch *nicht scharf optimal* bezüglich aperiodischer Antwortzeiten: Das Budget wird verworfen, wenn keine Anfragen vorliegen.

Nachteil: Eine aperiodische Anfrage, die kurz nach dem Verwerfen des Budgets ankommt, muss fast eine volle Serverperiode warten.

Laufzeit: $O(\log n)$ für Warteschlangenoperationen (RM-Heap), $O(1)$ für Budget-Verwaltung pro Aktivierung.

5.4 Deferrable Server – Kapazitätserhaltung

5.4.1 Informelle Beschreibung

Der Deferrable Server (DS) verbessert den PS, indem das Budget bei Inaktivität *nicht verworfen* wird, sondern bis zum Ende der Serverperiode erhalten bleibt. Kommt eine aperiodische Anfrage in der laufenden Periode an, steht das Budget sofort zur Verfügung.

Algorithm 3 Deferrable Server

- 1: $c_s \leftarrow 0$ ▷ Initialisierung
 - 2:
 - 3: **Zu jedem Serverperiodenbeginn:**
 - 4: $c_s \leftarrow C_s$ ▷ Budget auf Maximum auffüllen
 - 5:
 - 6: **Wenn aperiodische Anfrage J_a ankommt und $c_s > 0$:**
 - 7: Bediene J_a sofort (wenn keine höherprioritäre Task aktiv)
 - 8: $c_s \leftarrow c_s - \text{verbrauchte Zeit}$
 - 9:
 - 10: **Wenn Server aktiviert und $c_s > 0$, aber keine Anfrage:**
 - 11: Behalte Budget c_s (nicht werfen!)
 - 12: Server schläft bis zur nächsten Anfrage oder nächsten Periode
-

5.4.2 Warum DS KEINE äquivalente periodische Task ist

Der DS verletzt Annahme A5 (Tasks suspendieren sich nicht), da er sich suspendiert, obwohl er der höchstprioritäre Task wäre. Dies hat eine *niedrigere* Schedulierbarkeitsschranke zur Folge.

Satz 5.3 (DS Schedulierbarkeitsschranke – Strosnider, Lehoczky, Sha). Die Schedulierbarkeitsschranke für n periodische Tasks mit einem Deferrable Server mit Auslastung U_s ist:

$$U_p \leq n \left(\left(\frac{U_s + 2}{2U_s + 1} \right)^{1/n} - 1 \right).$$

Das Minimum über alle U_s ergibt $U_p^* \approx 0,652$ bei $U_s^* \approx 0,186$.

Beweis. Im schlimmsten Fall führt der DS dreimal innerhalb einer Periode T_1 der höchstprioritären periodischen Task aus: am Ende seiner eigenen Periode (aufgehoben) und zu Beginn der nächsten Periode. Daraus folgt die Worst-Case-Parameterisierung:

$$C_s = \frac{T_1 - T_s}{2}, \quad C_1 = T_2 - T_1, \dots, \quad C_n = \frac{3T_s + T_1 - 2T_n}{2}.$$

Minimierung der Auslastung über $R_i = T_{i+1}/T_i$ analog zum RM-Beweis (Abschnitt 4.2) ergibt:

$$K = \frac{U_s + 2}{2U_s + 1}, \quad U_{\text{lub}} = U_s + n(K^{1/n} - 1).$$

Da DS die periodischen Tasks stärker beeinträchtigt als PS (wegen der Kapazitätserhaltung), gilt $U_{\text{lub}}^{DS} \leq U_{\text{lub}}^{PS}$ für $U_s < 0,4$. ■ □

Vergleich PS vs. DS

Für $U_p = 0,6$: $U_s^{\max}(\text{PS}) = 2/P - 1$ und $U_s^{\max}(\text{DS}) = (2 - P)/(2P - 1)$ mit $P = \prod_i (U_i + 1)$. DS erlaubt typischerweise weniger Serverbandbreite als PS, liefert aber deutlich bessere aperiodische Antwortzeiten.

5.4.3 Aperiodische Antwortzeit unter DS

Sei $c_s(t)$ das verbleibende Budget zur Ankunftszeit r_a und $C_0 = \min(c_s(t), \text{next}(r_a) - r_a)$ die im aktuellen Intervall bedienbare Zeit:

$$R_a = \Delta_a + C_a - C_0 + F_a(T_s - C_s), \quad F_a = \left\lceil \frac{C_a - C_0}{C_s} \right\rceil.$$

Optimalitätsklassifikation: DS ist *nicht scharf optimal* bezüglich aperiodischer Antwortzeiten, aber besser als PS. Die niedrigere Schedulierbarkeitsschranke gegenüber PS ist der Preis für bessere aperiodische Responsivität.

Laufzeit: $O(\log n)$ pro Ereignis. Budgetverwaltung: $O(1)$ pro Aktivierung und Anfrage.

5.5 Priority Exchange Server

5.5.1 Informelle Beschreibung

Der Priority Exchange (PE) Server verbessert die Schedulierbarkeitsschranke gegenüber DS, indem Budget nicht einfach erhalten, sondern *getauscht* wird: Wenn keine aperiodischen Anfragen vorliegen, tauscht der Server seine Ausführungszeit gegen die Laufzeit der höchstpriorien aktiven periodischen Task. Das Budget wird dabei auf das Prioritätsniveau der getauschten Task degradiert.

Algorithm 4 Priority Exchange Server

```

1: Zu Beginn der Serverperiode:  $c_s \leftarrow C_s$  bei Serverpriorität
2:
3: Wenn Server höchste Priorität hat und  $c_s > 0$ :
4:   if aperiodische Anfrage vorhanden then
5:     Bediene Anfrage für  $\min(C_{\text{ape}}, c_s)$  Einheiten
6:   else if periodische Task  $\tau_k$  aktiv then
7:     Führe  $\tau_k$  für 1 Einheit aus (Tausch)
8:      $C_S^{d_k} \leftarrow C_S^{d_k} + 1$  (Budget auf  $\tau_k$ -Niveau)
9:      $c_s \leftarrow c_s - 1$  (Serverbudget reduzieren)
10:  else
11:    Leerlauf;  $c_s \leftarrow c_s - 1$ 
12:  end if

```

5.5.2 Schedulierbarkeitsnachweis

Satz 5.4 (PE Schedulierbarkeit). Ein Task-Set mit n periodischen Tasks und einem PE-Server mit Auslastung U_s ist unter RM schedulierbar, wenn:

$$U_p \leq n \left(\left(\frac{2}{U_s + 1} \right)^{1/n} - 1 \right).$$

Diese Schranke ist *identisch* mit der des Polling Servers (Satz 5.2).

Beweis. Im schlimmsten Fall verhält sich PE wie eine periodische Task mit Periode T_s und Ausführungszeit C_s : Die Tauschoperationen verschieben Rechenzeit nur zwischen Prioritätsniveaus, ändern aber nicht die Gesamtausführungszeit. Das erweiterte System hat damit dieselbe effektive Auslastungsstruktur wie PS, und der Liu-Layland-Bound gilt analog. ■ □

5.5.3 Optimalitätsklassifikation und Vergleich

PE ist damit gegenüber DS *besser schedulierbar*, aber auf Kosten komplexerer Implementierung. Für eine gegebene periodische Last U_p :

- PE erlaubt mehr Serverbandbreite als DS.
- Die aperiodischen Antwortzeiten von PE sind vergleichbar mit DS.

Laufzeit: $O(\log n)$ pro Ereignis, zusätzlich $O(1)$ für Tauschoperationen. Die Tracking-Verwaltung mehrerer Budget-Niveaus erfordert $O(n)$ Speicher.

5.6 Sporadic Server – Optimale Schedulierbarkeits-schranke unter RM

5.6.1 Informelle Beschreibung

Der Sporadic Server (SS), von Sprunt, Sha und Lehoczky, hat die Schlüsseleigenschaft: *Das verbrauchte Budget wird erst T_s Zeiteinheiten nach dem Verbrauch wiederhergestellt.* Dadurch verhält sich SS – im Gegensatz zu DS – exakt wie eine normale periodische Task in Bezug auf die Schedulierbarkeitsschranke.

Replenishment-Regeln:

- **Wiederauffüllzeitpunkt** RT: Wird gesetzt, wenn SS aktiv wird ($P_{\text{exe}} \geq P_s$) und $c_s > 0$. Sei t_A dieser Zeitpunkt: $RT = t_A + T_s$.
- **Wiederauffüllmenge** RA: Wird gesetzt, wenn SS idle wird ($P_{\text{exe}} < P_s$) oder $c_s = 0$. Sei t_I dieser Zeitpunkt: $RA = \text{verbrauchte Kapazität in } [t_A, t_I]$.
- Zum Zeitpunkt RT: $c_s \leftarrow c_s + RA$.

Algorithm 5 Sporadic Server

```
1:  $c_s \leftarrow C_s$ ;  $RT \leftarrow \infty$ ;  $t_A \leftarrow -\infty$ 
2:
3: Wenn SS aktiv wird ( $P_{\text{exe}} \geq P_s$ ) und  $c_s > 0$ :
4: if  $RT = \infty$  then
5:    $t_A \leftarrow$  aktueller Zeitpunkt;  $RT \leftarrow t_A + T_s$ 
6: end if
7:
8: Wenn SS idle wird ( $P_{\text{exe}} < P_s$ ) oder  $c_s = 0$ :
9:  $RA \leftarrow c_s(t_A) - c_s(t_I)$  ▷ Verbrauchtes Budget in  $[t_A, t_I]$ 
10: Plane Wiederauffüllung RA zu Zeitpunkt RT
11:  $RT \leftarrow \infty$  ▷ Nächster Aktivierungszeitpunkt wird neu gesetzt
12:
13: Zum geplanten Wiederauffüllzeitpunkt RT:
14:  $c_s \leftarrow c_s + RA$ 
```

5.6.2 Hauptsatz: SS verhält sich wie eine periodische Task

Theorem 5.1 (Sprunt, Sha, Lehoczky – SS Äquivalenz). Ein Task-Set, das mit einer periodischen Task $\tau_s(C_s, T_s)$ schedulierbar ist, bleibt schedulierbar, wenn τ_s durch einen Sporadic Server mit denselben Parametern (C_s, T_s) ersetzt wird.

Beweis. Wir analysieren das Verhalten von SS in den drei möglichen Fällen (Abbildung nach Buttazzo [1]):

Fall 1 – Kein Verbrauch in $[t_A, t_I]$: In diesem Intervall ist keine aperiodische Anfrage aktiv. SS behält sein Budget, aber keine Wiederauffüllung findet vor $t_I + T_s$ statt. Das bedeutet: In $[t_A, t_I + T_s]$ kann SS maximal C_s Zeiteinheiten für aperiodische Tasks verbrauchen – identisch mit einer periodischen Task τ_s mit verzögertem Release-Zeitpunkt t_I statt t_A . Da verzögerte Releases die Antwortzeiten anderer Tasks nicht verschlechtern, bleibt Schedulierbarkeit erhalten.

Fall 2 – Vollständiger Verbrauch: c_s wird in $[t_A, t_I]$ vollständig verbraucht ($RA = C_s$). Wiederauffüllung bei $t_A + T_s$. SS verhält sich exakt wie τ_s mit Release-Zeit t_A .

Fall 3 – Teilweiser Verbrauch: $RA = C_R$ (verbrauchter Teil) wird bei $t_A + T_s$ wiederaufgefüllt; verbleibendes Budget $C_s - C_R$ bleibt erhalten. SS verhält sich wie zwei periodische Tasks $\tau_x(C_R, T_s)$ und $\tau_y(C_s - C_R, T_s)$, wobei τ_x bei t_A und τ_y bei t_I released wird. Da τ_y verzögert ist, schadet es der Schedulierbarkeit nicht (analog zu Fall 1).

In allen Fällen ist der Beitrag von SS zur Auslastung $U_s = C_s/T_s$, genau wie eine periodische Task. ■ □

5.6.3 Schedulierbarkeitsanalyse

Satz 5.5 (SS Schedulierbarkeitsschranke). Eine Menge von n periodischen Tasks mit Auslastung U_p und ein Sporadic Server mit Auslastung U_s sind unter RM schedulierbar, wenn:

$$U_p + U_s \leq (n + 1) (2^{1/(n+1)} - 1).$$

Mit dem Hyperbolic Bound:

$$\prod_{i=1}^n (U_i + 1) \leq \frac{2}{U_s + 1}.$$

Die maximale Serverauslastung, die die Schedulierbarkeit erhält, ist:

$$U_s^{\max} = \frac{2 - P}{P}, \quad P = \prod_{i=1}^n (U_i + 1).$$

Beweis. Aus Theorem 5.1 folgt, dass SS wie eine periodische Task mit $U_s = C_s/T_s$ behandelt werden kann. Das erweiterte System $\{\tau_1, \dots, \tau_n, \tau_s\}$ hat $n + 1$ Tasks; der Hyperbolic Bound (Theorem 4.3) für $n + 1$ Tasks lautet $\prod_{i=1}^{n+1} (U_i + 1) \leq 2$. Einsetzen von $U_{n+1} = U_s$ liefert die Behauptung. ■ □

5.6.4 Optimalitätsklassifikation und Laufzeit

Optimalitätsklassifikation: SS ist *schwach optimal* für den Klasse fester Prioritätsserver: Es gibt keinen Server unter RM, der eine höhere Schedulierbarkeitsschranke *und* bessere aperiodische Antwortzeiten gleichzeitig bietet (Theorem 5.2).

Laufzeit: Jede Ereignisbehandlung (Serveraktivierung, Idle-Wechsel, Wiederauffüllung) erfordert $O(1)$ Operationen. Pro aperiodischer Anfrage entstehen höchstens $\lceil C_a/C_s \rceil$ Wiederauffüllungen, daher Gesamtoverhead pro Anfrage $O(\lceil C_a/C_s \rceil)$.

5.7 Slack Stealing – Maximale Responsivität

5.7.1 Informelle Beschreibung

Der Slack Stealing Algorithmus (Lehoczky & Ramos-Thuel 1992) nutzt nicht eine feste Serverbandbreite, sondern “stiehlt” alle verfügbaren Slack-Zeiten der periodischen Tasks für aperiodische Anfragen.

Grundidee: Wenn τ_i eine Slack-Zeit $\text{slack}_i(t) > 0$ hat, bedeutet das, dass sie nicht *sofort* ausgeführt werden muss. Der Slack Stealer kann diese Zeit für aperiodische Anfragen nutzen, ohne die Deadline von τ_i zu gefährden.

Algorithm 6 Slack Stealing (vereinfachte Darstellung)

- 1: Berechne offline $A(0, t) = \text{Slack-Funktion}$ für alle $t \in [0, H]$
 - 2:
 - 3: **Wenn aperiodische Anfrage J_a bei $t = r_a$ ankommt:**
 - 4: Berechne aktuellen Slack $A(r_a, t')$ für $t' > r_a$
 - 5: Finde frühestes t' mit $A(r_a, t') \geq C_a$
 - 6: Plane J_a so früh wie möglich innerhalb verfügbarer Slack-Zeiten
 - 7:
 - 8: **Wenn keine aperiodische Anfrage:**
 - 9: Normale RM-Einplanung der periodischen Tasks
-

5.7.2 Slack-Funktion

Definition 5.1 (Slack-Funktion $A(s, t)$). $A(s, t)$ ist die maximale Rechenzeitdauer, die im Intervall $[s, t]$ aperiodischen Tasks zugewiesen werden kann, ohne dass periodische Tasks ihre Deadlines verfehlen:

$$A(s, t) = (t - s) - \sum_{i=1}^n \left\lceil \frac{t - s}{T_i} \right\rceil C_i.$$

$A(s, \cdot)$ ist eine nichtfallende Treppenfunktion mit Sprüngen an den Zeitpunkten, an denen Slack entsteht.

Satz 5.6 (Slack Stealing – Schedulierbarkeit). Wenn das periodische Task-Set unter RM schedulierbar ist ($U_p \leq U_{\text{lub}}$), gefährdet Slack Stealing die Schedulierbarkeit nie: Aperiodische Tasks werden ausschließlich in Zeiten eingeplant, in denen kein periodischer Task laufen muss.

Beweis. Per Konstruktion: $A(s, t)$ gibt exakt die Zeitdauer an, die für periodische Tasks *nicht* benötigt wird. Jede aperiodische Zuteilung innerhalb von $A(s, t)$ lässt mindestens genug Prozessorzeit für alle periodischen Tasks bis zu ihren Deadlines. ■ □

5.7.3 Nicht-Existenz optimaler Server – Fundamentales Ergebnis

Theorem 5.2 (Tia, Liu, Shankar 1995 – Kein optimaler Server). Für jeden festen Prioritätsscheduler und jede Ankunftsdisziplin für aperiodische Tasks existiert kein Online-Algorithmus, der die Antwortzeit *jeder einzelnen* aperiodischen Anfrage minimiert.

Beweis. Durch Gegenbeispiel: Betrachte τ_1 ($C = 1, T = 3$), τ_2 ($C = 1, T = 4$), τ_3 ($C = 1, T = 6$) unter RM. Slack tritt bei $t = 2$ auf.

Ankunft J_1 bei $t = 2$ ($C = 1$):

- Option A: J_1 sofort (bei $t = 2$) einplanen $\Rightarrow J_1$ fertig bei $t = 3$, minimale Antwortzeit für J_1 .
- Option B: J_1 auf $t = 3$ verschieben $\Rightarrow J_1$ fertig bei $t = 4$.

Ankunft J_2 bei $t = 3$ ($C = 1$):

- Bei Option A (A sofort): Kein Slack in $[3, 6]$; J_2 frühestens bei $t = 7$.
- Bei Option B (A verschoben): Slack bei $t = 4$; J_2 fertig bei $t = 5$.

Option A minimiert J_1 , aber nicht J_2 . Option B minimiert J_2 , aber nicht J_1 . Kein Algorithmus kann beide gleichzeitig minimieren. ■ □

Laufzeit Slack Stealing: Die Offline-Berechnung der Slack-Funktion hat Komplexität $O(N)$ mit $N = \text{Anzahl Deadlines in der Hyperperiode}$. Die Online-Aktualisierung bei jeder aperiodischen Ankunft hat Komplexität $O(n)$.

5.8 Dimensionierung von Servern

Satz 5.7 (Optimale Server-Dimensionierung). Für ein gegebenes periodisches Task-Set mit Hyperbolic-Produkt $P = \prod_i (U_i + 1)$ sind die maximalen Serverauslastungen:

$$U_s^{\max}(\text{PS/PE/SS}) = \frac{2 - P}{P},$$

$$U_s^{\max}(\text{DS}) = \frac{2 - P}{2P - 1}.$$

Die optimale Serverperiode (beste Antwortzeit bei gegebenem U_s) ist $T_s = T_1$ (höchste Priorität), mit $C_s = U_s \cdot T_s$.

5.9 Zusammenfassender Vergleich der Fixed-Priority Server

Tabelle 5.1: Vergleich der Fixed-Priority Server – Vollständige Übersicht

Server	Budget-Mgmt.	Sched.-Schranke	Antwortzeit	Overhead
Background	Keine	Wie RM ($U_p \leq U_{\text{lub}}$)	Schlecht	$O(1)$
Polling (PS)	Verwerfen	Wie RM+1 Task	Mittel	$O(\log n)$
Deferrable (DS)	Erhalten	Niedriger als PS!	Gut	$O(\log n)$
Priority Exch. (PE)	Tauschen	Wie PS	Gut	$O(n \log n)$
Sporadic (SS)	Verzög. Auffüllen	Wie PS	Gut	$O(\log n)$
Slack Stealing	Slack aller Tasks	Maximal	Opt.*	$O(N)$

*Kein optimaler Server existiert (Thm. 5.2). Slack Stealing minimiert Antwortzeiten soweit möglich.

Kapitel 6

Dynamic Priority Server – Dynamische Prioritäts-Server unter EDF

6.1 Einführung und Motivation

Gegenüber Fixed-Priority-Servern aus Kapitel 5 bieten dynamische Prioritäts-Server einen entscheidenden Vorteil: Da EDF den Prozessor bis $U = 1$ auslasten kann, steht für aperiodische Tasks eine weit größere Bandbreite zur Verfügung. Für eine Menge periodischer Tasks mit Auslastung U_p verbleiben $(1 - U_p)$ für aperiodische Server – gegenüber deutlich weniger unter RM.

Bemerkung 6.1 (Vorteil dynamischer Server gegenüber RM-basierten Servern). Beispiel: $U_1 = U_2 = 0,3$, also $U_p = 0,6$.

- Unter RM+Sporadic Server: $U_s^{\max} = 2/P - 1$ mit $P = (0,3 + 1)^2 = 1,69$, also $U_s^{\max} \approx 0,18$.
- Unter EDF: $U_s^{\max} = 1 - U_p = 0,40$.

EDF erlaubt damit mehr als doppelt so viel Bandbreite für aperiodische Tasks.

Alle in diesem Kapitel vorgestellten Server arbeiten unter folgenden Annahmen:

- Periodische Tasks τ_i haben harte Deadlines mit $D_i = T_i$.
- Aperiodische Tasks J_i haben keine Deadlines, sollen aber so früh wie möglich fertiggestellt werden.
- Alle periodischen Tasks starten synchron bei $t = 0$.
- Die Ausführungszeit jeder aperiodischen Task ist bekannt.

6.2 Dynamic Priority Exchange Server (DPE)

6.2.1 Grundidee und Algorithmus

Der Dynamic Priority Exchange Server (DPE), entwickelt von Spuri und Buttazzo [25], überträgt das Prinzip des Fixed-Priority-Exchange-Servers auf EDF: Der Server tauscht

seine Laufzeit mit niederprioren periodischen Tasks (d. h. solchen mit späterer Deadline), anstatt sie zu verbrauchen. Dadurch bleibt Kapazität erhalten, auch wenn keine aperiodischen Anfragen vorliegen.

Informelle Beschreibung:

- Der DPE-Server hat eine Periode T_s und Kapazität C_s .
- Zu Beginn jeder Serverperiode mit Deadline d wird die aperiodische Kapazität $C_S^d \leftarrow C_s$ gesetzt.
- Wenn die höchste Priorität im System eine aperiodische Kapazität C hat, wird entweder eine aperiodische Task (falls vorhanden) oder die periodische Task mit kürzester Deadline ausgeführt. Im letzteren Fall wird deren Laufzeit in eine aperiodische Kapazität auf dem Prioritätsniveau der periodischen Task übertragen (Tausch).
- Leerlauf verbraucht die Kapazität bis auf null.

Algorithm 7 DPE Server – Ereignisbehandlung

```

1: Bei Serverperiodenbeginn mit Deadline  $d$ :
2:  $C_S^d \leftarrow C_s$ 
3:
4: Bei Ausführung der höchsten Priorität (aperiodische Kapazität  $C$ ):
5: if aperiodische Anfrage  $J$  vorhanden then
6:   Bediene  $J$  bis Fertigstellung oder Kapazität  $C$  erschöpft
7: else if periodische Task  $\tau_k$  mit frühester Deadline bereit then
8:   Führe  $\tau_k$  für 1 Zeiteinheit aus
9:    $C_S^{d_k} \leftarrow C_S^{d_k} + 1$ ;  $C \leftarrow C - 1$  ▷ Tausch: Kapazität auf Niveau  $d_k$  übertragen
10: else
11:   Leerlauf;  $C \leftarrow C - 1$ 
12: end if

```

6.2.2 Schedulierbarkeitsnachweis – Korrektheit

Theorem 6.1 (Spuri & Buttazzo – DPE Schedulierbarkeit). Gegeben eine Menge periodischer Tasks mit Auslastung U_p und ein DPE-Server mit Auslastung U_s , ist das gesamte System unter EDF genau dann schedulierbar, wenn

$$U_p + U_s \leq 1.$$

Beweis. **Konstruktion des äquivalenten Schedules σ' :**

Sei σ ein von DPE produzierter Schedule. Wir konstruieren σ' wie folgt:

- Ersetze den DPE-Server durch eine periodische Task τ_s mit Periode T_s und WCET C_s : In σ' läuft τ_s genau dann, wenn in σ Serverkapazität verbraucht wird.
- Periodische Instanzen, die in σ wegen Deadline-Tausch vorgezogen wurden, werden in σ' nach hinten verschoben.

Eigenschaft von σ' : Das resultierende σ' ist das EDF-Schedule für die Menge $\{\tau_1, \dots, \tau_n, \tau_s\}$ mit Gesamtauslastung $U_p + U_s$ – unabhängig von aperiodischen Anfragen.

Äquivalenz $\sigma \leftrightarrow \sigma'$: In σ sind die Fertigstellungszeiten jeder periodischen Instanz $\leq$ denen in σ' , da DPE Instanzen vorziehen kann. Also: σ' machbar $\Rightarrow \sigma$ machbar. Umgekehrt ist σ' der DPE-Schedule ohne aperiodische Anfragen, also gilt auch: σ machbar $\Rightarrow \sigma'$ machbar.

Da σ' ein reines EDF-Schedule über periodische Tasks mit Auslastung $U_p + U_s$ ist, gilt nach Theorem 4.5: σ' machbar $\iff U_p + U_s \leq 1$. ■ □

6.2.3 Laufzeitanalyse

Satz 6.1 (DPE Laufzeit). Die Laufzeit von DPE pro Ereignis (Ankunft, Fertigstellung, Kapazitätserschöpfung) beträgt $O(\log n)$ für die Verwaltung der EDF-Prioritätswarteschlange als Heap. Pro Serverperiode entsteht ein Overhead von $O(C_s)$ durch die Tauschoperationen, da jede Zeiteinheit maximal eine Tauschoperation erzeugt.

6.2.4 Optimalitätsklassifikation von DPE

DPE ist **nicht scharf optimal** bezüglich der Antwortzeit aperiodischer Tasks: Es gibt Szenarien, in denen ein clairvoyanter Algorithmus bessere Antwortzeiten liefern würde. DPE ist jedoch **schwach optimal** in dem Sinne, dass für jeden Auslastungswert $U_p + U_s \leq 1$ die Schedulierbarkeit aller periodischen Tasks garantiert wird – genau wie ein äquivalenter rein-periodischer EDF-Schedule.

6.3 Dynamic Sporadic Server (DSS)

6.3.1 Grundidee und Algorithmus

Der Dynamic Sporadic Server (DSS), ebenfalls von Spuri und Buttazzo [25], erweitert den Sporadic Server auf dynamische Prioritäten. Die Kapazität C_s wird nicht zu festen Zeiten aufgefüllt, sondern nach dem Verbrauch.

Deadline- und Wiederauffüllungsregeln:

- Wenn der Server aktiv wird ($C_s > 0$ und aperiodische Anfrage vorhanden) zum Zeitpunkt t_A : Setze $RT = d_s = t_A + T_s$.
- Wenn der Server idle wird (Anfrage abgeschlossen oder $C_s = 0$) zum Zeitpunkt t_I : Plane Wiederauffüllung $RA = (\text{verbrauchte Kapazität in } [t_A, t_I])$ zu Zeitpunkt RT .

Algorithm 8 DSS Server – Kapazitätsverwaltung

- 1: **Initialisierung:** $c_s \leftarrow C_s$, $d_s \leftarrow \infty$
 - 2:
 - 3: **Wenn aperiodische Anfrage J ankommt und $c_s > 0$:**
 - 4: $t_A \leftarrow$ aktueller Zeitpunkt
 - 5: $RT \leftarrow t_A + T_s$; $d_s \leftarrow t_A + T_s$
 - 6: Bediene J mit Deadline d_s ; decrementiere c_s bei Ausführung
 - 7:
 - 8: **Wenn Server idle wird (Zeitpunkt t_I):**
 - 9: $RA \leftarrow$ verbrauchte Kapazität in $[t_A, t_I]$
 - 10: Plane Wiederauffüllung $c_s += RA$ bei Zeitpunkt RT
-

6.3.2 Schedulierbarkeitsnachweis

Lemma 6.1 (DSS verhält sich wie eine periodische Task). In jedem Zeitintervall $[t_1, t_2]$, in dem t_1 der Aktivierungszeitpunkt des Servers ist, gilt:

$$C_{\text{ape}}(t_1, t_2) \leq \left\lfloor \frac{t_2 - t_1}{T_s} \right\rfloor C_s.$$

Beweis. Da die Wiederauffüllungsmenge stets gleich der verbrauchten Menge ist und das Wiederauffüllen frühestens T_s nach dem Aktivierungszeitpunkt erfolgt, kann in jedem T_s -langen Intervall maximal C_s Zeit für aperiodische Tasks verbraucht werden. Die Behauptung folgt durch Induktion. ■ □

Theorem 6.2 (Spuri & Buttazzo – DSS Schedulierbarkeit). Gegeben eine Menge periodischer Tasks mit U_p und ein DSS mit U_s , ist das System schedulierbar genau dann, wenn $U_p + U_s \leq 1$.

Beweis. Hinreichend ($\Rightarrow$): Angenommen, $U_p + U_s \leq 1$, aber es gibt einen Überlauf bei t . Dann gibt es ein $t' < t$ mit kontinuierlicher Auslastung. Für die Gesamtanforderung C in $[t', t]$ gilt nach Lemma 6.1:

$$C \leq \sum_{i=1}^n \left\lfloor \frac{t - t'}{T_i} \right\rfloor C_i + C_{\text{ape}} \leq (t - t')U_p + (t - t')U_s = (t - t')(U_p + U_s) \leq (t - t').$$

Dies widerspricht $t - t' < C$. Widerspruch.

Notwendig ($\Leftarrow$): Da DSS sich wie eine periodische Task mit U_s verhält, folgt aus dem EDF-Schedulierbarkeitstheorem direkt $U_p + U_s \leq 1$. ■ □

6.3.3 Laufzeitanalyse und Optimalitätsklassifikation

Satz 6.2 (DSS Laufzeit). Jede Ereignisbehandlung (Ankunft, Fertigstellung, Wiederauffüllung) erfordert $O(\log n)$ für Heap-Operationen. Pro aperiodischer Anfrage entstehen maximal $\lceil C_a/C_s \rceil$ Wiederauffüllungen, sodass der Gesamtoverhead pro Anfrage in $O(\lceil C_a/C_s \rceil \cdot \log n)$ liegt.

DSS ist wie DPE **nicht scharf optimal** bezüglich Antwortzeiten, aber **schedulierbar-korrekt**: Periodische Tasks werden genau dann garantiert, wenn $U_p + U_s \leq 1$.

6.4 Total Bandwidth Server (TBS)

6.4.1 Grundidee und Algorithmus

Der Total Bandwidth Server (TBS) ist der einfachste und effizienteste der hier vorgestellten Server [25]. Statt eine eigene Periode zu verwalten, weist er jeder aperiodischen Anfrage eine virtuelle Deadline zu, die die gesamte zugeordnete Bandbreite U_s sofort zuteilt.

Deadlinezuweisung: Wenn die k -te aperiodische Anfrage zum Zeitpunkt r_k mit Ausführungszeit C_k ankommt:

$$d_k = \max(r_k, d_{k-1}) + \frac{C_k}{U_s},$$

wobei $d_0 = 0$. Die Anfrage wird dann wie eine periodische Instanz mit Deadline d_k in die EDF-Warteschlange eingereiht.

Algorithm 9 Total Bandwidth Server

Require: Serverauslastung U_s , initiale Deadline $d_0 \leftarrow 0$

- 1:
 - 2: **Bei Anknunft der k -ten Anfrage** (r_k, C_k):
 - 3: $d_k \leftarrow \max(r_k, d_{k-1}) + C_k/U_s$
 - 4: Füge (C_k, d_k) in EDF-Warteschlange ein
 - 5: EDF-Schedule läuft weiter
-

Beispiel 6.1 (TBS Deadlinezuweisung). $U_p = 3/4$, $U_s = 1/4$. Periodische Tasks τ_1 ($C_1 = 3, T_1 = 6$), τ_2 ($C_2 = 2, T_2 = 8$).

Aperiodische Anfragen:

- J_1 : $r_1 = 3$, $C_1 = 1$. $d_1 = 3 + 1/0,25 = 7$.
- J_2 : $r_2 = 9$, $C_2 = 1$. $d_2 = \max(9, 7) + 1/0,25 = 13$.
- J_3 : $r_3 = 14$, $C_3 = 1$. $d_3 = \max(14, 13) + 1/0,25 = 18$.

Jede Anfrage wird als EDF-Instanz mit den genannten Deadlines eingeplant.

6.4.2 Korrektheitsbeweis: Auslastungsbeschränkung

Lemma 6.2 (TBS Auslastungsschranke). In jedem Intervall $[t_1, t_2]$ ist die von TBS bediente Ausführungszeit C_{ape} durch $(t_2 - t_1) \cdot U_s$ nach oben beschränkt:

$$C_{\text{ape}} = \sum_{t_1 \leq r_k, d_k \leq t_2} C_k \leq (t_2 - t_1)U_s.$$

Beweis. Seien $J_{k_1}, \dots, J_{k_2}$ die aperiodischen Tasks, die zu $[t_1, t_2]$ beitragen (d. h. $r_{k_j} \geq t_1$ und $d_{k_j} \leq t_2$). Aus der TBS-Deadlineregeln folgt durch Teleskopsumme:

$$\sum_{k=k_1}^{k_2} C_k = \sum_{k=k_1}^{k_2} [d_k - \max(r_k, d_{k-1})]U_s \leq [d_{k_2} - \max(r_{k_1}, d_{k_1-1})]U_s \leq (t_2 - t_1)U_s.$$

Hierbei wird genutzt, dass jede Klammer $[d_k - \max(\cdot)] \leq d_k - d_{k-1}$ und die Summe teleskopisch kollabiert. ■ □

Theorem 6.3 (Spuri & Buttazzo – TBS Schedulierbarkeit). Gegeben periodische Tasks mit U_p und ein TBS mit U_s , ist das System unter EDF genau dann schedulierbar, wenn $U_p + U_s \leq 1$.

Beweis. Hinreichend: Angenommen $U_p + U_s \leq 1$, aber es gibt einen Überlauf bei t . Sei $[t', t]$ das längste Intervall kontinuierlicher Auslastung vor t , in dem nur Tasks mit Deadline $\leq t$ ausgeführt werden. Die Gesamtanforderung:

$$C \leq \sum_i \left\lfloor \frac{t - t'}{T_i} \right\rfloor C_i + C_{\text{ape}} \leq (t - t')U_p + (t - t')U_s = (t - t')(U_p + U_s) \leq t - t'.$$

Widerspruch zu $t - t' < C$.

Notwendig: Wenn aperiodische Anfragen periodisch mit Periode T_s und Ausführungszeit $C_s = T_s U_s$ ankommen, verhält sich TBS wie eine periodische Task. Dann folgt aus EDF-Schedulierbarkeit $U_p + U_s \leq 1$. ■ □

6.4.3 Optimalitätsklassifikation von TBS

TBS ist **nicht scharf optimal** im Sinne minimaler Antwortzeiten: Es kann aperiodische Anfragen mit nichtminimaler Antwortzeit einplanen. Es gibt jedoch eine Erweiterung TB* (Total Bandwidth, optimiert), die durch iterative Deadline-Verkürzung die minimale Antwortzeit annähert.

Satz 6.3 (TB* Optimalität – Buttazzo & Sensini). Sei σ ein machbarer EDF-Schedule und f_k die Fertigstellungszeit einer aperiodischen Anfrage J_k mit Deadline d_k . Wenn $f_k = d_k$, dann ist $f_k = f_k^*$ (minimale erreichbare Fertigstellungszeit).

Laufzeit von TBS: $O(1)$ pro Ankunft (Deadlineberechnung) plus $O(\log n)$ für EDF-Heap-Einfügung. Gesamtoverhead vernachlässigbar.

6.5 Constant Bandwidth Server (CBS)

6.5.1 Definition und Motivation

Der Constant Bandwidth Server (CBS) von Abeni und Buttazzo [1] ist die robusteste der vorgestellten Techniken: Er garantiert temporale Isolation, d. h. eine Task kann den Server nicht über seine zugewiesene Bandbreite U_s hinaus belasten – selbst wenn sie mehr Rechenzeit beansprucht als erwartet. Dies unterscheidet CBS grundlegend von TBS, der die Überschreitung von WCETs nicht abfangen kann.

Definition 6.1 (CBS-Parameter und Zustandsvariablen). Ein CBS ist durch ein Maximum-Budget Q_s und eine Periode T_s mit Bandbreite $U_s = Q_s/T_s$ charakterisiert. Zustandsvariablen:

- c_s : aktuelles Budget ($0 < c_s \leq Q_s$ stets)
- $d_{s,k}$: aktuelle Serverdeadline
- k : Zähler der erzeugten Chunks

Initialisierung: $d_{s,0} \leftarrow 0$, $c \leftarrow 0$, $n \leftarrow 0$, $k \leftarrow 0$.

6.5.2 CBS-Algorithmus (Pseudocode)

Algorithm 10 Constant Bandwidth Server (CBS)

```

1: Bei Ankunft von Job  $J_j$  zum Zeitpunkt  $r_j$ :
2:  $n \leftarrow n + 1$ ; füge  $J_j$  in Serverwarteschlange ein
3: if  $n = 1$  then ▷ Server war idle
4:   if  $r_j + (c/Q_s) \cdot T_s \geq d_{s,k}$  then ▷ Regel 1: neues Deadline
5:      $k \leftarrow k + 1$ ;  $a_k \leftarrow r_j$ ;  $d_{s,k} \leftarrow r_j + T_s$ ;  $c \leftarrow Q_s$ 
6:   else ▷ Regel 2: alte Deadline
7:      $k \leftarrow k + 1$ ;  $a_k \leftarrow r_j$ ;  $d_{s,k} \leftarrow d_{s,k-1}$ 
8:   end if
9: end if
10:
11: Bei Fertigstellung von  $J_j$ :
12:  $n \leftarrow n - 1$ 
13: if  $n \neq 0$  then starte nächsten Job mit Deadline  $d_{s,k}$ 
14: end if
15:
16: Bei jeder Ausführungseinheit ( $c = c - 1$ ):
17: if  $c = 0$  then ▷ Regel 3: Budget erschöpft
18:    $k \leftarrow k + 1$ ;  $a_k \leftarrow$  aktueller Zeitpunkt
19:    $d_{s,k} \leftarrow d_{s,k-1} + T_s$ ;  $c \leftarrow Q_s$ 
20: end if

```

6.5.3 Isolationseigenschaft – formaler Beweis

Theorem 6.4 (CBS Isolationseigenschaft – Abeni & Buttazzo). Die CPU-Auslastung eines CBS S mit Parametern (Q_s, T_s) beträgt exakt $U_s = Q_s/T_s$, unabhängig von Ausführungszeiten und Ankunftsmodellen.

Beweis. Durch Regel 3 wird das Budget c_s bei jedem Verbrauch sofort auf Q_s aufgefüllt und die Serverdeadline um T_s nach hinten verschoben. In jedem Intervall der Länge T_s verbraucht der CBS genau Q_s Zeiteinheiten für Serverausführung. Die Ausführungsrate ist damit konstant $Q_s/T_s = U_s$.

Formaler: Betrachte zwei aufeinanderfolgende Serverdeadlines $d_{s,k}$ und $d_{s,k+1} = d_{s,k} + T_s$. In $[d_{s,k-1}, d_{s,k}]$ wird genau Q_s Budget verbraucht (sonst würde Regel 3 nicht auslösen). Damit ist die Bandbreite exakt Q_s/T_s für jeden Beobachtungszeitraum. ■ □

Lemma 6.3 (Schedulierbarkeit unter CBS). Gegeben periodische harte Tasks mit U_p und m CBS-Server mit Gesamtbandbreite $U_s = \sum_{i=1}^m U_{s,i}$, ist das System unter EDF genau dann schedulierbar, wenn $U_p + U_s \leq 1$.

Beweis. Aus Theorem 6.4 folgt, dass jeder CBS sich wie eine periodische Task mit Auslastung $U_{s,i}$ verhält. Die Gesamtauslastung beträgt $U_p + \sum_i U_{s,i} = U_p + U_s$, und EDF-Schedulierbarkeit gilt genau dann, wenn dieser Wert ≤ 1 . ■ □

6.5.4 Worst-Case-Antwortzeit und optimale CBS-Parameter

Die worst-case Antwortzeit R_i eines Jobs mit Ausführungszeit C_i unter CBS mit Budget Q_s und Periode T_s beträgt:

$$R_i = C_i + \left\lfloor \frac{C_i}{Q_s} \right\rfloor (T_s - Q_s).$$

Das Budget Q_s , das die mittlere Antwortzeit minimiert (unter Berücksichtigung eines Kontext-Switch-Overheads ϵ), ergibt sich zu:

$$T_s^{\text{opt}} = \frac{1}{U_s} \left(\epsilon + \sqrt{\frac{\epsilon \cdot C^{\text{avg}}}{1 - U_s}} \right).$$

6.5.5 Vergleich der dynamischen Server

Tabelle 6.1: Vergleich dynamischer Prioritäts-Server

Server	Sched. korrekt	Antwort- zeit	Overhead	Besonderheit
DPE	$U_p + U_s \leq 1$	Gut	Mittel	Deadline-Tausch
DSS	$U_p + U_s \leq 1$	Gut	Mittel	Dynamische Wiederbefüllung
TBS	$U_p + U_s \leq 1$	Sehr gut	Minimal	Einfachste Impl.
EDL	$U_p \leq 1$	Optimal	Hoch	Idle-Zeit-Ausnutzung
IPE	$U_p \leq 1$	Nahezu opt.	Mittel	Vorberechnete Zeiten
CBS	$U_p + U_s \leq 1$	Gut	Gering	Temporale Isolation

6.6 EDL-Server – Optimale aperiodische Antwortzeiten

Der Earliest Deadline Late (EDL) Server nutzt die Leerlaufzeiten eines EDL-Schedules (EDF-Variante, in der Tasks so spät wie möglich eingeplant werden) für aperiodische Anfragen.

Theorem 6.5 (Chetto & Chetto – EDL Optimalität). Für jede Online-Algorithmus A und jeden aperiodischen Job J_i gilt:

$$f^{\text{EDL-Server}}(J_i) \leq f^A(J_i).$$

Der EDL-Server minimiert die Fertigstellungszeit jeder aperiodischen Anfrage.

Beweisidee. Da EDL in jedem Intervall $[0, t]$ die maximale Leerlaufzeit unter allen Schedulingalgorithmen garantiert (Theorem von Chetto & Chetto), stehen für aperiodische Tasks stets die meisten Freiräume zur Verfügung. Jeder andere Algorithmus kann somit höchstens dieselben Freiräume nutzen. ■ □

Satz 6.4 (EDL-Server Laufzeit). Die Berechnung der Leerlaufzeiten unter EDL hat Komplexität $O(N \cdot n)$, wobei n die Anzahl der periodischen Tasks und N die Anzahl der Instanzen in der Hyperperiode ist. Im schlimmsten Fall kann N sehr groß sein (bei nicht-harmonischen Perioden), was EDL für viele praktische Systeme ungeeignet macht.

6.7 Nicht-Existenz optimaler Server – Fundamentales Ergebnis

Theorem 6.6 (Tia, Liu, Shankar 1995). Für jeden Task-Set mit fester Prioritätszuweisung und jede aperiodische Ankunftsdisziplin existiert kein Online-Algorithmus, der die Antwortzeit *jeder* aperiodischen Anfrage minimiert.

Beweis durch Gegenbeispiel. Betrachte drei periodische Tasks: τ_1 ($C = 1, T = 3$), τ_2 ($C = 1, T = 4$), τ_3 ($C = 1, T = 6$), scheduliert unter RM. Bei Ankunft von J_1 ($C = 1$) bei $t = 2$ hat jeder Algorithmus zwei Optionen:

- Option 1: Führe J_1 sofort aus. Wenn dann J_2 bei $t = 3$ ankommt, ist kein Slack bis $t = 6$ verfügbar; J_2 terminiert frühestens bei $t = 7$.
- Option 2: Verschiebe J_1 auf $t = 3$. Dann hat J_2 einen freien Slot bei $t = 4$; J_2 terminiert bei $t = 5$. Aber J_1 ist nicht minimal.

In keiner Option sind beide Antwortzeiten gleichzeitig minimal. ■ □

Bemerkung 6.2. Dieses fundamentale Ergebnis zeigt, dass die in diesem Kapitel präsentierten Server keine scharf optimalen Lösungen sind: Es gibt kein “perfektes” Server-Verfahren. Die vorgestellten Algorithmen sind stattdessen Kompromisse zwischen Antwortzeit, Overhead, Speicherbedarf und Implementierungskomplexität.

Kapitel 7

Ressourcenzugriffsprotokolle – Prioritätsinversion und Lösungsstrategien

7.1 Das Prioritätsinversionsproblem

Definition 7.1 (Kritischer Abschnitt). Ein **kritischer Abschnitt** ist ein Codebereich, der unter gegenseitiger Ausschluss-Bedingung ausgeführt wird. Nur eine Task darf einen kritischen Abschnitt, der eine Ressource R_k schützt, gleichzeitig betreten. Gesichert durch Semaphor S_k mit `wait( $S_k$ )` beim Eintritt und `signal( $S_k$ )` beim Austritt.

Definition 7.2 (Prioritätsinversion). **Prioritätsinversion** tritt auf, wenn eine hochpriore Task τ_H blockiert ist, während eine niederpriore Task τ_M (ohne gemeinsame Ressource mit τ_H) ausgeführt wird. Die Blockierzeit ist dann nicht auf die Dauer des kritischen Abschnitts begrenzt, sondern hängt von der gesamten Ausführungszeit aller Tasks mit $P_M < P < P_H$ ab.

Beispiel 7.1 (Klassische Prioritätsinversion – 3 Tasks). $\tau_1 > \tau_2 > \tau_3$ in Priorität. τ_1 und τ_3 teilen Ressource R .

1. t_0 : τ_3 startet, betritt kritischen Abschnitt auf R .
2. t_1 : τ_2 kommt, preemptiert τ_3 (höhere Priorität).
3. t_2 : τ_1 kommt, preemptiert τ_2 (höhere Priorität).
4. t_3 : τ_1 versucht R zu betreten – blockiert (von τ_3 gehalten).
5. t_4 : τ_2 läuft weiter und preemptiert τ_3 – τ_1 wartet!

τ_1 wartet also nicht auf τ_3 's kritischen Abschnitt, sondern auf die *gesamte* Ausführungszeit von τ_2 . Im schlimmsten Fall unbeschränkt.

Terminologie:

- B_i : maximale Blockierzeit von τ_i
- $\delta_{j,k}$: Dauer des längsten kritischen Abschnitts von τ_j auf S_k
- $C(S_k) = \max\{P_i \mid \tau_i \text{ benutzt } S_k\}$: Prioritätsgrenze von S_k
- γ_i : Menge der kritischen Abschnitte, die τ_i blockieren können

7.2 Non-Preemptive Protocol (NPP)

7.2.1 Algorithmus

Definition 7.3 (NPP). Bei **NPP** wird die Priorität einer Task beim Eintritt in jeden kritischen Abschnitt auf das systemweite Maximum angehoben:

$$p_i(R_k) = \max_h \{P_h\}.$$

Bei Verlassen des kritischen Abschnitts wird p_i auf P_i zurückgesetzt.

Algorithm 11 Non-Preemptive Protocol (NPP)

- 1: **wait**(S_k):
 - 2: $p_i \leftarrow \max_h \{P_h\}$ ▷ Priorität auf Maximum
 - 3: Belege Ressource R_k
 - 4:
 - 5: **signal**(S_k):
 - 6: Gib R_k frei
 - 7: $p_i \leftarrow P_i$ ▷ Nominal-Priorität wiederherstellen
-

7.2.2 Blockierzeit unter NPP

Da unter NPP nie zwei Tasks gleichzeitig in kritischen Abschnitten sein können (die Prioritätsanhebung verhindert Preemption), ist die maximale Blockierzeit:

$$B_i = \max\{\delta_{j,k} - 1 \mid P_j < P_i, k = 1, \dots, m\}.$$

Nachteil: NPP verursacht *unnötige* Blockierung: Wenn τ_1 keine gemeinsame Ressource mit τ_3 hat, aber τ_3 in einem langen kritischen Abschnitt ist, wird τ_1 dennoch blockiert.

7.3 Priority Inheritance Protocol (PIP)

7.3.1 Protokolldefinition

Definition 7.4 (PIP – Sha, Rajkumar, Lehoczky 1990). Im **Priority Inheritance Protocol** erbt eine Task τ_j , die eine Ressource hält, vorübergehend die höchste Priorität aller Tasks, die auf diese Ressource warten:

$$p_j(R_k) = \max\{P_j, \max\{P_h \mid \tau_h \text{ wartet auf } R_k\}\}.$$

Diese Vererbung ist *transitiv*: Wenn τ_3 von τ_2 blockiert wird und τ_2 von τ_1 , erbt τ_3 die Priorität von τ_1 .

Algorithm 12 Priority Inheritance Protocol (PIP)

```
1: wait( $S_k$ ):  
2:   if  $R_k$  von  $\tau_j$  gehalten ( $P_j < P_i$ ) then  
3:      $\tau_i$  wird blockiert  
4:      $p_j \leftarrow \max(p_j, P_i)$  ▷ Prioritätsvererbung  
5:     Setze  $\tau_j$  fort mit neuer Priorität  
6:   else  
7:      $\tau_i$  belegt  $R_k$   
8:   end if  
9:  
10: signal( $S_k$ ):  
11: Gib  $R_k$  frei; wecke höchstpriorre wartende Task  
12: if keine Tasks mehr von  $\tau_j$  blockiert then  
13:    $p_j \leftarrow P_j$  ▷ Nominal-Priorität  
14: else  
15:    $p_j \leftarrow \max\{P_h \mid \tau_j \text{ blockiert } \tau_h\}$   
16: end if
```

7.3.2 Korrektheit und Eigenschaften

Lemma 7.1. Ein Semaphor S_k kann bei τ_i nur dann Push-Through-Blockierung auslösen, wenn S_k sowohl von einer Task mit $P < P_i$ als auch von einer Task mit $P > P_i$ benutzt wird.

Lemma 7.2. Transitive Prioritätsvererbung tritt nur bei *geschachtelten* kritischen Abschnitten auf.

Theorem 7.1 (Sha, Rajkumar, Lehoczky – PIP Blockierzeit). Unter PIP kann τ_i für höchstens $\alpha_i = \min(l_i, s_i)$ kritische Abschnitte blockiert werden, wobei l_i die Anzahl der niederpriorren Tasks und s_i die Anzahl der eindeutigen Semaphore sind, die τ_i blockieren können.

Beweis. **Bound durch l_i (Lemma 7.3 in Buttazzo):** τ_i kann durch τ_j nur blockiert werden, solange τ_j in einem kritischen Abschnitt ist. Sobald τ_j diesen verlässt, kann er τ_i nicht mehr blockieren. Pro niederpriorer Task also maximal ein Blockiervorgang.

Bound durch s_i (Lemma 7.4 in Buttazzo): Da Semaphore binär sind, kann genau eine Task pro Semaphor aktiv in einem kritischen Abschnitt sein. Nach dem Freigeben kann S_k τ_i nicht mehr blockieren. Also maximal eine Blockierung pro Semaphor. Das Minimum beider Bounds gilt. ■ □

7.3.3 Blockierzeitberechnung unter PIP

Require: Tasks geordnet nach absteigender Priorität; keine Schachtelung

```
1: for  $i = 1$  to  $n - 1$  do  
2:    $B_i^l \leftarrow \sum_{j: P_j < P_i} \max_k \{\delta_{j,k} - 1 \mid C(S_k) \geq P_i\}$   
3:    $B_i^s \leftarrow \sum_{k=1}^m \max_{j: P_j < P_i} \{\delta_{j,k} - 1 \mid C(S_k) \geq P_i\}$   
4:    $B_i \leftarrow \min(B_i^l, B_i^s)$   
5: end for  
6:  $B_n \leftarrow 0$ 
```

Komplexität: $O(mn^2)$.

7.4 Priority Ceiling Protocol (PCP)

7.4.1 Protokolldefinition

Definition 7.5 (PCP – Sha, Rajkumar, Lehoczky 1990). Jeder Ressource R_k wird eine **Prioritätsgrenze** (Priority Ceiling) zugewiesen:

$$C(R_k) = \max\{P_i \mid \tau_i \text{ benutzt } R_k\}.$$

Eine Task τ_i darf R_k nur betreten, wenn ihre aktive Priorität *strikt höher* ist als die Prioritätsgrenze aller aktuell gesperrten Ressourcen:

$$p_i > \max_{R_h \text{ gesperrt}} C(R_h).$$

Algorithm 13 Priority Ceiling Protocol (PCP)

```

1: wait( $S_k$ ):
2:   if  $p_i > \max\{C(R_h) \mid R_h \text{ belegt}\}$  then
3:      $\tau_i$  belegt  $R_k$ ;  $p_i(R_k) \leftarrow C(R_k)$  ▷ Priorität auf Ceiling
4:   else
5:      $\tau_i$  wird blockiert (von Task mit höchster Ceiling-Priorität)
6:   end if
7:
8:   signal( $S_k$ ):
9:     Gib  $R_k$  frei;  $p_i \leftarrow P_i$  ▷ Nominal-Priorität
10:  Wecke höchstprioritäre wartende Task

```

7.4.2 Eigenschaften und Nachweis

Theorem 7.2 (PCP Eigenschaften). Unter PCP gilt:

1. Jede Task τ_i wird höchstens einmal blockiert.
2. Deadlocks sind unmöglich.
3. Gegenseitiger Ausschluss ist gewährleistet.

Beweis. Zu 1 (Einmalige Blockierung): Angenommen, τ_i wird zweimal blockiert – einmal durch τ_a auf R_a und einmal durch τ_b auf R_b . Damit beide gleichzeitig ausgeführt werden könnten, müsste eine von ihnen die andere innerhalb eines kritischen Abschnitts preemptiert haben. Aber nach PCP-Regel erfordert das $p > C(R_{\text{gehalten}})$, also $P_a > C(R_b)$ oder $P_b > C(R_a)$. Da $C(R_b) \geq P_i > P_a$ und $C(R_a) \geq P_i > P_b$ (beide haben niedrigere Priorität als τ_i), führt das zu einem Widerspruch.

Zu 2 (Deadlock-Freiheit): Ein Deadlock erfordert zyklisches Warten. Nach PCP kann τ_j R_k nur belegen, wenn $p_j > C(R_k)$. Für einen Zyklus $\tau_a \rightarrow \tau_b \rightarrow \tau_a$ müsste $P_a > C(R_b) \geq P_b > C(R_a) \geq P_a$ gelten – Widerspruch. ■ □

7.4.3 Schedulierbarkeitsanalyse unter PCP

Die Schedulierbarkeit unter RM+PCP kann mit folgender erweiterter Response Time Analysis geprüft werden (Sha et al.):

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j,$$

wobei $B_i = \max\{\delta_{j,k} - 1 \mid P_j < P_i, C(R_k) \geq P_i\}$ die maximale Blockierzeit ist. Da τ_i höchstens einmal blockiert wird, ist B_i durch einen einzigen kritischen Abschnitt beschränkt.

7.5 Stack Resource Policy (SRP)

Definition 7.6 (Preemptionsgrenze (Preemption Level)). Unter SRP wird jedem Task τ_i ein statischer **Preemptionslevel** π_i zugewiesen (unter RM gleich der Priorität). Jeder Ressource R_k wird eine **Ressourcengrenze** zugewiesen:

$$\Pi(R_k) = \max\{\pi_i \mid \tau_i \text{ benutzt } R_k\}.$$

Ein **System Ceiling** $\Pi_{\text{sys}}(t)$ ist das Maximum aller Ressourcengrenzen der aktuell gesperrten Ressourcen.

Definition 7.7 (SRP-Zugriffsregel). τ_i darf den Prozessor belegen (preemptieren), wenn $\pi_i > \Pi_{\text{sys}}(t)$. τ_i darf R_k betreten, ohne zusätzliche Prüfung – die Zugangskontrolle erfolgt bereits beim Preemptionszeitpunkt.

Satz 7.1 (SRP Eigenschaften). Unter SRP gilt:

- Jede Task wird höchstens *einmal* blockiert (bei der Aktivierung, nicht beim Ressourcenzugriff).
- Stack-Sharing ist möglich: Alle Tasks können denselben Stack nutzen, da niemals zwei Tasks gleichzeitig im kritischen Abschnitt sein können.
- SRP ist sowohl für feste als auch für dynamische Prioritäten anwendbar (OSEK, AUTOSAR, POSIX-ähnliche Systeme).

Beweis der einmaligen Blockierung unter SRP. τ_i kann nur dann nicht aktiviert werden, wenn $\pi_i \leq \Pi_{\text{sys}}(t)$. Da Π_{sys} monoton abnimmt (Ressourcen werden freigegeben), kann τ_i nach seiner Aktivierung nie wieder durch den System Ceiling blockiert werden. Damit ist genau eine Blockierphase möglich, nämlich vor der ersten Ausführung. ■ □

7.5.1 Vergleich der Protokolle

Tabelle 7.1: Vergleich der Ressourcenzugriffsprotokolle

Protokoll	Max. Block.	Deadlock- frei	Stack- Sharing	Bemerkung
NPP	1 Abschnitt	Ja	Ja	Unnötige Blockierung
HLP	1 Abschnitt	Ja	Nein	Ceiling bei Eintritt
PIP	$\min(l_i, s_i)$	Nein!	Nein	Komplexe Analyse
PCP	1 Abschnitt	Ja	Nein	Guter Kompromiss
SRP	1 Abschnitt	Ja	Ja	Optimal für Einzel-CPU

Kapitel 8

Limited Preemptive Scheduling – Begrenzte Preemption

8.1 Motivation: Kosten und Nutzen der Preemption

Vollständig preemptives Scheduling maximiert die Schedulierbarkeit, verursacht aber erheblichen Overhead: Cache-Misses durch zerstörte Lokalität, Pipeline-Flushes, Kontext-Switch-Kosten. In vielen eingebetteten Systemen (AUTOSAR, ISO 26262) ist dieser Overhead signifikant.

Definition 8.1 (Preemptionskosten-Komponenten). Jede Preemption verursacht vier Kostenkategorien:

1. **Scheduling-Overhead:** Suspendierung, Warteschlangenoperation, Dispatch.
2. **Pipeline-Kosten:** Flush beim Unterbrechen, Refill beim Fortsetzen.
3. **Cache-Kosten (CRPD):** Cache-Related Preemption Delay – Reload der Cache-Lines, die die preemptierende Task verdrängt hat.
4. **Bus-Kosten:** Zusätzliche RAM-Zugriffe durch Cache-Misses.

Empirisch: Auf PowerPC MPC7410 mit 2 MB L2-Cache kann CRPD bis zu 33 % der nicht-preemptiven WCET betragen [1].

Zentrales Ergebnis dieses Kapitels: Für ein präemptiv machbares Task-Set lässt sich stets ein Preemptionsschema finden, das die Schedulierbarkeit erhält und gleichzeitig die Preemptionsanzahl minimiert.

8.2 Nicht-Preemptives Scheduling

8.2.1 Blockierzeit und Schedulierbarkeit

Ohne Preemption ergibt sich für jede Task τ_i eine Blockierzeit durch die längste niederpriorige Task:

$$B_i = \max_{j: P_j < P_i} \{C_j - 1\}. \quad (8.1)$$

Satz 8.1 (Nicht-preemptiv – $U_{\text{lub}} = 0$). Unter nicht-preemptivem RM und EDF gilt $U_{\text{lub}} = 0$: Es gibt Task-Sets mit beliebig kleiner Auslastung, die unter nicht-preemptivem Scheduling nicht schedulierbar sind.

Beweis. Betrachte τ_1 ($C_1 = \varepsilon$, $T_1 = 2\varepsilon$) und τ_2 ($C_2 = T_1 + \varepsilon = 3\varepsilon$, $T_2 = M$ für großes M). Auslastung: $U = \varepsilon/(2\varepsilon) + 3\varepsilon/M = 0,5 + 3\varepsilon/M$, beliebig nah an 0,5 für großes M . Aber: Nicht-preemptiv kann τ_2 nicht unterbrochen werden, also τ_1 muss $C_2 = 3\varepsilon > T_1 = 2\varepsilon$ warten – Deadline-Verletzung. ■ □

8.2.2 Machbarkeitsanalyse für nicht-preemptives Scheduling

Das *Self-Pushing-Phänomen* (Bril et al. 2009) zeigt, dass unter nicht-preemptivem Scheduling die worst-case Antwortzeit nicht beim ersten Job (kritischer Augenblick), sondern bei einem späteren Job auftreten kann. Daher muss die Analyse innerhalb der längsten Level- i -Aktivitätsperiode L_i durchgeführt werden:

$$\begin{aligned} L_i^{(0)} &= B_i + C_i, \\ L_i^{(s)} &= B_i + \sum_{h: P_h \geq P_i} \left\lfloor \frac{L_i^{(s-1)}}{T_h} \right\rfloor C_h. \end{aligned} \quad (8.2)$$

Für alle Jobs $k \in [1, K_i]$ mit $K_i = \lfloor L_i/T_i \rfloor$ wird der Startzeitpunkt $s_{i,k}$ iterativ berechnet:

$$s_{i,k}^{(\ell+1)} = B_i + (k-1)C_i + \sum_{h: P_h > P_i} \left(\left\lfloor \frac{s_{i,k}^{(\ell)}}{T_h} \right\rfloor + 1 \right) C_h.$$

Die Antwortzeit ergibt sich zu $R_i = \max_{k \in [1, K_i]} \{s_{i,k} + C_i - (k-1)T_i\}$.

8.3 Preemption Thresholds

8.3.1 Modell und Algorithmus

Definition 8.2 (Preemption Threshold – Wang & Saxena 1999). Jede Task τ_i erhält zwei Prioritätsniveaus:

- Nominale Priorität P_i (für die Ready-Queue und Einplanung),
- Preemption Threshold $\theta_i \geq P_i$ (für die Ausführung).

τ_h kann τ_i während der Ausführung nur preemptieren, wenn $P_h > \theta_i$.

Algorithm 14 Assign Minimum Preemption Thresholds (Wang & Saksena)

Require: Task-Set $\mathcal{T}$ mit $\{C_i, T_i, D_i, P_i\}$, preemptiv machbar

```
1: for  $i \leftarrow n$  downto 1 do ▷ Von niedrigster zu höchster Priorität
2:    $\theta_i \leftarrow P_i$ 
3:   Berechne  $R_i$  nach Gl. (8.3)
4:   while  $R_i > D_i$  do
5:      $\theta_i \leftarrow \theta_i + 1$ 
6:     if  $\theta_i > P_1$  then return INFEASIBLE
7:     end if
8:     Berechne  $R_i$  nach Gl. (8.3)
9:   end while
10: end for
11: return FEASIBLE
```

8.3.2 Machbarkeitsanalyse

Blockierzeit unter Preemption Thresholds:

$$B_i = \max\{C_j - 1 \mid P_j < P_i \leq \theta_j\}.$$

Antwortzeit (innerhalb der längsten Level- i -Aktivitätsperiode):

$$R_i = S_i + C_i + \sum_{h: P_h > \theta_i} \left(\left\lfloor \frac{R_i}{T_h} \right\rfloor - \left\lfloor \frac{S_i}{T_h} \right\rfloor - 1 \right) C_h, \quad (8.3)$$

wobei $S_i = B_i + \sum_{h: P_h > P_i} (\lfloor S_i/T_h \rfloor + 1) C_h$ der Startzeitpunkt ist.

Satz 8.2 (Optimalität des Threshold-Algorithmus). Der Algorithmus ist optimal: Wenn irgendeine Threshold-Zuweisung das Task-Set machbar macht, findet er eine solche Zuweisung.

8.3.3 Optimalitätsklassifikation

Preemption Thresholds sind **nicht scharf optimal**: Es gibt Task-Sets, die weder vollständig preemptiv noch vollständig nicht-preemptiv, aber mit geeigneten Thresholds schedulierbar sind. Innerhalb der Klasse der Threshold-Scheduler ist der Algorithmus **schwach optimal** (findet Lösung, wenn eine existiert).

8.4 Deferred Preemptions – Aufgeschobene Preemption

8.4.1 Floating Model und Aktivierungsbasiertes Modell

Definition 8.3 (Maximales nicht-preemptives Intervall). Jede Task τ_i hat ein maximales nicht-preemptives Intervall q_i , in dem sie nicht unterbrochen werden kann. Im **Floating Model** platziert der Programmierer explizite Grenzen; im **Activation-Triggered Model** wird beim Eintreffen einer höherprioren Task ein Timer auf q_i gesetzt.

Blockierzeit: $B_i = \max_{j: P_j < P_i} \{q_j - 1\}$.

8.4.2 Längste machbare nicht-preemptive Intervalle

Die *Blocking Tolerance* β_i ist die maximale Blockierzeit, die τ_i tolerieren kann, ohne eine Deadline zu verpassen:

$$\beta_i = \max_{t \in TS_i} \{t - W_i(t)\},$$

wobei $W_i(t)$ die Level- i -Arbeitslast und TS_i die Testmenge ist.

Theorem 8.1 (Yao, Buttazzo, Bertogna – Längste nicht-preemptive Intervalle). Die längsten nicht-preemptiven Intervalle Q_i , die die Schedulierbarkeit erhalten, können rekursiv berechnet werden:

$$Q_i = \min\{Q_{i-1}, \beta_{i-1} + 1\},$$

mit $Q_1 = \infty$ und $\beta_1 = D_1 - C_1$.

Beweis. Q_i ist das minimale $\beta_k + 1$ über alle Tasks mit höherer Priorität als τ_i . Durch Einsetzen der Rekursionsdefinition:

$$\min_{k: P_k > P_i} \{\beta_k + 1\} = \min\left\{\min_{k: P_k > P_{i-1}} \{\beta_k + 1\}, \beta_{i-1} + 1\right\} = \min\{Q_{i-1}, \beta_{i-1} + 1\}. \quad \blacksquare$$

□

Algorithm 15 Berechnung der längsten nicht-preemptiven Intervalle

- 1: $\beta_1 \leftarrow D_1 - C_1$; $Q_1 \leftarrow \infty$
 - 2: **for** $i \leftarrow 2$ **to** n **do**
 - 3: $Q_i \leftarrow \min\{Q_{i-1}, \beta_{i-1} + 1\}$
 - 4: Berechne β_i aus $W_i(t)$
 - 5: **end for**
-

8.4.3 Laufzeitanalyse

Berechnung aller Q_i : $O(n)$ Iterationen, jede mit $O(|TS_i|)$ Aufwand für die Berechnung von β_i . Im ungünstigsten Fall ist $|TS_i|$ pseudopolynomiell (abhängig von den Perioden).

8.5 Task Splitting – Kooperatives Scheduling

8.5.1 Modell und optimale Preemptionspunkt-Auswahl

Definition 8.4 (Task Splitting). Task τ_i wird in m_i nicht-preemptive *Chunks* (Teilaufgaben) zerlegt. Preemption ist nur an Chunkgrenzen (Effective Preemption Points, EPPs) erlaubt.

Algorithm 16 SelectEPP – Optimale Preemptionspunktauswahl (Bertogna et al. 2011)

Require: Task τ_i mit Basisblöcken $\delta_1, \dots, \delta_N$, maximales NPR Q

```
1:  $\hat{C}_0 \leftarrow 0$ ;  $Prev_0 \leftarrow \{\delta_0\}$ 
2: for  $k \leftarrow 1$  to  $N$  do
3:   Entferne aus  $Prev_k$  alle  $\delta_j$ , die Bed. (8.4) verletzen
4:   if  $Prev_k = \emptyset$  then return INFEASIBLE
5:   end if
6:    $\hat{C}_k \leftarrow \min_{\delta_j \in Prev_k} \left\{ \hat{C}_{j-1} + \xi_{j-1} + \sum_{\ell=j}^k b_\ell \right\}$ 
7:   Speichere  $\delta_{\text{prev}}(\delta_k)$ ;  $Prev_k \leftarrow Prev_k \cup \{\delta_k\}$ 
8: end for
9: Rekonstruiere EPPs rückwärts von  $\delta_{\text{prev}}(\delta_N)$ 
10: return FEASIBLE
```

Dabei ist die Bedingung für nicht-preemptive Regionen:

$$\xi_{j-1} + \sum_{\ell=j}^k b_\ell \leq Q. \quad (8.4)$$

Theorem 8.2 (Bertogna et al. 2011 – Optimalität von SelectEPP). Das von SelectEPP gefundene EPP-Muster minimiert den Preemptions-Overhead von τ_i unter der Bedingung, dass alle nicht-preemptiven Regionen die Länge Q nicht überschreiten.

Beweis. Der Algorithmus implementiert eine dynamische Programmierung über die Basisblöcke. Für jedes δ_k berechnet er das global optimale $\hat{C}_k$ (kleinste kumulative WCET der ersten k Blöcke). Da $\hat{C}_k$ das Minimum über alle machbaren Vorgänger δ_j bildet, werden alle möglichen machbaren EPP-Platzierungen für die ersten k Blöcke berücksichtigt. Durch Induktion über k ist $\hat{C}_N = C_i^{\min}$ die minimale WCET des gesamten Tasks. ■ □

8.5.2 Laufzeitanalyse von SelectEPP

Satz 8.3 (Laufzeit SelectEPP). SelectEPP hat eine Laufzeit von $O(N^2)$, wobei N die Anzahl der Basisblöcke ist. Für jeden der N Basisblöcke wird über die Menge $Prev_k$ (höchstens N Einträge) iteriert. Der Gesamtaufwand für das gesamte Task-Set mit n Tasks ist $O(\sum_i N_i^2)$.

8.6 Vergleich der Ansätze

Tabelle 8.1: Bewertung der Limited-Preemption-Methoden

Methode	Impl.- Aufwand	Vorhersage- barkeit	Effektivität	Optimalität
Non-Preemptive	Gering	Hoch	Niedrig	$U_{\text{lub}} = 0$
Preem. Thres- hold	Gering	Niedrig	Mittel	Schwach opti- mal
Deferred Preem.	Mittel	Mittel	Hoch	Lokal optimal
Task Splitting	Mittel	Hoch	Hoch	Scharf opti- mal

Kapitel 9

Überlastbehandlung – Scheduling unter Überlast

9.1 Einführung: Arten und Ursachen der Überlast

Überlast ($U > 1$) tritt auf, wenn die Gesamtrechenanforderung die verfügbare Prozessor-kapazität überschreitet. Ursachen sind:

- **Schlechtes Systemdesign:** Keine Worst-Case-Analyse durchgeführt.
- **Gleichzeitige Ereignisse:** Peak-Load-Szenarien überschätzt.
- **Hardwarefehler:** Anomale Interrupt-Sequenzen sättigen CPU.
- **Umgebungsveränderungen:** Unvorhergesehene Lastspitzen.

Definition 9.1 (Instantane Last). Die **instantane Last** zum Zeitpunkt t ist:

$$\rho(t) = \max_i \rho_i(t), \quad \rho_i(t) = \frac{\sum_{d_k \leq d_i} c_k(t)}{d_i - t},$$

wobei $c_k(t)$ die verbleibende Ausführungszeit von Job J_k ist. Ein System ist überlastet, wenn $\rho(t) > 1$.

9.2 Aperiodische Überlast – Value-Based Scheduling

Bei aperiodischer Überlast ist es nicht möglich, alle Deadlines einzuhalten. Dann ist das Ziel, den *Gesamtwert* der rechtzeitig fertiggestellten Tasks zu maximieren.

Definition 9.2 (Value Function). Jede Task J_i hat eine **Wertfunktion** $V_i(t)$, die den Nutzen angibt, wenn sie zum Zeitpunkt t fertiggestellt wird:

$$V_i(t) = \begin{cases} v_i & t \leq d_i \\ 0 & t > d_i \end{cases}$$

(harte Wertfunktion) oder eine monoton fallende Funktion (weiche Wertfunktion).

9.2.1 Algorithmus EARLIEST DEADLINE LATE (EDL) / RED

Unter Überlast versucht der **Robust Earliest Deadline (RED)** Algorithmus, den Gesamtwert zu maximieren, indem Tasks mit niedrigem Verhältnis v_i/C_i abgelehnt werden (Value Density):

Algorithm 17 Value-Based Scheduling unter Überlast (vereinfacht)

- 1: Sortiere aktive Tasks nach v_i/C_i absteigend
 - 2: $\rho \leftarrow \rho(t_{\text{aktuell}})$
 - 3: **while** $\rho > 1$ **do**
 - 4: Verwerfe Task mit kleinstem v_i/C_i
 - 5: Berechne ρ neu
 - 6: **end while**
 - 7: Plane verbleibende Tasks mit EDF ein
-

9.3 Überlastbehandlung unter RM vs. EDF

9.3.1 Permanente Überlast unter RM

Unter RM mit fester Priorität bei $U > 1$:

- Alle niederprioren Tasks werden vollständig blockiert.
- Tasks mit $P_i < P_j$ für alle j erhalten keinen Prozessor mehr.
- Das System degeneriert: Nur hochpriore Tasks laufen; alle anderen verpassen ihre Deadlines.

9.3.2 Permanente Überlast unter EDF – Cervin-Theorem

Theorem 9.1 (Cervin 2003 – EDF bei permanenter Überlast). Bei n periodischen Tasks mit $U > 1$ unter EDF gilt im eingeschwungenen Zustand: Die effektive mittlere Periode jeder Task τ_i beträgt

$$\bar{T}_i = T_i \cdot U.$$

EDF führt also eine **automatische Periodenskalierung** durch.

Beweisidee. In einem EDF-Schedule wird jede Task mit relativer Häufigkeit C_i/T_i ausgeführt (proportional zu ihrer Auslastung). Bei $U > 1$ wird die verfügbare Zeit 1 auf die Tasks im Verhältnis ihrer Auslastungen aufgeteilt. Task τ_i erhält den Anteil $U_i/U = C_i/(T_i U)$ der Prozessorzeit, was einer effektiven Rate von $1/\bar{T}_i = 1/(T_i U)$ entspricht. ■ □

Satz 9.1 (Vergleich RM vs. EDF bei Überlast). – **RM:** Niederpriore Tasks werden vollständig blockiert; hochpriore Tasks laufen normal. Ungerecht bei Überlast.

- **EDF:** Alle Tasks skalieren gleichmäßig; keine Task wird vollständig blockiert. Fairer Umgang mit Überlast.

Dies ist ein weiterer Vorteil von EDF gegenüber RM, zusätzlich zur höheren Auslastungsschranke.

9.4 Elastic Scheduling – Dynamische Periodeneanpassung

Definition 9.3 (Elastic Task). Eine elastische periodische Task τ_i hat einen nominellen Periodenbereich $[T_i^{\min}, T_i^{\max}]$ und einen Elastizitätskoeffizienten $e_i \geq 0$. Bei Überlast kann die Periode erhöht (und damit die Auslastung reduziert) werden, mit einem “Kostenfaktor” proportional zu e_i .

Satz 9.2 (Elastic Scheduling Feasibility). Ein elastisches Task-Set ist unter EDF schedulierbar, wenn für die optimalen Perioden T_i^* gilt:

$$\sum_{i=1}^n \frac{C_i}{T_i^*} \leq 1, \quad T_i^{\min} \leq T_i^* \leq T_i^{\max}.$$

Die optimalen Perioden können durch einen iterativen Algorithmus in $O(n^2)$ berechnet werden.

9.5 (π, λ) -Algorithmen für harte Überlast

Für Systeme, in denen nicht alle Tasks weich sind, werden **Overload Manager** eingesetzt, die Entscheidungen über Akzeptanz oder Ablehnung neuer Tasks treffen.

Definition 9.4 (Admission Control). Ein **Admission Controller** akzeptiert eine neue Anfrage J_{new} , wenn das erweiterte System $\mathcal{J} \cup \{J_{\text{new}}\}$ schedulierbar bleibt:

$$\rho(\mathcal{J} \cup \{J_{\text{new}}\}, t) \leq 1.$$

Andernfalls wird J_{new} abgelehnt oder eine niederpriorige Task verdrängt.

Satz 9.3 (Güte von EDF-basiertem Overload Management). EDF maximiert unter allen Online-Algorithmen bei harter Überlast den erwarteten Gesamtwert, wenn Tasks homogene Wertfunktionen haben (d. h. $v_i/C_i = \text{const}$).

Kapitel 10

Der EDF⁺ Algorithmus – Optimales Scheduling mit Blocking-Penalty

10.1 Motivation und Einordnung

Dieses Kapitel präsentiert **EDF⁺** (Earliest Deadline First Plus), einen neuartigen Echtzeit-Scheduling-Algorithmus, der die klassische EDF-Strategie um ein dynamisches Blocking-Penalty-Verfahren erweitert [40].

Der fundamentale Kernsatz gilt auch für EDF⁺:

**Wenn es einen machbaren Schedule gibt, findet EDF diesen sofort.
Die kürzeste Ausführungszeit aller Tasks findet EDF sofort. EDF ist
scharf optimal.**

EDF⁺ erweitert diese scharfe Optimalität auf Systeme mit stochastischen Blocking-Ereignissen. Während klassisches EDF bei Blocking-Ereignissen seine Optimalität verlieren kann, stellt EDF⁺ sicher, dass nach jedem Blocking-Ereignis der optimale EDF-Schedule für die verbleibenden Tasks berechnet wird.

EDF⁺ eignet sich besonders für eingebettete Systeme nach AUTOSAR und ISO 26262, da der WCET-Overhead durch Neuplanungen analytisch begrenzt ist und der Algorithmus selbststabilisierend und fair ist.

10.2 Grundlegende Definitionen

Definition 10.1 (Task-System). Ein **periodisches Task-System** ist ein Tupel

$$\Sigma = (\mathcal{T}, \mathbf{C}, \mathbf{D}, \mathbf{P})$$

mit Task-Menge $\mathcal{T} = \{T_1, \dots, T_n\}$, Worst-Case-Ausführungszeiten $\mathbf{C} \in \mathbb{R}_{>0}^n$, absoluten Deadlines $\mathbf{D} \in \mathbb{R}_{>0}^n$ und Perioden $\mathbf{P} \in \mathbb{R}_{>0}^n$ mit $D_i \leq P_i$.

Definition 10.2 (Blocking-Ereignis). Task T_i ist zum Zeitpunkt t **blockiert**, falls sie nicht fortschreiten kann, obwohl der Prozessor ihr zugeteilt ist. Die Blocking-Indikatorvariable $B_i \in \{0, 1\}$ erfüllt

$$P(B_i = 1) = p, \quad P(B_i = 0) = 1 - p,$$

wobei $p \in (0, 1)$ die Blocking-Wahrscheinlichkeit ist.

Definition 10.3 (Effektive Deadline). Die **effektive Deadline** von T_i nach einem Blocking-Ereignis ist

$$D_i^{\text{eff}} = \begin{cases} D_i & \text{falls } T_i \text{ nicht blockiert hat,} \\ +\infty & \text{falls } T_i \text{ blockiert hat.} \end{cases}$$

Definition 10.4 (Geometrische Verteilung der Blockings). Die Anzahl der Ausführungsversuche bis zur erfolgreichen Ausführung von T_i folgt einer geometrischen Verteilung mit Parameter p :

$$P(X = k) = (1 - p)^{k-1} \cdot p, \quad k = 1, 2, 3, \dots$$

mit Erwartungswert $E[X] = 1/p$.

10.3 Der EDF⁺-Algorithmus

10.3.1 Algorithmus-Beschreibung

EDF⁺ arbeitet in fünf Phasen:

Phase 1: EDF-Initialisierung

$$\sigma^{(0)} = \text{MergeSort}(\mathcal{T}, \mathbf{D}).$$

Phase 2: Ausführung – Tasks werden in Reihenfolge $\sigma^{(k)}$ ausgeführt.

Phase 3: Blocking-Erkennung und Penalty

Blockiert T_i in Schritt k :

$$D_i^{\text{eff}} \leftarrow +\infty \quad (\text{niedrigste Priorität}).$$

Phase 4: Sofortige Neuplanung

$$\sigma^{(k+1)} = \text{MergeSort}(\mathcal{T}, \mathbf{D}^{\text{eff}}).$$

Phase 5: Terminierung – Wiederholung bis alle Tasks abgearbeitet sind.

10.3.2 Pseudocode

Algorithm 18 EDF⁺ Scheduling Algorithm

Require: Task-Menge $\mathcal{T} = \{T_1, \dots, T_n\}$, Deadlines D , Periode $\mathcal{P}$

Ensure: Schedule π über $[0, \mathcal{P}]$

```

1:  $\mathbf{D}^{\text{eff}} \leftarrow \mathbf{D}$ 
2:  $\sigma \leftarrow \text{MERGESORT}(\mathcal{T}, \mathbf{D}^{\text{eff}})$ 
3:  $\mathcal{Q} \leftarrow \sigma, t \leftarrow 0, \pi \leftarrow \emptyset$ 
4: while  $\mathcal{Q} \neq \emptyset$  and  $t < \mathcal{P}$  do
5:    $T_i \leftarrow \text{DEQUEUE}(\mathcal{Q})$ 
6:    $t_{\text{start}} \leftarrow t$ 
7:   Führe  $T_i$  aus bis Fertigstellung oder Blocking
8:   if  $T_i$  blockiert then
9:      $D_i^{\text{eff}} \leftarrow +\infty$ 
10:     $\mathcal{Q} \leftarrow \text{MERGESORT}(\mathcal{Q} \cup \{T_i\}, \mathbf{D}^{\text{eff}})$ 
11:    continue
12:   end if
13:    $\pi \leftarrow \pi \cup \{(T_i, t_{\text{start}}, t)\}$ 
14:    $t \leftarrow t + C_i$ 
15: end while
16: return  $\pi$ 

```

10.4 Optimalitätsbeweis für EDF⁺

10.4.1 Fundament: EDF-Optimalitätstheorem

Das gesamte Optimalitätsargument für EDF⁺ beruht auf dem in Kapitel 3 bewiesenen EDF-Optimalitätstheorem (Satz 3.2):

Satz 10.1 (EDF-Optimalitätstheorem – Wiederholung). Sei Σ ein präemptives Single-Processor-Task-System mit $U \leq 1$ und Blocking-Wahrscheinlichkeit $p = 0$. Dann findet EDF stets einen gültigen Schedule:

$$U \leq 1, p = 0 \implies \text{EDF verpasst keine Deadline.}$$

Dieses Theorem ist das zentrale Werkzeug: EDF⁺ erzeugt nach jedem Blocking-Ereignis eine Teilmenge von Tasks, die die Voraussetzungen von Satz 10.1 erfüllt.

10.4.2 Hauptsatz: Optimalität von EDF⁺

Satz 10.2 (Optimalität von EDF⁺). EDF⁺ ist **optimal** für präemptive Single-Processor-Systeme mit stochastischen Blocking-Ereignissen: Wenn ein schedularbares Task-System mit Blocking-Wahrscheinlichkeit $p \in [0, 1)$ existiert, so findet EDF⁺ einen gültigen Schedule, der alle erreichbaren Deadlines einhält.

Beweis. Teil 1: Basisfall – kein Blocking ($p = 0$). Ohne Blocking reduziert sich EDF⁺ auf klassisches EDF: $D_i^{\text{eff}} = D_i$ für alle i . Das System erfüllt $U \leq 1$ und $p = 0$. Nach Satz 10.1 gilt: EDF⁺ ($\equiv$ EDF) verpasst keine Deadline.

Teil 2: Penalty-Schritt – eine blockierte Task ($p > 0$, ein **Blocking-Ereignis**). Blockiert T_i zum Zeitpunkt $t^* < D_i$. EDF⁺ setzt $D_i^{\text{eff}} \leftarrow +\infty$, was T_i aus dem laufenden Wettbewerb ausschließt. Die verbleibende Task-Menge $\mathcal{T}' = \{T_j \in \mathcal{T} \mid D_j^{\text{eff}} < +\infty\}$ hat Auslastung:

$$U' = U - \frac{C_i}{P_i} < U \leq 1.$$

Für alle $T_j \in \mathcal{T}'$ gilt zum Zeitpunkt t^* : $p_j = 0$. Damit erfüllt $\mathcal{T}'$ die Voraussetzungen von Satz 10.1:

$$U' \leq 1, p' = 0 \implies \text{EDF findet auf } \mathcal{T}' \text{ einen Deadline-freien Schedule.}$$

Teil 3: Induktionsschritt – R Blocking-Ereignisse. Per vollständiger Induktion über R : Beim R -ten Blocking von T_{i_R} erzeugt EDF⁺ die Menge:

$$\mathcal{T}^{(R)} = \{T_j \mid D_j^{\text{eff}} < +\infty\}, \quad U^{(R)} = U - \sum_{k=1}^R \frac{C_{i_k}}{P_{i_k}} < U \leq 1.$$

Auf $\mathcal{T}^{(R)}$ sind keine Tasks blockiert: $p^{(R)} = 0$. Nach Satz 10.1 verpasst EDF auf $\mathcal{T}^{(R)}$ keine Deadline. EDF⁺ berechnet via Merge Sort exakt diesen optimalen Schedule. Per Induktionshypothese war die Behandlung der vorherigen $R - 1$ Blockings bereits optimal. Die Gesamtaussage folgt durch vollständige Induktion. ■ □

10.4.3 Korollare

Korollar 10.1 (Dominanz über RMS). Für jedes präemptive Task-System gilt:

$$\text{Miss-Rate}(\text{EDF}^+) \leq \text{Miss-Rate}(\text{RMS}).$$

Die Ungleichung ist strikt für $U > \ln 2 \approx 0,693$.

Beweis. RMS ist nach Liu & Layland [6] nur für $U \leq \ln 2$ deadline-safe. Für $U > \ln 2$ können RMS-Schedules Deadlines verfehlen. EDF⁺ hingegen reduziert bei jedem Blocking auf eine neue EDF-Instanz $\mathcal{T}^{(k)}$ mit $U^{(k)} < U \leq 1$; auf diese wendet Satz 10.1 an, der die Deadline-Freiheit garantiert. ■ □

Korollar 10.2 (Minimax-Eigenschaft). Unter allen Scheduling-Algorithmen, die bei Blocking-Ereignissen neu planen, minimiert EDF⁺ die erwartete Anzahl verpasster Deadlines:

$$E[\text{Misses}_{\text{EDF}^+}] = \min_{\mathcal{A}} E[\text{Misses}_{\mathcal{A}}].$$

10.5 Laufzeitanalyse

Satz 10.3 (Laufzeit von EDF⁺). Die Laufzeit von EDF⁺ für $N = 40$ Tasks mit Blocking-Wahrscheinlichkeit p :

- **Initialisierung:** $\Theta(n \log n)$ durch Merge Sort.
- **Pro Blocking-Ereignis:** $\Theta(n \log n)$ für Neuplanung.
- **Erwartete Neuplanungen:** $E[R] = np/(1 - p)$ nach geometrischer Verteilung.

- **Gesamte erwartete Laufzeit:** $T_{\text{gesamt}} = \Theta\left(n \log n \cdot \frac{1}{1-p}\right)$.

Tabelle 10.1 zeigt die erwartete Anzahl von Neuplanungen für $N = 40$ Tasks:

Tabelle 10.1: Erwartete Neuplanungen für EDF⁺ bei $N = 40$ Tasks

p (Blocking)	$E[R]$ Neuplanungen	T_{gesamt} (relativ zu EDF)
0,01	0,40	1,01×
0,05	2,11	1,05×
0,10	4,44	1,11×
0,20	10,0	1,25×
0,50	40,0	2,00×

10.6 EDF⁺ für dynamische Task-Mengen

Die Erweiterung auf dynamisch wachsende Task-Mengen ist möglich. Bei Ankunft einer neuen Task T_{n+1} zum Zeitpunkt a_{n+1} wird die laufende Ausführung von T_{cur} preemptiert, falls $D_{n+1} < D_{\text{cur}}$, und der Schedule für die erweiterte Menge neu berechnet.

Satz 10.4 (Optimalität von EDF⁺ für dynamische Task-Mengen). EDF⁺ ist optimal für dynamische Task-Mengen mit stochastischen Blocking-Ereignissen und dynamischen Task-Ankünften. Bei jedem Ankunftsereignis oder Blocking-Ereignis erzeugt EDF⁺ eine Teilmenge aktiver Tasks, die die Voraussetzungen des EDF-Optimalitätstheorems (Satz 10.1) erfüllt.

10.7 Integration in AUTOSAR und ISO 26262

EDF⁺ ist direkt anwendbar in eingebetteten Systemen:

- **AUTOSAR Classic Platform:** EDF⁺ kann als OS Application mit Tasks nach OSEK/VDX implementiert werden.
- **ISO 26262 (ASIL-D):** Der analytisch begrenzte WCET-Overhead durch Neuplanungen ermöglicht eine zertifizierbare Implementierung.
- **Vorteil gegenüber RM:** Bei Auslastungen $U > \ln 2 \approx 0,69$ (typisch in modernen ECUs) übertrifft EDF⁺ RM systematisch.

Kapitel 11

Kernel-Design-Aspekte für Echtzeit-Systeme

11.1 Struktur eines Echtzeit-Kernels

Ein Echtzeit-Kernel unterscheidet sich fundamental von einem Timesharing-Kernel: Statt Durchschnittsdurchsatz zu maximieren, muss er Vorhersagbarkeit und die Einhaltung individueller Zeitschranken garantieren.

Definition 11.1 (Kernelprimitiven für Echtzeit). Ein Echtzeit-Kernel muss folgende Primitive mit *beschränkter Ausführungszeit* bereitstellen:

- **create**(τ, C, T, D): Task erstellen mit expliziten Zeitschranken.
- **activate**(τ, t): Task zum Zeitpunkt t aktivieren.
- **delay**(τ, δ): Task für δ Zeiteinheiten suspendieren.
- **wait**(S): Semaphorzugriff mit Prioritätsprotokoll.
- **signal**(S): Semaphore freigeben mit Prioritätsreset.

Alle Primitive müssen *preemptierbar* sein und dürfen keine unbeschränkten Schleifen enthalten.

11.1.1 Prozesszustände in einem Echtzeit-Kernel

Zustand	Beschreibung
RUNNING	Task wird aktuell ausgeführt
READY	Task bereit, wartet auf CPU-Zuteilung
BLOCKED	Task wartet auf Ressource oder Ereignis
SUSPENDED	Task deaktiviert (bis zum nächsten Aktivierungszeitpunkt)
IDLE	Leerlauf-Task (niedrigste Priorität)

Abbildung 11.1: Prozesszustände in einem Echtzeit-Kernel

11.1.2 Warteschlangen-Implementierung

Satz 11.1 (Komplexität der Scheduling-Warteschlange). – **Feste Prioritäten (RM)**: Multi-Level-Queue mit P Prioritätsstufen. Einfügen und Entnehmen: $O(1)$.

- **Dynamische Prioritäten (EDF):** Heap (balancierter Binärbaum). Einfügen und Entnehmen: $O(\log n)$.

Für kleine Task-Sets ($n < 100$) ist der $O(\log n)$ -Overhead von EDF vernachlässigbar. RM bietet nur bei sehr großen Task-Sets ($n > 1000$) einen wesentlichen Vorteil.

11.2 Interrupt-Behandlung in Echtzeit-Systemen

Satz 11.2 (Interrupt-Ansatz C – Empfohlene Methode für Echtzeit). Der beste Ansatz für Echtzeit-Systeme: Interrupts aktivieren eine dedizierte *Task* zur Gerätebehandlung, anstatt das Gerät direkt im Interrupt-Handler zu bedienen. Der Handler reduziert sich auf:

- 1: **interrupt_handler**(E):
- 2: activate(J_E) ▷ Aktiviere dedizierte Task J_E
- 3: return

Die Gerätebehandlungs-Task J_E wird dann vom Scheduler mit normaler Priorität verwaltet und kann von höherprioritären Tasks preemptiert werden. Damit ist die Ausführungszeit von Interrupt-Handlern vollständig in die Schedulierbarkeitsanalyse integrierbar.

11.3 Speicherverwaltung

Demand Paging ist inkompatibel mit harten Echtzeit-Anforderungen, da Page Faults zu unvorhersagbaren Verzögerungen führen. Empfohlene Alternativen:

- **Statische Speicherpartitionierung:** Feste Segmente für jede Task, keine Verdrängung. Vorhersagbar, aber weniger flexibel.
- **Memory Locking:** Alle Seiten kritischer Tasks werden im RAM gesperrt (z. B. `mlockall()` unter Linux/POSIX).
- **Stack-Sharing unter SRP:** Durch die SRP-Eigenschaft (keine zwei Tasks gleichzeitig im kritischen Abschnitt) kann ein gemeinsamer Stack verwendet werden, was den Speicherbedarf erheblich reduziert.

11.4 Kernel-Overhead und Schedulierbarkeitsanalyse

In der Praxis müssen Kernel-Overheads in der Schedulierbarkeitsanalyse berücksichtigt werden. Sei Δ_{cs} die Kontextwechselzeit und Δ_{int} der Interrupt-Handler-Overhead, dann wird jede WCET aufgestockt zu:

$$C_i^{\text{eff}} = C_i + n_i \cdot \Delta_{cs} + k_i \cdot \Delta_{int},$$

wobei n_i die maximale Anzahl von Kontextwechseln und k_i die maximale Anzahl von Interrupts während der Ausführung von τ_i sind.

Kapitel 12

Anwendungsdesign für Echtzeit-Systeme

12.1 Zeitschrankenableitung aus Anwendungsanforderungen

Die kritischste und schwierigste Phase beim Entwurf eines Echtzeit-Systems ist die Ableitung von Task-Perioden und Deadlines aus den Anwendungsanforderungen.

Definition 12.1 (Nyquist-Rate für Regelungsanwendungen). Für eine Regelschleife mit Systemdynamik bis zur Frequenz $f_{\max}$ (Hz) muss die Abtastrate (und damit die Task-Periode) nach dem Nyquist-Theorem mindestens $T_i \leq 1/(2f_{\max})$ betragen. In der Praxis wird $T_i \leq 1/(10f_{\max})$ empfohlen, um Implementierungstoleranzen zu berücksichtigen.

12.2 Hierarchisches Entwurfsmuster

Definition 12.2 (Hierarchische Dekomposition). Ein Echtzeit-System wird in Ebenen unterteilt:

1. **Sensor-Ebene:** Datenerfassung, höchste Frequenz ($T \approx 1$ ms).
2. **Filter-/Vorverarbeitungs-Ebene:** Rauschunterdrückung, Kalman-Filter.
3. **Regelungs-Ebene:** PID, MPC, mittlere Frequenz ($T \approx 10$ ms).
4. **Planungs-Ebene:** Trajektorienplanung, niedrige Frequenz ($T \approx 100$ ms).
5. **Überwachungs-Ebene:** Zustandsanzeige, Logging ($T \approx 1$ s).

12.3 Robotersteuerungsbeispiel – Vollständige Analyse

Tabelle 12.1: Task-Set einer 6-DOF-Robotersteuerung

Task	C_i (ms)	T_i (ms)	U_i	RM-Prio	Funktion
τ_1 Sensor	0,5	2	0,250	5	Gelenk-Encoder
τ_2 Filter	1,0	5	0,200	4	Kalman-Filter
τ_3 Regler	2,0	10	0,200	3	Kaskadenregelung
τ_4 Kinematik	3,0	20	0,150	2	Inverse Kinematik
τ_5 Monitor	2,0	100	0,020	1	Überwachung
Σ			0,820		

Schedulierbarkeitsanalyse:

$U = 0,820$. Liu-Layland-Bound: $U_{\text{lub}}(5) = 5(2^{1/5} - 1) \approx 0,743$.

Da $U = 0,820 > 0,743$, gibt der Liu-Layland-Test *keine Garantie*. Wir verwenden Response Time Analysis:

$R_1 = 0,5 \leq 2$. ✓

$R_2^{(0)} = 1,5$; $R_2^{(1)} = 1,0 + \lceil 1,5/2 \rceil \cdot 0,5 = 2,0 > 1,5$; $R_2^{(2)} = 1,0 + \lceil 2,0/2 \rceil \cdot 0,5 = 2,0$.

Konvergenz: $R_2 = 2,0 \leq 5$. ✓

Analog für τ_3, τ_4, τ_5 : Alle scheduleierbar unter RM.

Da $U = 0,82 \leq 1$: Das System ist auch unter EDF scheduleierbar – mit mehr Spielraum für aperiodische Tasks (TBS mit $U_s = 1 - 0,82 = 0,18$ möglich).

12.4 AUTOSAR-Anwendungsbeispiel

In AUTOSAR Classic wird EDF üblicherweise nicht direkt verwendet; stattdessen nutzt AUTOSAR feste Prioritäten (RM-basiert). Mit EDF⁺ (Kapitel 10) kann jedoch ein optimales Scheduling auch mit Blocking-Ereignissen erreicht werden.

Tabelle 12.2: AUTOSAR-Task-Konfiguration für ein Motorsteuergerät (ECU)

Task (OSEK-Kategorie)	C_i (μ s)	T_i (ms)	Funktion
PreemptTask10ms	200	10	Einspritzregelung (ASIL-D)
PreemptTask20ms	500	20	Zündzeitpunkt (ASIL-C)
PreemptTask100ms	2000	100	Diagnose (ASIL-B)
BasicTask1s	5000	1000	NVM-Update (QM)

Auslastung: $U = 0,02 + 0,025 + 0,020 + 0,005 = 0,070$. Weit unter U_{lub} : komfortabler Sicherheitspuffer für Spitzenlasten und aperiodische Diagnose-Tasks.

Kapitel 13

Zusammenfassung, Schlussfolgerungen und Ausblick

13.1 Kernaussagen dieses Buches

Die zentralen Ergebnisse lassen sich im fundamentalen Kernsatz zusammenfassen:

Wenn es einen machbaren Schedule gibt, findet RM diesen sofort.
RM ist schwach optimal.

Wenn es einen machbaren Schedule gibt, findet EDF diesen sofort.
Die kürzeste Ausführungszeit aller Tasks findet EDF sofort. EDF ist
scharf optimal.

13.2 Schwache vs. Scharfe Optimalität – Gesamtüberblick

Definition 13.1 (Schwache Optimalität von RM). RM ist **schwach optimal**: optimal innerhalb fester Prioritäten, $U_{\text{lub}} = \ln 2 \approx 0,693$ im worst case.

Definition 13.2 (Scharfe Optimalität von EDF). EDF ist **scharf optimal**: $U_{\text{lub}} = 1$. EDF scheduliert *jedes* Task-Set mit $U \leq 1$.

13.3 EDF⁺ – Erweiterung der Schärfe auf Blocking-Szenarien

EDF⁺ [40] überträgt die scharfe Optimalität von EDF auf Systeme mit stochastischen Blocking-Ereignissen. Bei jedem Blocking erzeugt EDF⁺ eine Teilmenge nicht-blockierter Tasks, auf die das EDF-Optimalitätstheorem anwendbar ist. EDF⁺ dominiert RM für alle $U > \ln 2$ und minimiert die erwartete Anzahl verpasster Deadlines unter allen neuplanenden Algorithmen.

13.4 Praktische Empfehlungen

1. Verwende EDF statt RM wenn $U > 0,693$ und dynamische Prioritäten akzeptabel sind.

2. **Verwende EDF⁺ statt EDF** wenn Blocking-Ereignisse möglich sind.
3. **Verwende RM** wenn $U < 0,693$ und einfache $O(1)$ -Implementierung wichtig ist (AUTOSAR Classic, OSEK).
4. **Implementiere SRP** wenn gemeinsame Ressourcen vorhanden sind.
5. **Prüfe Schedulierbarkeit offline** (RTA für RM, Auslastungstest für EDF).
6. **Verwende CBS** bei variablen Ausführungszeiten und temporaler Isolation.

13.5 Ausblick: Offene Forschungsfragen und Zukünftige Entwicklungen

13.5.1 Mehrprozessor-Scheduling – Das größte offene Problem

Das fundamentalste ungelöste Problem der Echtzeit-Theorie ist die Erweiterung der Uniprocessor-Ergebnisse auf Mehrprozessorsysteme. Der Dhall-Effekt zeigt, dass EDF auf Mehrprozessoren nicht mehr optimal ist.

Globales vs. Partitioniertes Scheduling

Definition 13.3 (Globales Scheduling). Beim **globalen Scheduling** können Tasks auf jedem der m Prozessoren ausgeführt werden. Vorteil: maximale Flexibilität. Nachteil: Task-Migrationen und Cache-Invalidierungen erhöhen den Overhead erheblich.

Definition 13.4 (Partitioniertes Scheduling). Beim **partitionierten Scheduling** wird jede Task statisch einem Prozessor zugewiesen. Die Zuweisung selbst ist ein NP-schweres Bin-Packing-Problem. Auf jedem Prozessor läuft ein unabhängiger Uniprocessor-Scheduler.

Vielversprechende Ansätze: *Semi-Partitioned Scheduling* (bestimmte Tasks dürfen auf zwei Prozessoren migrieren) und *Clustered Scheduling* (Gruppen von Prozessoren mit gemeinsamem EDF-Scheduler).

Pfair – Optimales Mehrprozessor-Scheduling

Definition 13.5 (Pfair-Scheduling (Baruah et al. 1996)). **Pfair** (Proportionate Fair) ist das erste optimal auf homogenen Mehrprozessorsystemen: Jede Task τ_i erhält in jedem Zeitquantum $[t, t + 1)$ exakt U_i Zeiteinheiten (proportional zur Auslastung). Abweichungen um mehr als 1 Einheit werden als Pfair-Verletzung gewertet.

Nachteil: Enorme Anzahl von Kontext-Switches (proportional zu n pro Zeitquantum) macht Pfair für praktische Systeme kaum einsetzbar. LLREF (Least Laxity and Required Execution time First) und DP-Fair reduzieren die Preemptionsrate bei Beibehaltung optimaler Eigenschaften für spezifische Szenarien.

13.5.2 Mixed-Criticality Scheduling

Definition 13.6 (Mixed-Criticality System (MCS)). In einem MCS haben Tasks unterschiedliche Kritikalitätsstufen (ASIL A–D nach ISO 26262, DAL A–E nach DO-178C). Jede Task τ_i hat zwei WCET-Werte:

- C_i^{LO} : WCET bei niedrigem Kritikalitätsniveau (gemessen).
- $C_i^{\text{HI}} \gg C_i^{\text{LO}}$: WCET bei hohem Niveau (formal verifiziert, erheblich größer).

Das System wechselt vom Lo-Mode in den Hi-Mode, wenn eine Task C_i^{LO} überschreitet.

Vestal-Modell und AMC

Das Vestal-Modell (2007) formalisierte MCS. Der **Adaptive Mixed Criticality (AMC)** Algorithmus von Burns & Davis (2011) ist der meistzitierte Ansatz: RM-basiert für Lo-Mode, Wechsel zu modifizierten Deadlines im Hi-Mode. Bekannte Grenzen: Keine Garantien für niedrigkritische Tasks im Hi-Mode; Modusübergänge können Deadline-Verletzungen bei mittelkritischen Tasks verursachen.

EDF-VD – Theoretisch fundierter MCS-Algorithmus

EDF mit Virtual Deadlines (EDF-VD) von Baruah et al. (2012) weist hochkritischen Tasks virtuelle (kürzere) Deadlines zu. Scharf optimal für zwei Kritikalitätsstufen auf Uniprocessoren. Erweiterungen auf $k > 2$ Stufen und Mehrprozessoren sind aktive Forschungsthemen.

Offene Frage: Gibt es einen universell optimalen MCS-Algorithmus, der für alle Systeme mit $U^{\text{LO}} \leq 1$ und $U^{\text{HI}} \leq 1$ machbar ist?

13.5.3 Maschinelles Lernen im Echtzeit-Scheduling

Klassische ML-Modelle sind probabilistisch und inkompatibel mit harten Echtzeit-Garantien. Dennoch gibt es vielversprechende Ansätze:

ML-basierte WCET-Schätzung

- **Extreme Value Theory (EVT)**: Statistische Analyse seltener Ausführungspfade für probabilistische WCET-Schranken (pWCET). Ermöglicht engere WCET-Schätzungen als statische Analyse.
- **Random Forest / Gradient Boosting**: Schätzen WCET aus Programm-Features (Schleifenanzahl, Cachestruktur, Abhängigkeitstiefe).
- **Messdaten-Regression**: Nutzung historischer Ausführungsdaten zur Verbesserung der Analysegenauigkeit bei seltenen Pfaden.

Reinforcement Learning für adaptives Scheduling

In Systemen mit nicht vollständig bekannten Task-Parametern (Robotik, autonome Fahrzeuge) lernen RL-basierte Scheduler online optimale Scheduling-Policies. Da harte Garantien fehlen, wird *Safe Reinforcement Learning* untersucht: Der RL-Agent schlägt Entscheidungen vor; ein formaler Verifikator prüft Schedulierbarkeit. Bei Verletzung greift ein klassischer sicherer Algorithmus.

13.5.4 Cyber-Physical Systems und das Jitter-Problem

Control-Aware Scheduling

Jitter in der Ausführung eines Regelalgorithmus verschlechtert direkt die Regelgüte. Klassische Echtzeit-Theorie minimiert nur Deadlines – nicht Jitter. *Control-Aware Scheduling* (Cervin, Henriksson, Lincoln 2003–2010) berücksichtigt regelungstechnische Gütekriterien direkt im Scheduling-Algorithmus und optimiert gemeinsam Scheduling und Regelgüte.

Time-Sensitive Networking (TSN)

IEEE 802.1Qbv integriert Scheduling auf Netzwerkebene. In verteilten Echtzeit-Systemen (Industrial IoT, Industrie 5.0) muss Task-Scheduling auf CPU-Ebene mit Netzwerkscheduling koordiniert werden:

- Uhrensynchronisation (PTP/IEEE 1588) als Grundlage.
- Verteiltes EDF über mehrere Knoten mit Netzwerkverzögerungen.
- Ressourcen-Orchestrierung: CPU, GPU, FPGA und Netzwerkbandbreite in heterogenen Systemen gemeinsam planen.

13.5.5 Formale Verifikation von Scheduling-Algorithmen

Modellprüfung

Werkzeuge wie UPPAAL (zeitbehaftete Automaten) und TLA+ ermöglichen die formale Verifikation:

- Nachweis der Deadline-Freiheit für konkrete Task-Sets.
- Verifikation von PIP/PCP auf Korrektheit und Deadlock-Freiheit.
- Verifikation von EDF⁺ für endliche Blocking-Szenarien.

Herausforderung: Zustandsexplosion bei $n > 10$ Tasks. Abstraktion und parametrische Verifikation (für beliebiges n) sind offene Forschungsfragen.

Theorem-Prover

Isabelle/HOL, Coq und Lean ermöglichen maschinengeprüfte Beweise:

- Der Liu-Layland-Bound wurde von Paulin (2006) in Isabelle formalisiert.
- Das EDF-Optimalitätstheorem ist für Formalisierung zugänglich (Austauschargument als induktiver Beweis).
- **Ziel:** Vollständig verifizierte Scheduling-Bibliothek für sicherheitskritische Systeme (ASIL-D-Zertifizierung).

13.5.6 Der Subgraph Algorithmus im verteilten Echtzeit-Kontext

Der **Subgraph Algorithmus** $\sigma_j = \sum A_{ij} \cdot 2^i + j \cdot 2^n$ [40] bietet mit seiner $O(n^3)$ -Komplexität neue Möglichkeiten:

- **Topologieanalyse von Task-Abhängigkeitsgraphen:** Effiziente Erkennung von Subgraph-Beziehungen zwischen Task-Sets.
- **Deduplizierung:** Identifikation äquivalenter Task-Konfigurationen in großen Scheduling-Problemen.
- **Verteilte Systeme:** Schnelle Topologieanpassung bei Knotenausfällen oder Neukonfigurationen in Echtzeit-Netzwerken.

Zukünftige GPU-Parallelisierung (Tensoroperationen für Matrixberechnungen) könnte die Komplexität auf $O(n^2 \log n)$ reduzieren.

13.5.7 Hardware-Beschleunigung für Scheduling

Für Systeme mit sehr vielen Tasks ($n > 10.000$) oder extrem kurzen Perioden (Mikrosekunden-Bereich) werden Hardware-beschleunigte Scheduler untersucht:

- **FPGA-basierter EDF-Scheduler:** Parallele Heap-Operationen in $O(1)$ statt $O(\log n)$. Erste Prototypen zeigen 10-fachen Speedup.
- **Content-Addressable Memory (CAM):** Hardware-Implementierung von Priority Queues durch assoziative Speicherung.
- **Near-Memory Computing:** Scheduling-Logik direkt im Speichercontroller für datenintensive Echtzeit-Workloads.

13.5.8 Sicherheitszertifizierung – ISO 26262 und POSIX

AUTOSAR Adaptive und EDF⁺

ISO 26262 ASIL-D fordert Nachweis der Schedulierbarkeit, beschränkte Blockierzeiten und temporale Isolation. EDF⁺ [40] erfüllt diese Anforderungen durch analytisch beschränkten Neuplanungs-Overhead ($E[R] = np/(1-p)$). Dies macht EDF⁺ zum aussichtsreichen Kandidaten für ASIL-D-Zertifizierung in AUTOSAR Adaptive Platform.

POSIX SCHED_DEADLINE

SCHED_DEADLINE implementiert seit Linux 3.14 exakt den CBS-Algorithmus (Kapitel 6) und ist der erste produktionsreife EDF-Scheduler im Linux-Kernel. Aktuelle Forschung erweitert ihn auf Mehrprozessoren (GEDF, partitioniertes EDF).

13.5.9 Zusammenfassung der Forschungsrichtungen

Tabelle 13.1: Forschungsrichtungen im Echtzeit-Scheduling

Richtung	Stand	Offen
Mehrprozessor	Pfair, LLREF, DP-Fair	Effizienz, Jitter
Mixed-Criticality	EDF-VD (2 Stufen)	$k > 2$ Stufen, Multi-core
ML im Scheduling	pWCET, Safe RL	Harte Garantien
Formale Verifikation	Isabelle, UPPAAL	Skalierung, n parametrisch
Hardware-Scheduler	FPGA-Prototypen	Kernel-Integration
Autonome Systeme	Elastic + CBS	ASIL-D Zertifizierung
Verteiltes EDF	TSN, PTP	Globale Schedulierbarkeit
EDF ⁺ / Subgraph	Formal bewiesen	Multicore, Zertifizierung

13.5.10 Schlusswort

Die Echtzeit-Scheduling-Theorie hat seit Liu & Layland (1973) enorme Fortschritte gemacht. Die hier behandelten Algorithmen – von RM über EDF bis zu EDF⁺ – bilden eine solide Grundlage, die täglich in realen Systemen eingesetzt wird.

Dennoch bestehen zentrale Herausforderungen: Die Lücke zwischen theoretisch optimalen Algorithmen (Pfair) und praktisch einsetzbaren Lösungen bleibt offen. Autonome Systeme, vernetzte Plattformen und KI-Anwendungen stellen die Echtzeit-Theorie vor fundamental neue Fragen.

Der Kernsatz bleibt die beständige Richtschnur:

**Wenn es einen machbaren Schedule gibt, findet EDF diesen sofort.
EDF ist scharf optimal.**

Alle Erweiterungen – EDF⁺, CBS, MCS EDF-VD – bauen auf dieser Grundwahrheit auf. Die Echtzeit-Scheduling-Theorie ist lebendig und offen – ein faszinierendes Forschungsfeld mit direkten Auswirkungen auf Sicherheit und Zuverlässigkeit technischer Systeme.

Literaturverzeichnis

- [1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3. Aufl., New York: Springer, 2011.
- [2] J. W. S. Liu, *Real-Time Systems*, Upper Saddle River, NJ: Prentice Hall, 2000.
- [3] A. Burns und A. J. Wellings, *Real-Time Systems and Programming Languages*, 4. Aufl., Harlow: Addison-Wesley, 2009.
- [4] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2. Aufl., New York: Springer, 2011.
- [5] P. A. Laplante, *Real-Time Systems Design and Analysis: Tools for the Practitioner*, 4. Aufl., Hoboken, NJ: IEEE Press/Wiley, 2011.
- [6] C. L. Liu und J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, Bd. 20, Nr. 1, S. 46–61, 1973.
- [7] W. A. Horn, "Some simple scheduling algorithms," *Naval Research Logistics Quarterly*, Bd. 21, Nr. 1, S. 177–185, 1974.
- [8] M. L. Dertouzos, "Control robotics: The procedural control of physical processes," in *Proc. IFIP Congress*, S. 807–813, 1974.
- [9] J. R. Jackson, "Scheduling a production line to minimize maximum tardiness," Research Report 43, Management Science Research Project, UCLA, 1955.
- [10] E. L. Lawler, "Optimal sequencing of a single machine subject to precedence constraints," *Management Science*, Bd. 19, Nr. 5, S. 544–546, 1973.
- [11] P. Bratley, M. Florian und P. Robillard, "On sequencing with earliest deadlines and latest completion times," *Journal of Computer and System Sciences*, Bd. 4, Nr. 5, S. 268–275, 1971.
- [12] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal of Applied Mathematics*, Bd. 17, Nr. 2, S. 416–429, 1969.
- [13] S. K. Dhall und C. L. Liu, "On a real-time scheduling problem," *Operations Research*, Bd. 26, Nr. 1, S. 127–140, 1978.
- [14] J. P. Lehoczky, L. Sha und Y. Ding, "The rate monotonic scheduling algorithm: Exact characterization and average case behavior," in *Proc. 10th IEEE Real-Time Systems Symposium (RTSS)*, S. 166–171, 1989.

- [15] J. Y.-T. Leung und J. Whitehead, "On the complexity of fixed-priority scheduling of periodic real-time tasks," *Performance Evaluation*, Bd. 2, Nr. 4, S. 237–250, 1982.
- [16] N. C. Audsley, A. Burns, M. Richardson, K. Tindell und A. J. Wellings, "Applying new scheduling theory to static priority pre-emptive scheduling," *Software Engineering Journal*, Bd. 8, Nr. 5, S. 284–292, 1993.
- [17] E. Bini, G. C. Buttazzo und G. M. Buttazzo, "A hyperbolic bound for the rate monotonic algorithm," in *Proc. 13th Euromicro Conference on Real-Time Systems*, S. 59–66, 2001.
- [18] M. Joseph und P. Pandya, "Finding response times in a real-time system," *The Computer Journal*, Bd. 29, Nr. 5, S. 390–395, 1986.
- [19] S. K. Baruah, L. E. Rosier und R. R. Howell, "Algorithms and complexity concerning the preemptive scheduling of periodic, real-time tasks on one processor," *Real-Time Systems*, Bd. 2, Nr. 4, S. 301–324, 1990.
- [20] H. Chetto, M. Silly und T. Bouchentouf, "Dynamic scheduling of real-time tasks under precedence constraints," *Real-Time Systems*, Bd. 2, Nr. 3, S. 181–194, 1990.
- [21] J. A. Stankovic, "Misconceptions about real-time computing: A serious problem for next-generation systems," *IEEE Computer*, Bd. 21, Nr. 10, S. 10–19, 1988.
- [22] B. Sprunt, L. Sha und J. P. Lehoczky, "Aperiodic task scheduling for hard-real-time systems," *Real-Time Systems*, Bd. 1, Nr. 1, S. 27–60, 1989.
- [23] J. K. Strosnider, J. P. Lehoczky und L. Sha, "The deferrable server algorithm for enhanced aperiodic responsiveness in hard real-time environments," *IEEE Transactions on Computers*, Bd. 44, Nr. 1, S. 73–91, 1995.
- [24] J. P. Lehoczky und S. Ramos-Thuel, "An optimal algorithm for scheduling soft-aperiodic tasks in fixed-priority preemptive systems," in *Proc. 13th IEEE RTSS*, S. 110–123, 1992.
- [25] M. Spuri und G. C. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, Bd. 10, Nr. 2, S. 179–210, 1996.
- [26] L. Abeni und G. C. Buttazzo, "Integrating multimedia applications in hard real-time systems," in *Proc. 19th IEEE RTSS*, S. 4–13, 1998.
- [27] T. S. Tia, Z. Deng, M. Shankar, M. Storch, J. Sun, L.-C. Wu und J. W. S. Liu, "Probabilistic performance guarantee for real-time tasks with varying computation times," in *Proc. Real-Time Technology and Applications Symp.*, S. 164–173, 1995.
- [28] L. Sha, R. Rajkumar und J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, Bd. 39, Nr. 9, S. 1175–1185, 1990.
- [29] T. P. Baker, "Stack-based scheduling of realtime processes," *Real-Time Systems*, Bd. 3, Nr. 1, S. 67–99, 1991.
- [30] M. I. Chen und K. J. Lin, "Dynamic priority ceilings: A concurrency control protocol for real-time systems," *Real-Time Systems*, Bd. 2, Nr. 4, S. 325–346, 1990.

- [31] Y. Wang und M. Saksena, “Scheduling fixed-priority tasks with preemption threshold,” in *Proc. 6th Intl. Conf. on Real-Time Computing Systems and Applications (RTCSA)*, S. 328–335, 1999.
- [32] M. Bertogna, O. Khani, M. Marinoni, F. Esposito und G. C. Buttazzo, “Optimal selection of preemption points to minimize preemption overhead,” in *Proc. 23rd Euromicro Conference on Real-Time Systems*, S. 217–227, 2011.
- [33] A. Cervin, *Integrated Control and Real-Time Scheduling*, Diss., Lunds Tekniska Högskola, 2003.
- [34] G. C. Buttazzo und L. Abeni, “Elastic scheduling for flexible workload management,” *IEEE Transactions on Computers*, Bd. 51, Nr. 3, S. 289–302, 2002.
- [35] S. K. Baruah, N. Cohen, C. G. Plaxton und D. A. Varvel, “Proportionate progress: A notion of fairness in resource allocation,” *Algorithmica*, Bd. 15, Nr. 6, S. 600–625, 1996.
- [36] J. H. Anderson, V. Bud und U. C. Devi, “An EDF-based scheduling algorithm for multiprocessor soft real-time systems,” in *Proc. 17th Euromicro Conference on Real-Time Systems*, S. 199–208, 2005.
- [37] S. Vestal, “Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance,” in *Proc. 28th IEEE RTSS*, S. 239–243, 2007.
- [38] S. Baruah, V. Bonifaci, G. D’Angelo, H. Li, A. Marchetti-Spaccamela, N. Megow und L. Stougie, “Scheduling real-time mixed-criticality jobs,” *IEEE Transactions on Computers*, Bd. 61, Nr. 8, S. 1140–1152, 2012.
- [39] C. Paulin-Mohring, “Scheduling-based real-time analysis,” in *Proc. 4th Intl. Workshop on Formal Methods for Industrial Critical Systems (FMICS)*, 2006.
- [40] S. Epp, “Optimaler Scheduling-Algorithmus EDF⁺: Prioritätsstrafe und stochastische Analyse bei 40 Tasks,” 2026.
- [41] AUTOSAR Consortium, *AUTOSAR Classic Platform – Specification of Operating System*, Release R23-11, 2023.
- [42] ISO, *ISO 26262: Road Vehicles – Functional Safety*, 2. Aufl., 2018.
- [43] IEEE und The Open Group, *POSIX.1-2017 (IEEE Std 1003.1): Base Specifications*, Issue 7, 2017.
- [44] IEEE, *IEEE Std 802.1Qbv-2015: Enhancements for Scheduled Traffic*, 2016.